TUNKU ABDUL RAHMAN UNIVERSITY OF MANAGEMENT AND TECHNOLOGY

FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY (FOCS)

ACADEMIC YEAR 2023/2024

BACS3013 DATA SCIENCE ASSIGNMENT

COURSE	**:**	**BACS3013 Data Science**
PROGRAMME	**:**	**Bachelor of Computer Science in Data Science**
SUBMISSION DEADLINE	**:**	**27 September 2024 (Friday, 11:59 pm)**
TOTAL MARK	**:**	**100%**
WEIGHTAGE TO FINAL MARK	**:**	**42%**
TUTORIAL GROUP	**:**	**RDS1S3 G3**

Assignments	**Marks**
TOTAL (100%)	

Team Members:

No.	Student Name	Student ID	Signature
1.	Low Qing Ying	23WMR14586	
2.	Wong Jin Xuan	23WMR14630	
3.	Dorcas Lim Yuan Yao	23WMR14535	
4.	Tan Yen Ping	23WMR14615	

BACS3013 Data Science

CLO1 - Apply appropriate data science concepts, statistics, and machine learning to make predictions on data.
CLO2 - Practice exploratory data analysis and visualization methods with the aid of appropriate data science tools.
CLO3 - Analyze relevant techniques, statistical methods, and machine learning algorithms for data analytics.

Criterion	Missing or Unacceptable (0-5 mark)	Poor (6-10 marks)	Good (11-15 marks)	Excellent (16-20 marks)	Marks
Data Preprocessing (CLO3)	The data preprocessing step is missing or very poor. Irrelevant, inaccurate, or inappropriate information.	The data preprocessing steps are not listed and described clearly.	The data preprocessing steps are relevant, offering details about the data science components.	The data preprocessing steps introduction are informative, succinct, and offer sufficiently specific details about the data science components.	
Descriptive Analytics and Exploratory Data Analytics (CLO2)	Analytical methods and results are not properly explained. The explanations are not aligned with the data science and business intelligence.	Analytical methods and results are explained. However, the explanations are confusing, incomplete or lacked relevance to the data science and business intelligence.	Analytical methods and results are explained. The explanations are appropriate and related to the data science and business intelligence.	Analytical methods and results are explained well. The explanations are clear, structured, appropriate and related to the data science and business intelligence.	
Graphing and Visualization (CLO2)	Graphing and visualization are omitted or not irrelevant.	Limited graphing and visualization are presented. The information is unclear or confusing.	Graphing and visualization are presented. The information is clear.	Graphing and visualization are presented systematically. The information is comprehensive and informative. All graphs and visualizations are linked to EDA and advanced analytics appropriately.	

Advanced Analytics and Discussion (CLO3)	Discussions are omitted or confusing. No or very little advanced analytics.	Little discussions are presented. Advanced analytics on the results are unclear or confusing.	Discussions of the results are presented. Advanced analytics on the results are presented.	The significance of the results of the work is discussed, sufficiently inclusive of the advanced analytics. Limitations and future improvements of the studies are identified.	
Model Selection and Report Structure (CLO1)	No justification on the model selection. The structure of the report is incomprehensible, irrelevant, or confusing. Transition from one to another is awkward.	Little justifications are presented on the model selection. The structure of the is weak. Transition from one to another is weak and sometimes difficult to understand.	Adequate justifications are presented on the model selection. The structure of the report is good. Transition from one to another is smooth to show the data science concepts.	Strong justifications are presented on the model selection. The structure of the report is excellent. Transition from one to another is smooth and organized to show the data science concepts.	

Table of Contents

Page Numbers

A. Business Understanding **1**
Data Mining Goals 2
Project Planning 3
Business Success Criteria 4
B. Data Understanding **5**
Initial Data Collection 5
Data Description 5
Data Exploration 6
Data Quality 14
C. Data Preparation **18**
Data Selection / Feature Selection 18
Data Cleaning 20
Construct Required Data 23
Data Aggregation 24
Drop Columns 25
Transforming Categorical Values 26
D. Modelling **28**
K-Means Clustering Modelling 28
Implementation of K-Means Clustering Modelling With Cosine Distance Metric 48
MiniBatch K-Means Clustering 54
Implementation of MiniBatch K-Means Clustering Modelling With Cosine Distance Metric 70
E. Evaluation **75**
Review of Process 75
Model Selection 77
Model Analysis 78
Evaluation of Results 84
Possible Actions and Decisions 87
F. Deployment **91**
G. Dashboard/Visualization/Presentation **94**
H. Conclusion **109**
Advantages of the model (KMeans) 109
Weaknesses / Limitations of the model 110
I. References **111**
J. Appendix **113**
Appendix A 113

A. Business Understanding

Dataset: Online Retail

Subject area: Business

Description:

This dataset is considered large, which consists of transactional data in a year for an online retail in the UK selling all kinds of gifts for a variety of occasions.

Primary Objective:

Use data-driven insights to retain customers and increase revenue and growth by understanding the customer base by identifying distinct customer segments based on purchasing behaviour. This will help to optimise inventory management, decision making for marketing strategies which then in turn enhance sales growth and customer retention.

Related Business Questions:

- Which customer segments contribute most to the sales, and how can we target them effectively?
- Are there differences in purchasing behaviour across different regions, what marketing strategies can be made?
- Which products category are the best sellers, and which underperform?
- Are there particular products or categories that are frequently purchased together, and how can this information inform cross-selling and upselling strategies?

Data Mining Goals

1. **Customer segmentation**

   Discover insights and identify specific segments of customers who purchased in the past in order to understand customer purchasing behaviour, frequency, high spenders, or regional purchasing trends which helps in business decisions in order to maximise the value of each customer to the business.

   Approach: Use clustering algorithms such as K-means to segment customers based on features like *CustomerID*, purchase frequency, product category, month of purchase, *Country*, *UnitPrice* and *Quantity* purchased.

2. **Product Category Analysis**

   Evaluate the performance of various products in terms of product type or category based on the frequency and quantity in the transactional data. This allows the retailer to identify best seller and less popular items categories to optimise inventory management and product offerings.

   Approach: Analyse sales data to determine patterns in product categories that contribute significantly to revenue, and use these insights to improve stock levels, promotional activities, and product assortment.

3. **Geographical and Seasonal Trends Analysis**

   Understand how purchasing behaviours vary by geographic region and seasonal trends to better align marketing strategies, inventory management during peak or low season, and promotional activities that could be done.

   Approach: Analyse customer purchase patterns across different countries (UK and international) and identify seasonal trends by examining monthly sales data. The information can help to detect peak purchasing periods, region-specific product preferences, and adjust marketing and inventory strategies accordingly.

Project Planning

Activity	Description	Tools and techniques
Data Understanding	Fully understand the structure and content of the dataset, find the key features and transaction trends. Load the dataset into a data analysis environment such as python for initial exploration and identify data issues such as missing values or outliers.	Python (Pandas, NumPy), Excel Visualisation tools: Python (Matplotlib, Seaborn)
Data Preparation	Prepare the data to make it "clean" and suitable for further modelling analysis. This involves the process of removing duplicate or irrelevant records, fixing structural errors, filtering unwanted outliers, handling missing data, transforming data as needed that suits the modelling algorithm and creating new features such as subtotal (*Quantity* x *UnitPrice*).	Data cleaning and transformation: Python (Pandas, NumPy) To identify outliers (Matplotlib, Seaborn)
Modelling	Implement clustering techniques for customer segmentation, grouping customers based on purchasing patterns. Tasks: ● Apply K-Means and MiniBatchKMeans to segment customers based on identified features ● Determine the best number of number of clusters with Elbow Method ● Interpret and label each customer segment based on the purchasing behaviour.	Python (Scikit-learn for clustering algorithm) Machine learning or statistical methods (e.g., K-Means Clustering and MiniBatchK-Means for segmentation) Visualisation tools: Python (Matplotlib, Seaborn)

Evaluation	Evaluate the clustering model's effectiveness in segmenting customers and identify whether the model meets the business objectives. (Clusters are distinct, clear and meaningful) Use plots / graphs to visualise the clusters and their characteristics.	Python (Scikit-learn for model evaluation metrics) Visualisation tools: Python (Matplotlib, Seaborn)
Deployment	Implement the actionable insights obtained from the clustering analysis in a real-world business environment to make actionable business decisions. Develop strategies for marketing, inventory management, promotion and sales based on customer segments.	Online research
Data Visualization and Presentation	Presentation of the clustering results and insights by creating dashboards, reports findings, charts or tables that summarise the customer segments.	Python (Dash, Plotly)

Business Success Criteria

- Increased sales and revenue by 15% in the next quarter year.
- Increase the number of repeat purchases from customers by 20% within six months.
- Enhance customer satisfaction by 15% through feedback and survey by fulfilling the customer needs and preferences.

B. Data Understanding

Initial Data Collection

The data source was provided by the lecturer in the assignment question.

- Name of data source: Online Retail Dataset
- Location: UCI Machine Learning Repository (*UCI Machine Learning Repository*, n.d.)
- Description: The dataset was downloaded as an Excel file 'Online Retail.xlsx' from the repository and extracted from the ZIP file. It contains the transactional data from a UK-based online retailer from December 2010 to December 2011.
- Tool for data understanding: Data loading and exploration using Python's 'pandas' library via Jupyter Notebook
- Problems encountered: Large file size and slow loading when reading data via pandas
  Solution: The excel file was converted into CSV format which is more efficient in handling the data and easier to work with.

Data Description

Format: Excel (.xlsx)

```
#Understand the data type of each feature
retail.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
 #   Column        Non-Null Count   Dtype
---  ------        --------------   -----
 0   InvoiceNo     541909 non-null  object
 1   StockCode     541909 non-null  object
 2   Description   540455 non-null  object
 3   Quantity      541909 non-null  int64
 4   InvoiceMonth  541909 non-null  int64
 5   UnitPrice     541909 non-null  float64
 6   CustomerID    406829 non-null  float64
 7   Country       541909 non-null  object
dtypes: float64(2), int64(2), object(4)
memory usage: 33.1+ MB
```

Number of data fields: 8

Number of records: 541909

8 key attributes:

- InvoiceNo (Object, Categorical) – Identifier for each transaction
- StockCode (Object, Categorical) – Identifier for each product
- Description (Object, Categorical) – Product name
- Quantity (Integer, Numeric) – Number of each units purchased per transaction
- InvoiceMonth (Integer, Numeric) – Month value of date of transaction
- UnitPrice (Float, Numeric) – Price per unit of the product
- CustomerID (Float, Categorical) – Identifier for each customer
- Country (Object, Categorical) – Customer's country

Data Exploration

<u>Basic statistic:</u>

Overview of the statistical calculations of "Quantity", "UnitPrice", and "CustomerID" (no interpretative value since the numerical value is not meaningful, CustomerID is categorical as an unique identifier)

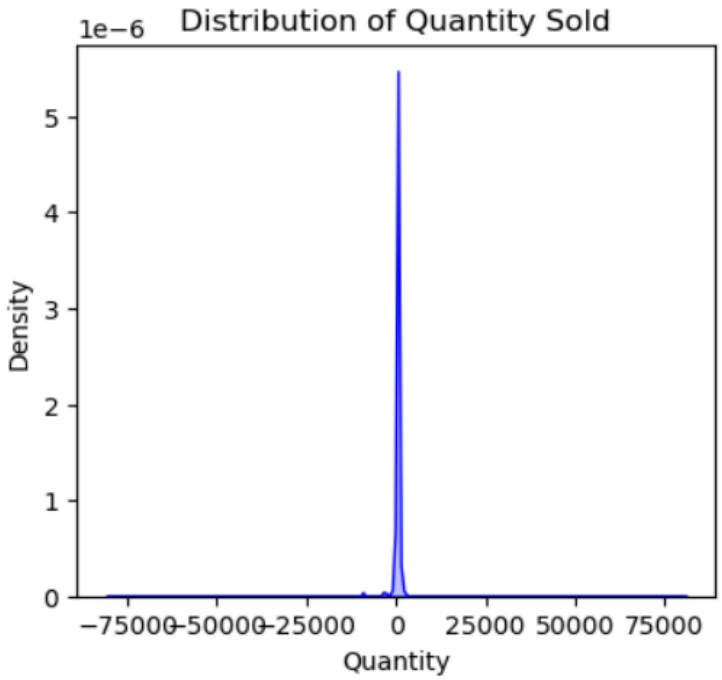
```
#Basic statistics
retail.describe()
```

[6]:

	Quantity	InvoiceMonth	UnitPrice	CustomerID
count	541909.000000	541909.000000	541909.000000	406829.000000
mean	9.552250	7.553128	4.611114	15287.690570
std	218.081158	3.509055	96.759853	1713.600303
min	-80995.000000	1.000000	-11062.060000	12346.000000
25%	1.000000	5.000000	1.250000	13953.000000
50%	3.000000	8.000000	2.080000	15152.000000
75%	10.000000	11.000000	4.130000	16791.000000
max	80995.000000	12.000000	38970.000000	18287.000000

<u>Explore data distribution:</u>

1. **Kernel Density Estimate (KDE) Plot for *"Quantity"***

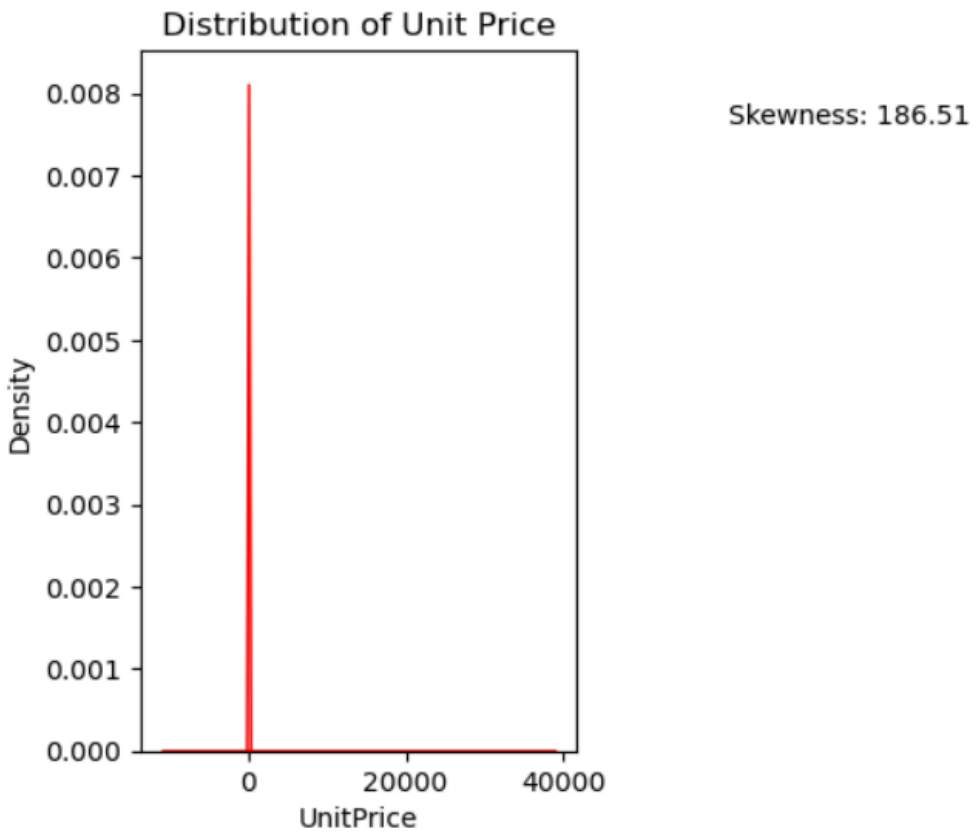


Skewness: -0.26

The plot shows a very wide range of x-axis from around -75,000 to 75,000 indicates that there might exist some extreme values (outliers) in the *'Quantity'* data. The distribution of majority data is highly concentrated around 0 (peak sharply around 0), with most transactions involving small quantities and very few transactions involving large quantities.

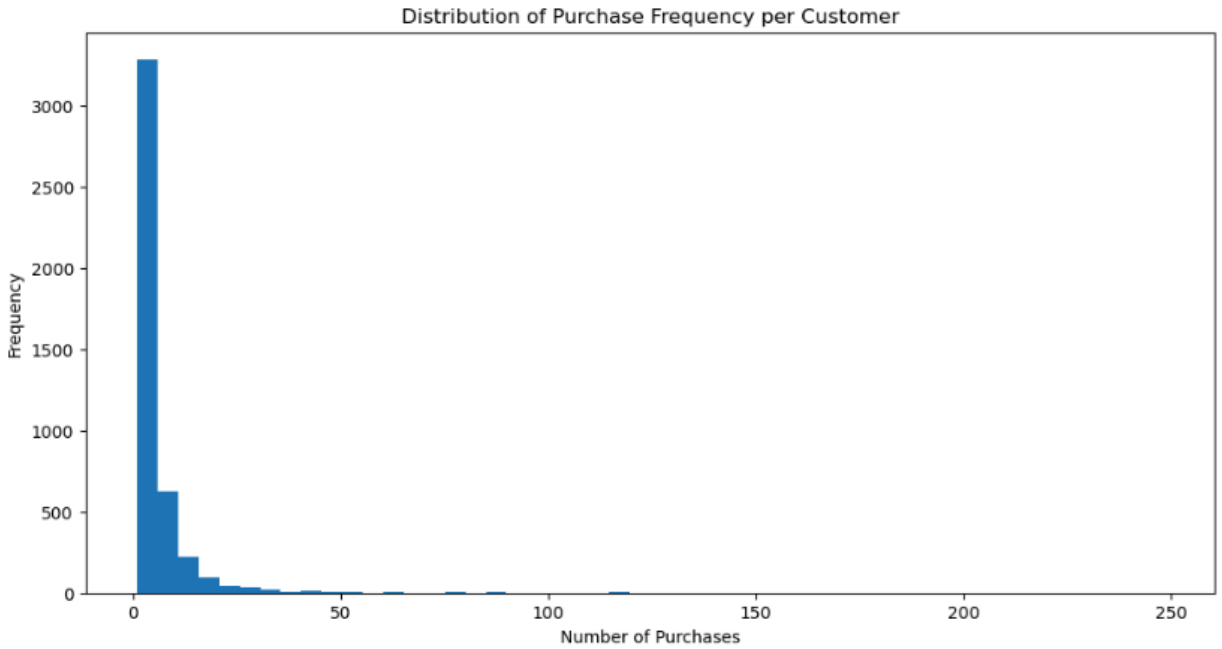
A skewness: -0.26 indicates that the distribution of 'Quantity' is slightly negatively skewed, with a slightly longer tail on the left side. Since the skewness value is close to 0, the data is almost symmetric, with most quantities close to median value (3.0).

2. **Kernel Density Estimate (KDE) Plot for *"UnitPrice"***



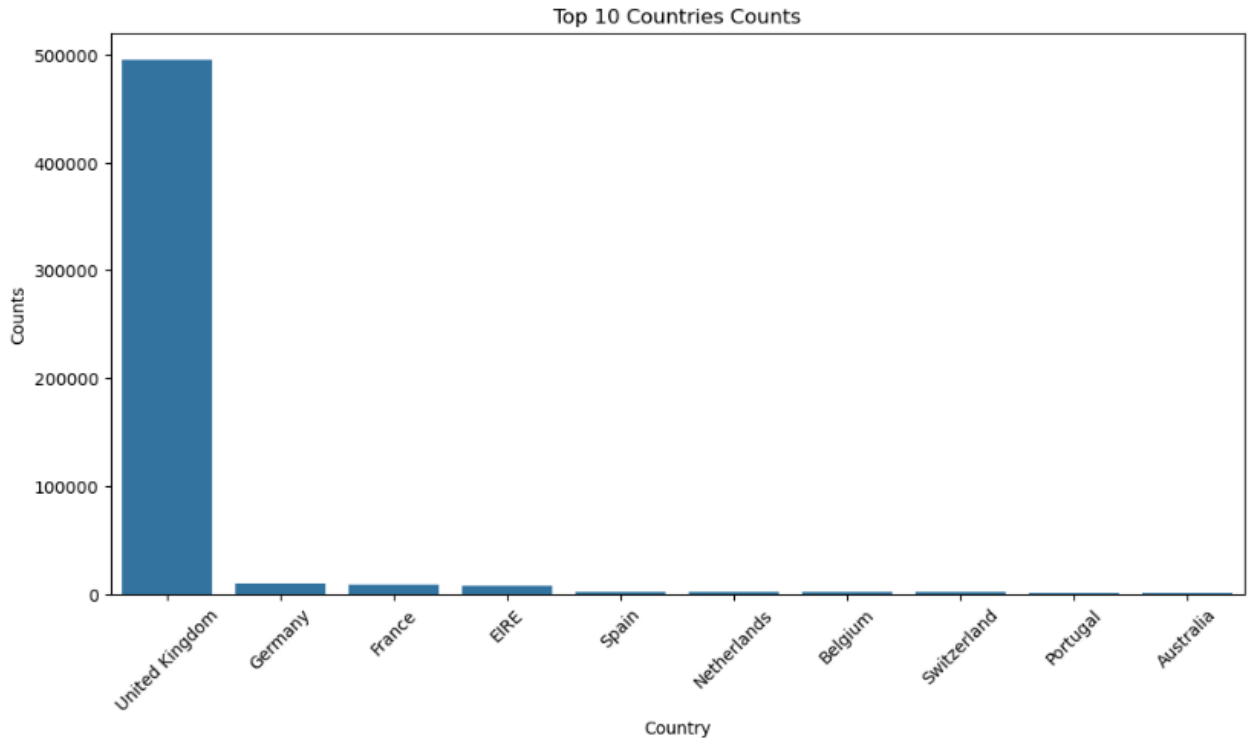
The sharp peak around 0 shows that the majority data points are concentrated near together with a mean value of 4.61, which is relatively inexpensive. The skewness of 186.51 is extremely high, which implies that the distribution has a very long tail extending to the right. There may be a few extremely large value outliers in the dataset, far greater than the majority, such as luxury or luxury items.

3. Distribution of purchase frequency per customer



The distribution is positively skewed, showing that there are only few customers with higher purchases frequency. There is a sharp peak near the lower end of the x-axis, which indicates that the majority of customers have a low number of purchases (<30). Most customers are occasional shoppers making only a few purchases as the company mainly sells all-occasion gifts.

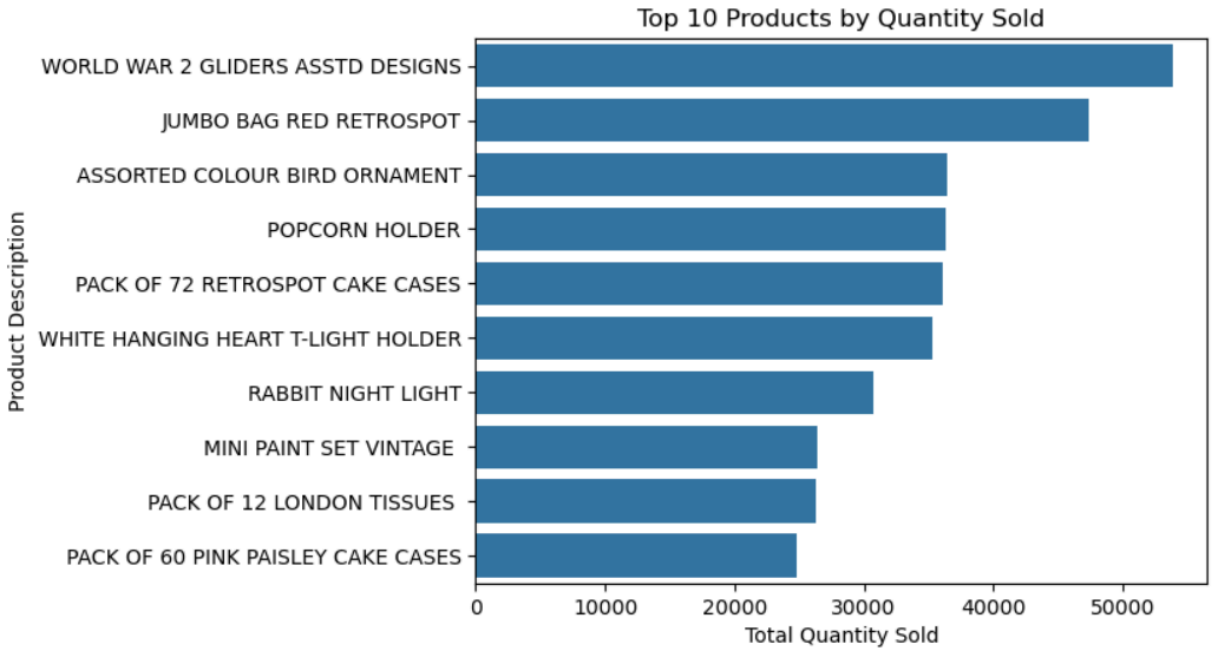
4. Top 10 Countries




```
Lowest Count:'Saudi Arabia', 10 transactions
```

From the bar chart, it shows that the UK was the main country where the target customers came from. More than half of the total purchases came from the UK, being the dominant market in this dataset while Saudi Arabia holds the smallest percentage of transaction counts. The top 10 countries were mostly in the European region, well economically-developed countries with higher customer purchasing power.

5. Top 10 Products by Quantity Sold

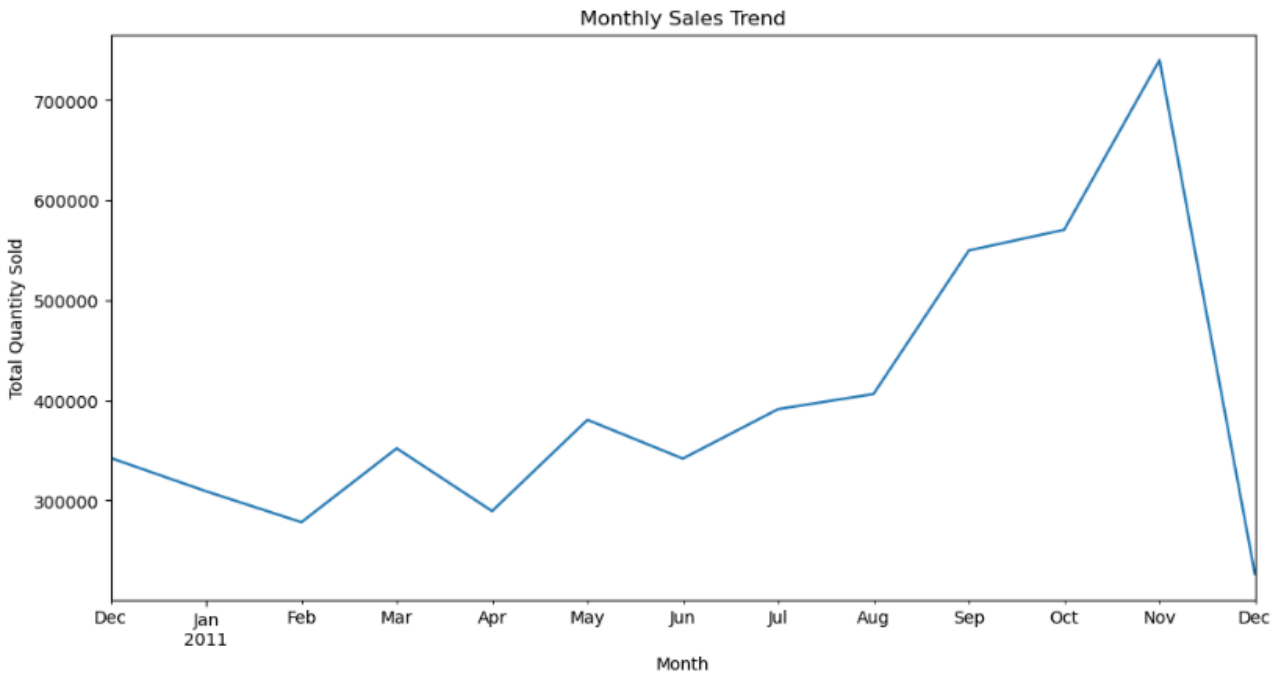



```
Product with Lowest Quantity Sold: 'printing smudges/thrown away', Quantity Sold: -19200
```

The top 10 products indicate that they were popular and high in demand, with significant quantities sold. The best seller was "WORLD WAR 2 GLIDERS ASSTD DESIGNS". Most of the top-selling products were decorative and gift items (bird ornament, night light, paint). On the other hand, the lowest-selling product is "printing smudges/thrown away" with 19200 items for refund, which may be due to faulty issues.

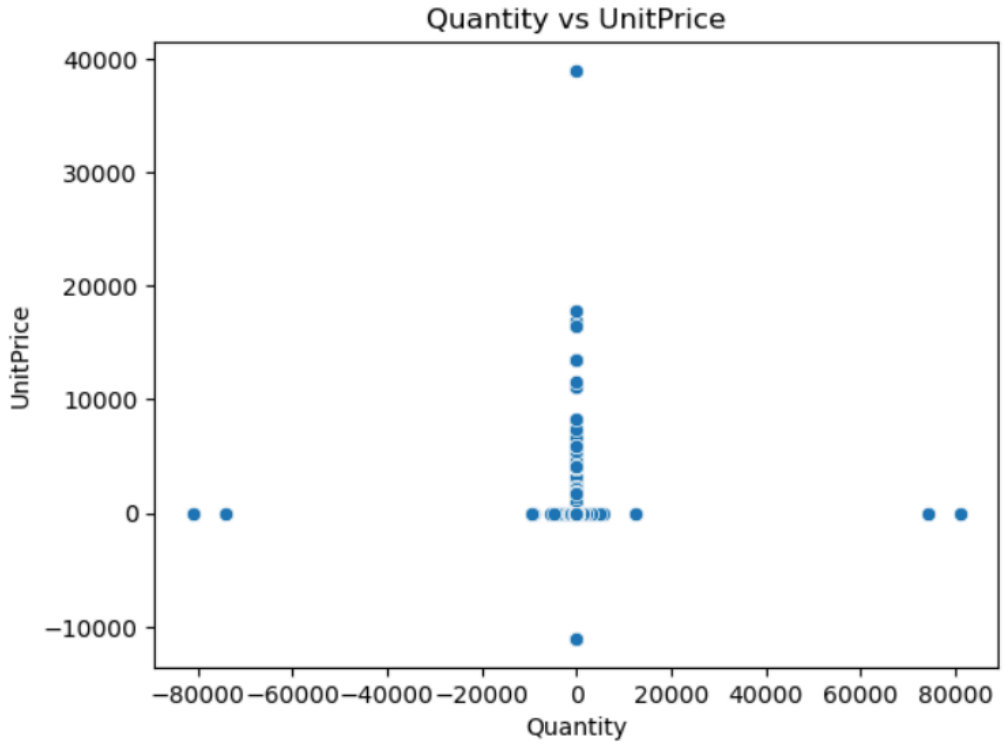
<u>Explore data relationship:</u>

1. **Monthly Sales Trend over a Year (Relationship of *"Quantity"* and *"InvoiceDate"*)**



Overall, the graph shows an upward trend in sales over the year (from DEC 2010 to NOV 2011), with sales starting lower at the beginning of the year and becoming higher at the end of the year. Lowest peak would be February while the highest peak leads to over 70,0000 in November due to some possible seasonal factors such as holiday seasons, end-of-year promotions for online retail such as "11.11", or upcoming Christmas gifts.

2. Quantity vs. Unit Price



These are the only two numerical features in the dataset. From the scatter plot, the data points are concentrated around zero for quantity while the unit price is also low. This shows that most transactions involve a low quantity of items with a price that is not relatively expensive. It seems that there is not much association and relationship between each other. However, there are a few extreme outliers that need to be handled. The presence of quite a number of negative quantities indicates the returns or refunds of items.

Data Quality

<u>Missing Values</u>

```
retail.isna().sum()
```

```
InvoiceNo           0
StockCode           0
Description      1454
Quantity            0
InvoiceDate         0
UnitPrice           0
CustomerID     135080
Country             0
dtype: int64
```

There are 1454 missing values in "Description" and 135080 missing values in "CustomerID".

Proposed Solution:

- **"*Description*" missing values**

  Delete: Since there are a total of 541909 records available in the dataset, the proportion of missing values (0.27%) is very small, direct deleting these records will have minimal impact on overall dataset while maintaining data quality.

- **"*CustomerID*" missing values**

  Keep separate category: While "*CustomerID*" has 135080 missing values, with a proportion of 24.92%, it is not appropriate to delete these records as it may significantly lose nearly a quarter of the dataset. "CustomerID" is categorical data in which imputing with mean or median might not make sense. On the other hand, mode imputation for huge amounts of records may lead to unrealistic repetition of a single customer. Hence, the suggested solution is to impute these missing values with a distinct category of "Unknown" (require conversion data type from float to object).

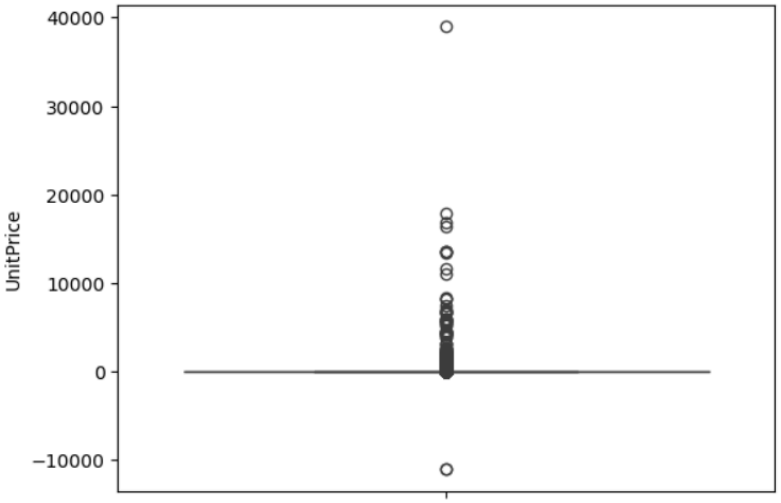
<u>Duplication</u>

Due to the nature of the dataset for retail transactional data where an invoice contains multiple products, a product can appear in multiple invoice and a customer may have repeated purchase, this kind of duplication will not be considered as a data quality issue.

Outliers

Identify outliers through **boxplot** and computation of **interquartile range**.

1. **"*UnitPrice*" Outliers**




```
Q1: 1.25
Q3: 4.13
IQR: 2.88
Median: 2.08
Lower Whisker: -3.0700000000000003
Upper Whisker: 8.45
Number of Outliers: 39627
Outliers: 16         9.95
45        18.00
65        10.95
141       27.50
151       14.95
          ...
541768    18.00
541786    11.95
541831     8.50
541849     9.95
541892     8.95
Name: UnitPrice, Length: 39627, dtype: float64
```

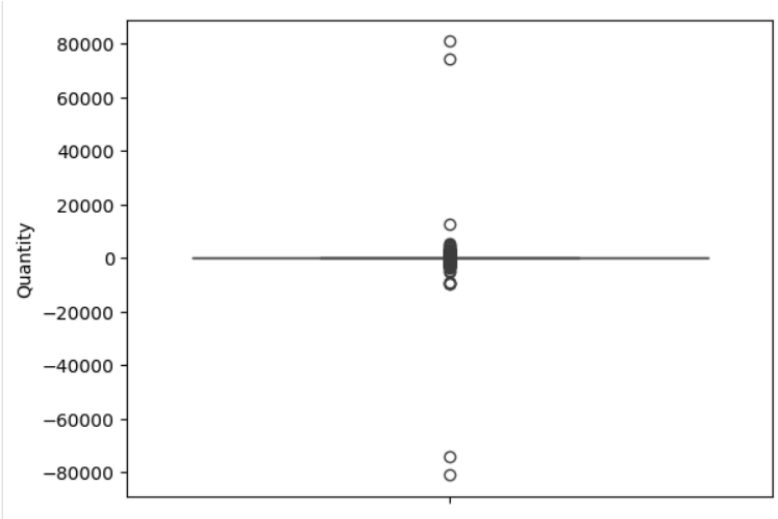
From the boxplot, there are a significant number of extreme outliers. The interquartile range is short, indicating that most prices are clustered closely around the lower and median values. Overall, the values that were out of the range of -3.07 to 8.45 are considered outliers, with the number of outliers as much as 39627.

However, it is not reasonable to have negative values in the context of "*UnitPrice*". Hence, we set the lower whisker to 0 and replace all negative values with zero to ensure that all values in "*UnitPrice*" are non-negative.

```
Q1: 1.25
Q3: 4.13
IQR: 2.88
Median: 2.08
Lower Whisker: 0
Upper Whisker: 8.45
Number of Outliers: 39625
Outliers: 16         9.95
45        18.00
65        10.95
141       27.50
151       14.95
          ...
541768    18.00
541786    11.95
541831     8.50
541849     9.95
541892     8.95
```

The new range would be 0 to 8.45 with 39625 number of outliers.

2. "*Quantity*" Outliers




```
Q1: 1.0
Q3: 10.0
IQR: 9.0
Median: 3.0
Lower Whisker: -12.5
Upper Whisker: 23.5
Number of Outliers: 58619
Outliers: 9         32
26        24
27        24
30        24
31        48
          ..
541876    24
541882    24
541883    25
541891    24
541892    24
```

The boxplot of *"Quantity"* shows a similar pattern as for the boxplot for *"UnitPrice"*. The interquartile range is short, showing that the majority of the data are concentrated around 1 to 10. Overall, the values that were out of the range of -12.5 to 23.5 are considered outliers, with the number of outliers as much as 58619.

Proposed solution: Capping outliers would be suitable as it retains the outlier transactions within a reasonable range and avoids distortion or loss of potential important data ("Outlier Detection and Treatment: Methods for Cleaner Data," 2024). This is because it is possible that extreme transactions came from large orders or returns from wholesalers.

Data inconsistency

```
       InvoiceNo StockCode      Description  Quantity  InvoiceMonth  \
7313      537032     21275                ?       -30            12
21518     538090     20956                ?      -723            12
38261     539494     21479                ?       752            12
43662     540100     22837                ?      -106             1
50806     540558     21258                ?       -29             1
...          ...       ...              ...       ...           ...
514649    579734    84859C             wet?      -153            11
514735    579742     85204           lost??     -1131            11
524370    580547     21201              ???      -390            12
535324    581201     22217         damages?      -155            12
535327    581204     85104  ????damages????      -355            12

        UnitPrice  CustomerID         Country
7313          0.0         NaN  United Kingdom
21518         0.0         NaN  United Kingdom
38261         0.0         NaN  United Kingdom
43662         0.0         NaN  United Kingdom
50806         0.0         NaN  United Kingdom
...           ...         ...             ...
514649        0.0         NaN  United Kingdom
514735        0.0         NaN  United Kingdom
524370        0.0         NaN  United Kingdom
535324        0.0         NaN  United Kingdom
535327        0.0         NaN  United Kingdom

[100 rows x 8 columns]
```

There are approximately 100 records that are ambiguous in the "*Description*" and normally contain "?" or the word "wrong" in it.

Proposed Solution: Impute these values with "Unknown / Issue Reported".

C. Data Preparation

Data Selection / Feature Selection

For customer segmentation and sales analysis:

- ***InvoiceNo***

  A unique identifier for each transaction in the dataset.

  Exclusion: This attribute is excluded because it does not provide meaningful information for customer segmentation or sales analysis. It serves as a transactional identifier and cannot be used to derive customer behaviour or trends.

- ***StockCode***

  A unique identifier for each product sold by the retailer.

  Exclusion: Similar to invoice number, product identification does not give any meaningful value to for customer segmentation and analysis or sales trends. Besides that, the retailer has too many types of products, which may be sophisticated for the machine learning process to handle a significant amount of categorical values.

- ***Description***

  A text description of the product or product name.

  Exclusion: Similar to stock code, it can be excluded as the segmentation because it has too many different types of product values. Alternatively, we categorised the products into several categories based on the description to make it more interpretable.

- ***Category***

  A derived feature based on the product description, categorising products into groups (e.g. Christmas items, Decorative items, Kitchenware, etc.)

  Inclusion: Categorising products into broader groups helps in understanding customer preferences for specific product types and their buying behaviour across categories. It also aids in analysing the types of products that contribute the most to sales and customer segmentation based on product preferences.

- ***Quantity***

  Number of unit(s) of product bought in a single transaction.

  Inclusion: This is a critical feature to understand the customer purchasing behaviour and help in segmenting customers into bulk buyers or occasional buyers.

- ***InvoiceMonth***

  Date of transaction.

  Inclusion: Invoice month derived from the invoice date is crucial for time-based analysis and seasonal trends as well as determining customer purchasing timeline.

- ***UnitPrice***

  Price of product per unit.

  Inclusion: It is necessary to have unit price for customer segmentation and sales analysis because it helps to identify the value of transactions and may affect the customer spending behaviour, such as luxury buyers and normal spenders.

- ***TotalPrice***

  The total price of each transaction, calculated as *Quantity x UnitPrice*

  Inclusion: It helps to identify the high-value transactions and customer segments based on spending levels.

- ***CustomerID***

  A unique identifier for each customer of the retailer.

  Exclusion: The main function of CustomerID is to identify customers, which does not contribute much to the behaviour analysis directly. However, it is later used to assign clusters of segments back to specific customers after the clustering process.

- ***Country***

  Country of the customer.

  Inclusion: It may reveal customer preferences over different geographical areas and will be useful for segmenting customers by location.

Data Cleaning

1. "*Description*" missing values

```
#Handling missing value for "Description"
missing_desc = retail['Description'].isnull().sum()
print(f"Total Missing Description: {missing_desc}")

retail.dropna(subset=['Description'], inplace=True)

missing_desc = retail['Description'].isnull().sum()
print(f"Total Missing Description after data cleaning: {missing_desc}")
```

```
Total Missing Description: 1454
Total Missing Description after data cleaning: 0
```

As mentioned earlier, since the proportion of missing values for "*Description*" is low (0.27%), deleting the rows of these records does not affect much on the overall dataset. Instead, it helps to eliminate "garbage values" that might not be useful and improves the data quality, ensuring the completeness of data and its reliability. This approach is chosen as it simplifies the data cleaning process, imputation techniques are not applicable for "Description". For example, replacing it with mode could misrepresent the actual data distribution.

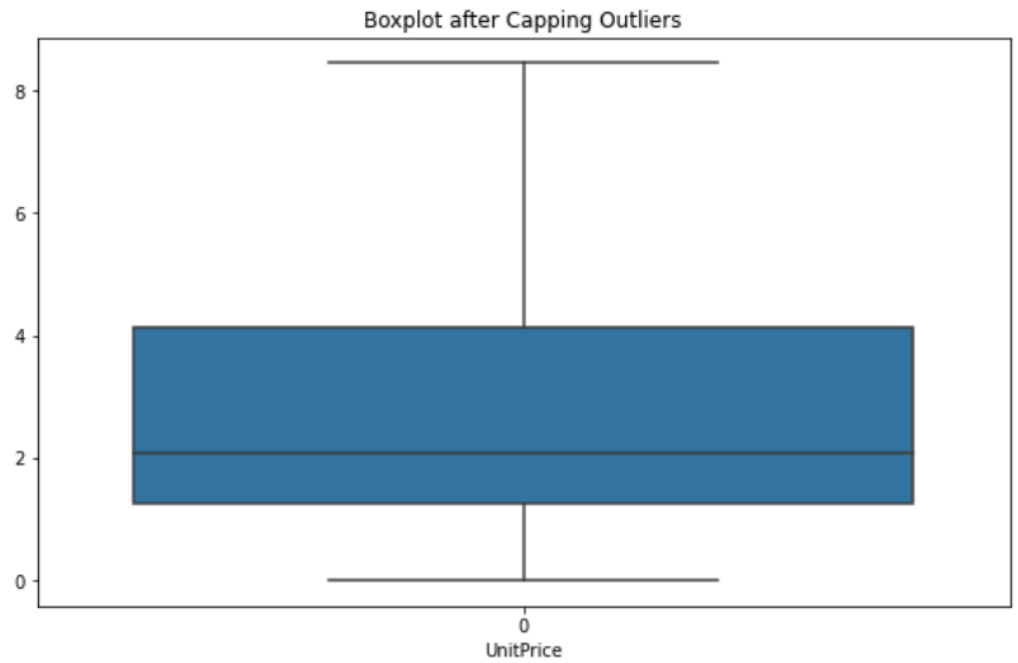
2. "*CustomerID*" missing values

```
#Handling missing value for "CustomerID"

missing_cust = retail['CustomerID'].isnull().sum()
print(f"Total Missing CustomerID: {missing_cust}")

retail['CustomerID'] = retail['CustomerID'].fillna('Unknown')
# Convert CustomerID attributes value to string(object) as categorical value
retail['CustomerID'] = retail['CustomerID'].apply(lambda x: str(int(float(x))) if x != 'Unknown' else x)

missing_cust = retail['CustomerID'].isnull().sum()
print(f"Total Missing CustomerID after data cleaning: {missing_cust}")
```

```
Total Missing CustomerID: 133626
Total Missing CustomerID after data cleaning: 0
```

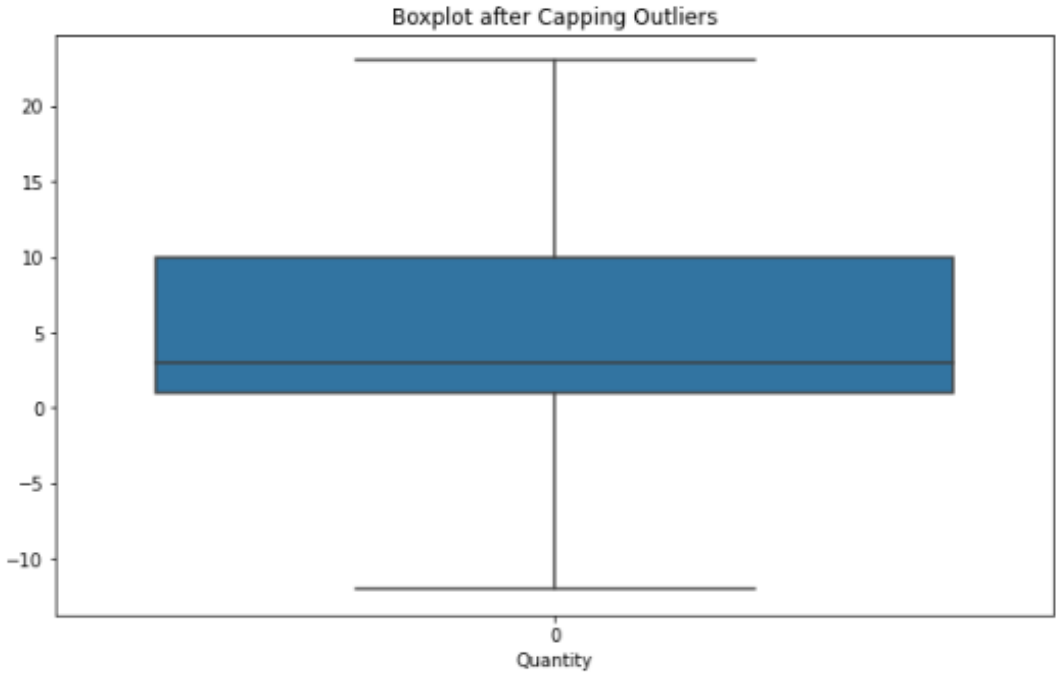
There is a large proportion of missing values for "*CustomerID*" (24.92%). Hence, we cannot directly delete these records as it may contain many important information for modelling. Although the data type is numerical, it functions as a categorical identifier in which imputation of statistical value may not be meaningful. Imputing mode value for "*CustomerID*" may cause the same customers to have over more than 10k of records, which is not logical. Therefore, imputing these missing values with "Unknown" preserves the data integrity and provides clear indication of missing values which may be crucial for further analysis. The data type of "*CustomerID*" is then converted from float to object as categorical value.

3. "*UnitPrice*" Outliers



As compared to the previous box plot when the outliers had not yet been handled, the box plot after capping the outliers appears clean and nice. The original data may have included negative values for "UnitPrice" which is unreasonable from the context of pricing. There are also extreme outliers that appear as separate points far from the clusters of centralised data which skew the overall distribution before capping. By using this approach, we get a clearer view of the main distribution of "*UnitPrice*", making it easier for interpretation and analysis. Overall, the price per unit of product has a relatively low price range.

4. **"*Quantity*" Outliers**



Similar to above, we get to see a clearer view of the key statistics. Before this, there existed extreme values in "Quantity" which may represent unusually large orders / refunds or errors in data entry. This approach helps us to focus on the typical range of quantities and prevent the outliers from distorting the distribution. Overall, the quantity of products per transaction concentrates from 0 to 10 although there are also some returns of products.

5. **"*Description*" Ambiguous Data**

The ambiguous description found in the dataset normally contains '?' or the word "wrong" in the string. Hence, we use some regex techniques to find those records and replace it with "Unknown / Issue Reported". This effort helps to improve the data consistency and make the dataset more suitable for analysis by providing clarity for further analysis.

Construct Required Data

1. TotalPrice

	InvoiceNo	StockCode	Description	Quantity	InvoiceMonth	UnitPrice	CustomerID	Country	TotalPrice
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12	2.55	17850	United Kingdom	15.30
1	536365	71053	WHITE METAL LANTERN	6	12	3.39	17850	United Kingdom	20.34
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12	2.75	17850	United Kingdom	22.00
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12	3.39	17850	United Kingdom	20.34
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12	3.39	17850	United Kingdom	20.34

The "*TotalPrice*" feature is derived by multiplying the "*Quantity*" of items purchased by the "*UnitPrice*", which represents the total amount of payment for the transactional items. This feature directly tells us how much the customer is spending for the product and would be useful for clustering algorithms to distinguish the customer spending behaviour and their purchasing power.

2. InvoiceMonth

The "*InvoiceMonth*" is derived from the "*InvoiceDate*" which can help us to get the seasonal trends information. It is being extracted using Microsoft Excel, the "*InvoiceDate*" is being replaced with it before loading the dataset. Sometimes, the customer spending habits can be determined based on seasonal events or months. For example, some customers would likely buy things during holiday season or sales promotion season while some purchase on a monthly basis.

3. Category

```python
unique_values = retail['StockCode'].nunique()
print(f"There are {unique_values} different values in the column.")
print(retail['StockCode'].head(100))
```

```
There are 3958 different values in the column.
0     85123A
1      71053
2     84406B
3     84029G
4     84029E
       ...
95     22352
96     21212
97     21975
98     21977
99     84991
Name: StockCode, Length: 100, dtype: object
```

```python
unique_values = retail['Description'].nunique()
print(f"There are {unique_values} different values in the column.")
print(retail['Description'].head(100))
```

```
There are 4218 different values in the column.
0      WHITE HANGING HEART T-LIGHT HOLDER
1                    WHITE METAL LANTERN
2          CREAM CUPID HEARTS COAT HANGER
3     KNITTED UNION FLAG HOT WATER BOTTLE
4          RED WOOLLY HOTTIE WHITE HEART.
                         ...
95      LUNCH BOX WITH CUTLERY RETROSPOT
96        PACK OF 72 RETROSPOT CAKE CASES
97         PACK OF 60 DINOSAUR CAKE CASES
98     PACK OF 60 PINK PAISLEY CAKE CASES
99            60 TEATIME FAIRY CAKE CASES
Name: Description, Length: 100, dtype: object
```

It is found that there are 3958 different values of "*StockCode*" and 4218 different values of "*Description*"in the dataset. This may not be efficient for doing clustering algorithms, like K-means because it works by grouping similar data points together based on their features. If the categorical variable has high cardinality, it makes the clustering process inefficient and less meaningful.

Hence, we consider grouping the products into broader categories based on their characteristics from "*Description*".

	Description	Category
0	WHITE HANGING HEART T-LIGHT HOLDER	Decorative Items
1	WHITE METAL LANTERN	Decorative Items
2	CREAM CUPID HEARTS COAT HANGER	Decorative Items
3	KNITTED UNION FLAG HOT WATER BOTTLE	Household Items
4	RED WOOLLY HOTTIE WHITE HEART.	Other
...	...	...
541904	PACK OF 20 SPACEBOY NAPKINS	Other
541905	CHILDREN'S APRON DOLLY GIRL	Kitchenware
541906	CHILDRENS CUTLERY DOLLY GIRL	Kitchenware
541907	CHILDRENS CUTLERY CIRCUS PARADE	Kitchenware
541908	BAKING SET 9 PIECE RETROSPOT	Kitchenware

We categorise the products based on the keywords recognised from the "*Description*" attribute. The above is the sample data with the newly derived feature "*Category*".

Data Aggregation

	CustomerID	TotalQuantity	TransactionNum	AvgPurchaseAmount	TotalSpent	\
0	12346	11	2	5.720000	11.44	
1	12347	2105	7	21.052912	3831.63	
2	12348	627	4	16.252581	503.83	
3	12349	600	1	18.541370	1353.52	
4	12350	196	1	17.764706	302.00	

	PercentReturned
0	50.0
1	0.0
2	0.0
3	0.0
4	0.0

Aggregation process involves computing new values from several records (Smart Vision Europe, 2020). In this case, we aggregate the purchasing data by customer, creating features that help to summarise the customer's behaviour.

"***TotalQuantity***" – Sum of the "*Quantity*" of items purchased by each customer

"***TransactionNum***" – Count of unique "*InvoiceNo*" by each customer, in other words, total transactions of invoice made by customer

"***AvgPurchaseAmount***" – Mean of the "*TotalPrice*", the average spending per invoice by the customer

"***TotalSpent***" – Sum of the "*TotalPrice*" by each customer

"***PercentReturned***" – The percentage of the returned transactions ("*Quantity*" with negative value) out of the total.

Drop Columns

1. **InvoiceNo**

	StockCode	Description	Quantity	InvoiceMonth	UnitPrice	CustomerID	Country	TotalPrice
0	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12	2.55	17850	United Kingdom	15.30
1	71053	WHITE METAL LANTERN	6	12	3.39	17850	United Kingdom	20.34

After doing the data aggregation to calculate data such as *TransactionNum*, the *InvoiceNo* does not provide any other meaningful information for clustering or for further analysis. It is simply a unique identifier for each transaction.

2. **StockCode & Description**

Both of them are excluded from the feature selection for modelling (still remained in the original online retail dataset) because they are identifiers or text descriptions that do not directly be meaningful for clustering, instead they add complexity due to the large number of unique values.

3. **CustomerID**

CustomerID is used for identifying individual customers and is not directly useful for clustering analysis. It is excluded from the features selection for modelling (still remained in the original online retail datasets). It serves as an identifier rather than a feature that contributes to understanding purchasing behaviour or trends.

Standardising and Scaling Data

Scaling data is an important step for clustering models. Scaling ensures that the features contribute equally to the distance calculations used in clustering algorithms and prevent biassed clusters (Nailman, 2024). The features that would possibly be used for the clustering modelling are scaled, which includes "*TotalQuantity*", *"Quantity", "InvoiceMonth" and "UnitPrice"* from the selected features dataset and the "*TransactionNum*", "*AvgPurchaseAmount*", "*TotalSpent*", and "*PercentReturned*" from the aggregated data. Binary indicators for product categories (e.g., "*Category_Christmas Items", "Category_Decorative Items"*,

etc.) and "*Country_United Kingdom"* are already binary and do not require scaling but are included to capture categorical information.

Transforming Categorical Values

1. **Country**

   This feature is essential because it helps to identify the customer behaviour based on geographical location. Customers of different countries may have different purchasing power based on their economical standards or other factors. However, around 90% of the records were "United Kingdom" as the retail store was UK-based. Hence, we decided to simplify the feature "Country" into binary classification of the UK and other countries. It might reveal different purchasing patterns between domestic and international customers. After that, we will apply one-hot encoding so that it appears numerically, which is important for the clustering algorithms (Nailman, 2024).

```
Country_United Kingdom
                   1.0
                   1.0
                   1.0
                   1.0
                   1.0
```

   This is the dataset after the conversion. "United Kingdom" will now be represented as a numerical value of 1 while the other countries will be 0.

2. Category

```
   Category_Christmas Items  Category_Decorative Items  Category_Easter Items  \
0                     False                       True                  False
1                     False                       True                  False
2                     False                       True                  False
3                     False                      False                  False
4                     False                      False                  False

   Category_Fashion Acessories  Category_Gardening  Category_Gift Items  \
0                        False               False                False
1                        False               False                False
2                        False               False                False
3                        False               False                False
4                        False               False                False

   Category_Household Items  Category_Kitchenware  Category_Other  \
0                     False                 False           False
1                     False                 False           False
2                     False                 False           False
3                      True                 False           False
4                     False                 False            True

   Category_Stationery  Category_Toys
0                False          False
1                False          False
2                False          False
3                False          False
4                False          False
```

This attribute appears as categorical value of different product categories like Decorative Items, Kitchenware, Household Items, and etc. Most machine learning algorithms, including clustering algorithms like K-means, require numerical inputs. One-hot encoding is applied to transform categorical values into a numerical / boolean format that these algorithms can process.

D. Modelling

Selection of Modelling Technique

In this case of online retail dataset, the primary goal is to segment customers based on their purchasing behaviour to identify distinct customer segments, patterns and insights. Hence, we have selected unsupervised learning, particularly K-Means clustering modelling and MiniBatchKMeans clustering modelling as the actual modelling technique that will be used.

The online retail dataset contains customer purchasing information such as transaction history, product stock code, quantities, unit price, invoice date etc. However, it does not have a predefined target variable (labelled data) that indicates customer segments or behaviours such as "high spender", "low engagement" etc. to train a supervised model. Since we don't have this, unsupervised learning is appropriate.

K-Means Clustering Modelling

K-Means clustering modelling is the most commonly used clustering method (McGregor, 2020). It also is a popular unsupervised machine learning algorithm that is used for partitioning a datasets into a pre-defined number of clusters. In this case, it helps us to group similar customers based on their purchasing behaviours and product trading, which is critical in business context to understand their customer segments.

Modelling Assumptions

- **Numeric data** - K-Means requires numerical data, so attributes that are categorical like Country, or Categorised Description were transformed using encoding methods, specifically one-hot encoding.
- **Equal variance** - K-means assumes that the variance of each cluster is the same, which means that the distribution of data points around the cluster centre is almost the same for each cluster (GeeksforGeeks, 2023). To make sure that every feature contributes equally to the distance computation, the features were scaled using StandardScaler.
- **Cluster Shape** - K-means assumes that the clusters are spherical and isotropic and are equally sized, in which the algorithm determines from the average of the data points (GeeksforGeeks, 2023).

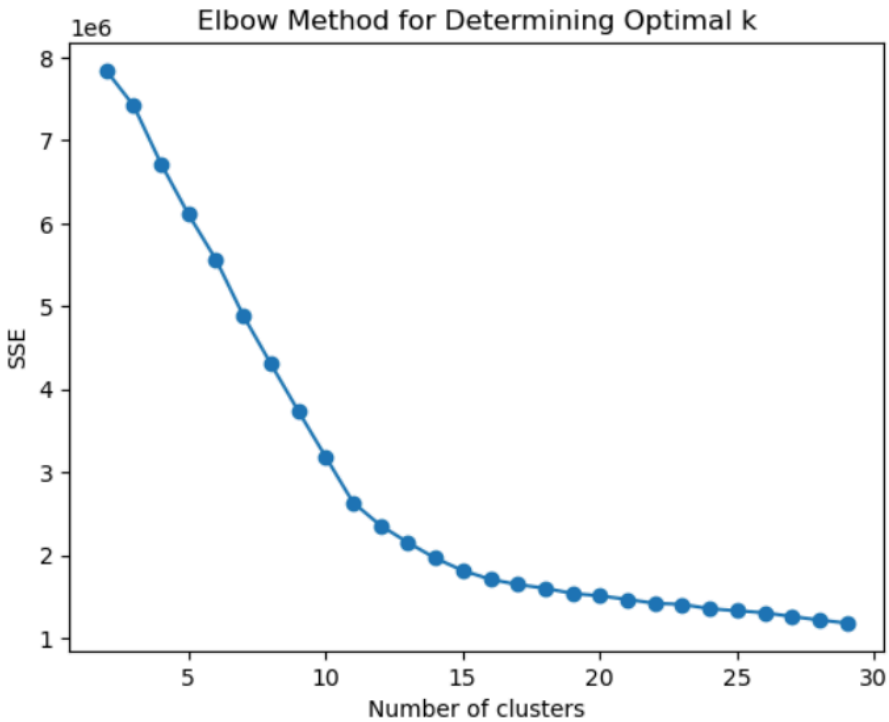
Generate Test Design

Unlike supervised learning that commonly split the dataset into train / test splits, for unsupervised learning algorithm (customer segmentation), the evaluation is based on internal validation measures such as:

1. **Elbow Method**

   It is a crucial step for unsupervised learning algorithms in order to determine the optimal number of clusters (k) into which the data may be clustered. The **sum of squared errors (SSE or inertia)** was computed for different values of k (from 2 to 30). The "elbow" in the plot was used to identify the point where the inertia starts decreasing in a linear fashion.

```
from sklearn.cluster import KMeans
sse={}
for k in range(2,30):
    kmeans=KMeans(n_clusters=k,n_init=10,init="k-means++",random_state=50)
    kmeans.fit(scaled_features_df)
    sse[k]=kmeans.inertia_

plt.plot(list(sse.keys()), list(sse.values()), marker='o')
plt.xlabel("Number of clusters")
plt.ylabel("SSE")
plt.title("Elbow Method for Determining Optimal k")
plt.show()
```



From the graph obtained, the "elbow" point is roughly around 10 – 11 clusters.

2. Silhouette Score

To further test and validate the number of clusters identified, the silhouette score is often used. This statistic evaluates the quality of data clustering results by assessing how well each data point fits within its assigned cluster and how distinct it is from points in other clusters. The silhouette score is particularly useful for evaluating clustering algorithms such as K-Means and Mini-Batch K-Means (Gültekin, 2023).

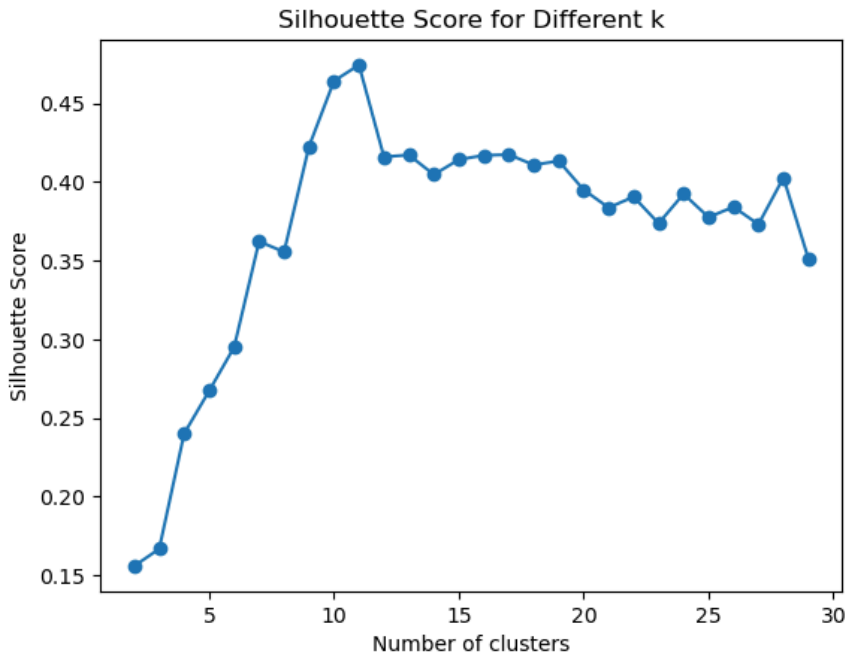
To determine the optimal number of clusters, the silhouette score can be calculated for k values ranging from 2 to 30. The score ranges from -1 to 1, where a value closer to 1 indicates that clusters are well-separated, and points are appropriately assigned. A score near zero suggests overlapping clusters, while a negative score indicates that a data point has likely been assigned to the wrong cluster.

By comparing the silhouette scores across different k values, one can better assess the performance and quality of the clustering results. In this situation, the highest silhouette score will be the number of clusters chosen.

```
from sklearn.metrics import silhouette_score


scaled_features_df = pd.DataFrame(scaled_features_df)

# Sample from the DataFrame
sample_size = min(40000, len(scaled_features_df))  # Take a sample of 40000 from dataset
sample_data = scaled_features_df.sample(n=sample_size, random_state=42)

silhouette_scores = []
for k in range(2, 30):  # start from 2 clusters, as silhouette score is undefined for k=1
    kmeans = KMeans(n_clusters=k, n_init=10, random_state=42)  # Reduced n_init
    cluster_labels = kmeans.fit_predict(sample_data)
    silhouette_avg = silhouette_score(sample_data, cluster_labels)
    silhouette_scores.append(silhouette_avg)
    print(f"Number of clusters: {k}, Silhouette Score: {silhouette_avg}")
```

```
Number of clusters: 2, Silhouette Score: 0.15595317527377145
Number of clusters: 3, Silhouette Score: 0.16679027564563179
Number of clusters: 4, Silhouette Score: 0.24013079474356636
Number of clusters: 5, Silhouette Score: 0.2672270711411411
Number of clusters: 6, Silhouette Score: 0.2949957699369899
Number of clusters: 7, Silhouette Score: 0.36220553662067706
Number of clusters: 8, Silhouette Score: 0.35552594478274413
Number of clusters: 9, Silhouette Score: 0.4225691259947137
Number of clusters: 10, Silhouette Score: 0.464188724341372
Number of clusters: 11, Silhouette Score: 0.47434679843992905
```

```
Number of clusters: 12, Silhouette Score: 0.41597769505144744
Number of clusters: 13, Silhouette Score: 0.4171329539934211
Number of clusters: 14, Silhouette Score: 0.4046537039641736
Number of clusters: 15, Silhouette Score: 0.41422524857096354
Number of clusters: 16, Silhouette Score: 0.4169137517653875
Number of clusters: 17, Silhouette Score: 0.4173838006410758
Number of clusters: 18, Silhouette Score: 0.41084295845622665
Number of clusters: 19, Silhouette Score: 0.4133277379073936
Number of clusters: 20, Silhouette Score: 0.39498242553255286
Number of clusters: 21, Silhouette Score: 0.3836662877836511
Number of clusters: 22, Silhouette Score: 0.3905621508494178
Number of clusters: 23, Silhouette Score: 0.37366295440524977
Number of clusters: 24, Silhouette Score: 0.3923986152747513
Number of clusters: 25, Silhouette Score: 0.37758301337319267
Number of clusters: 26, Silhouette Score: 0.38412621065531677
Number of clusters: 27, Silhouette Score: 0.37305877585964736
Number of clusters: 28, Silhouette Score: 0.4027126327766921
Number of clusters: 29, Silhouette Score: 0.35123447770865096
```



Based on the silhouette score results and the plotted graph, the number of clusters that gives the highest silhouette score is k = 11 with Silhouette Score:0.47434679843992905.

Hyperparameter Tuning

The purpose of doing hyperparameters is to find the optimal combination of clustering parameters based on the silhouette score which act as a performance metric. Hyperparameter tuning aims to find the best combination of parameters that lead to the highest quality clustering results.

1. **Sampling the Data**

   Purpose: Clustering can be computationally expensive when working with large datasets To reduce computation time while still maintaining the integrity of the data, a sample is taken.

   Sample size:

   - If the dataset has more than 50,000 rows, a sample of 50,000 points is used. If the datasets are smaller, the entire dataset is taken.
   - Sampling strikes a balance between computational efficiency and data representativeness.

   Random Seed: Setting random_state=42 ensures that the same random sample is selected each time the code is executed, making the process reproducible (Kumaratunga, 2021).

```
#---Hyperparameter Tuning
from sklearn.model_selection import ParameterGrid

sample_size = min(50000, len(scaled_features_df))  # Adjust sample size as needed
sample_data = scaled_features_df.sample(n=sample_size, random_state=42)

# Define parameter grid
param_grid = {
    'n_clusters': [10,11],
    'n_init': [10, 15, 20,25]
}
```

2. **Defining the Hyperparameter Grid**

   Parameter Grid: The parameter grid specifies the different values of hyperparameters to try during the tuning process.

   Among the key parameters to tune by applying grid search are:

   - **n_clusters** : This is the number of clusters (k) to generate. This is the most critical hyperparameter for KMeans because it directly affects how the data is grouped. In this case it is tested in ranges from 10 to 11. It is set according to the result of the silhouette score and elbow method.
   - **n_init**: Number of times the algorithm is run with different centroid seeds (10, 15, 20, 25). More runs will improve stability of the clustering.

```
# Define parameter grid
param_grid = {
    'n_clusters': [10,11],
    'n_init': [10, 15, 20,25]
}
```

3. **Evaluating and Storing Results**

Iterating Over Parameter Combinations: The parameter grid generates all possible combinations of parameters in the grid. Here it produces 8 combinations with 2 n_clusters * 4 n_init.

KMeans model:

- For each parameter combination, a KMeans model is created with the current n_clusters and n_init values.
- fit_predict () trains the model on the sample data and assigns each data point to a cluster.

Silhouette score calculation : The silhouette score is computed for each model to assess the quality of the clusters.

- A higher score indicates well-separated clusters while lower or negative scores suggest overlapping or poorly defined clusters.

Result Storage: Each result is stored in a list which includes the n_clusters and n_init with the corresponding silhouette score.

Best Parameter Tracking: The best-performing combination of hyperparameters is tracked by comparing each silhouette score with the current best score. If a new combination performs better than the previous, the best score and parameters are updated accordingly.

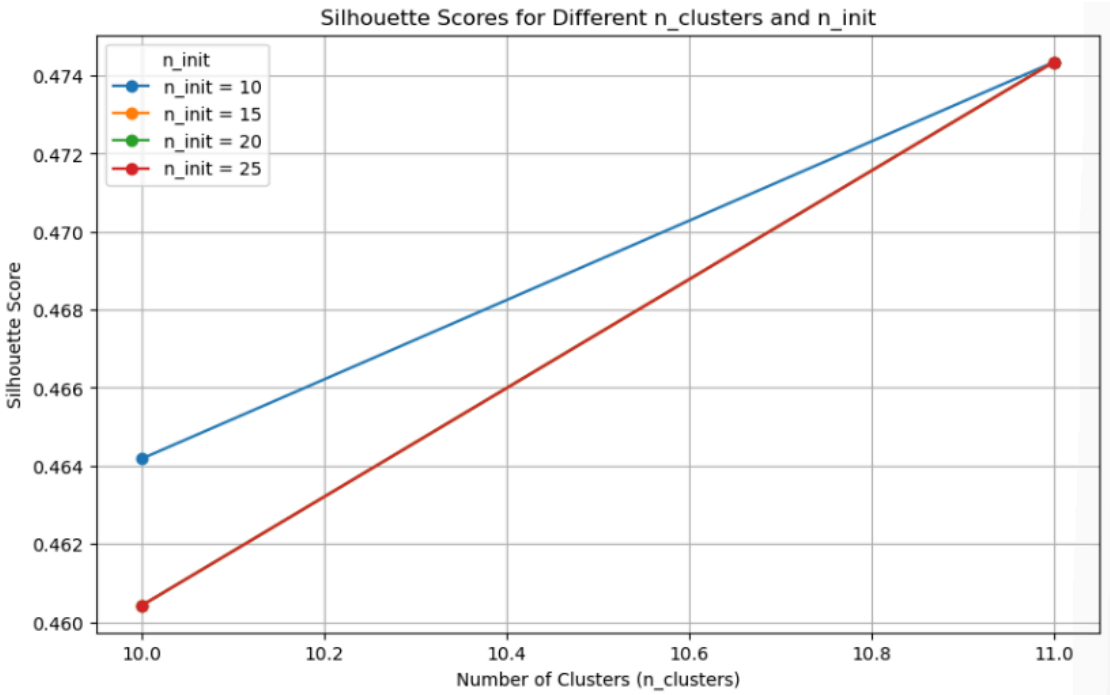
```python
results = []
best_params = None
best_score = -1

# Evaluate different parameter combinations
for params in ParameterGrid(param_grid):
    kmeans = KMeans(n_clusters=params['n_clusters'], n_init=params['n_init'], init="k-means
    cluster_labels = kmeans.fit_predict(sample_data)
    silhouette_avg = silhouette_score(sample_data, cluster_labels)

    results.append({
        'n_clusters': params['n_clusters'],
        'n_init': params['n_init'],
        'silhouette_score': silhouette_avg
    })

    # Track best parameters
    if silhouette_avg > best_score:
        best_score = silhouette_avg
        best_params = params

results_df = pd.DataFrame(results)

print("Hyperparameter Tuning Results:")
for result in results:
    print(result)
```

```
Hyperparameter Tuning Results:
{'n_clusters': 10, 'n_init': 10, 'silhouette_score': 0.464188724341372}
{'n_clusters': 10, 'n_init': 15, 'silhouette_score': 0.4604306615117417}
{'n_clusters': 10, 'n_init': 20, 'silhouette_score': 0.4604306615117417}
{'n_clusters': 10, 'n_init': 25, 'silhouette_score': 0.4604306615117417}
{'n_clusters': 11, 'n_init': 10, 'silhouette_score': 0.47434679843992905}
{'n_clusters': 11, 'n_init': 15, 'silhouette_score': 0.47434679843992905}
{'n_clusters': 11, 'n_init': 20, 'silhouette_score': 0.47434679843992905}
{'n_clusters': 11, 'n_init': 25, 'silhouette_score': 0.47434679843992905}
```

4. Visualising Results:

A) Line Plot

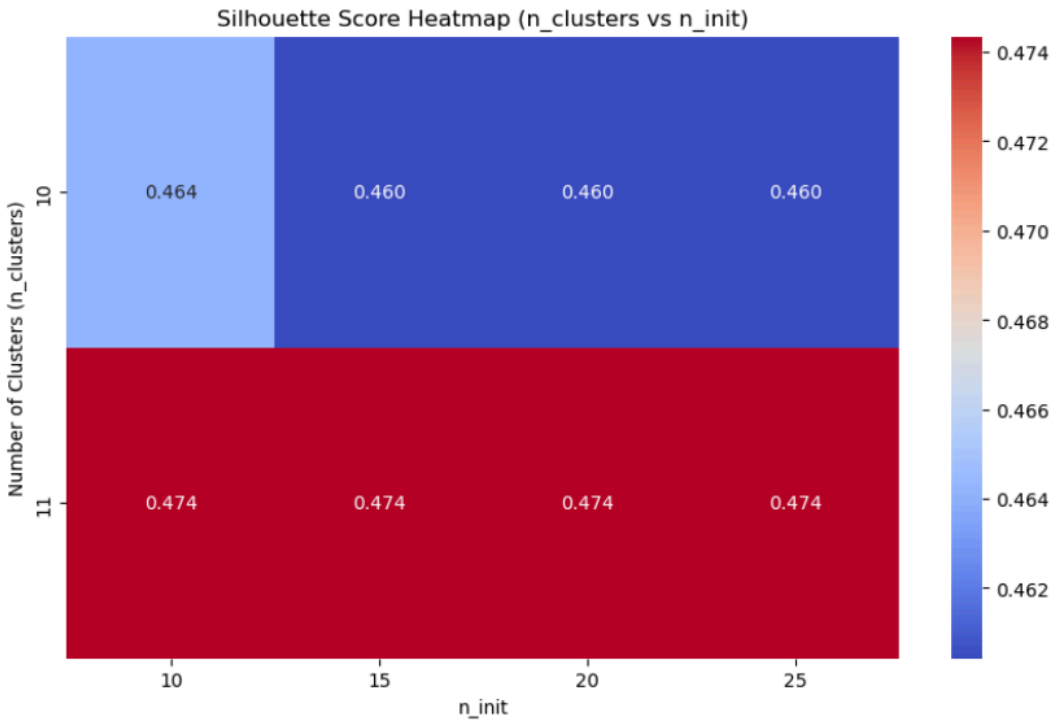
```
# Plot the silhouette scores for different n_clusters and n_init combinations
plt.figure(figsize=(10, 6))

# Plotting silhouette scores for each n_clusters
for n_init in results_df['n_init'].unique():
    subset = results_df[results_df['n_init'] == n_init]
    plt.plot(subset['n_clusters'], subset['silhouette_score'], marker='o', label=f'n_init = {n_init}')

plt.title('Silhouette Scores for Different n_clusters and n_init')
plt.xlabel('Number of Clusters (n_clusters)')
plt.ylabel('Silhouette Score')
plt.legend(title="n_init")
plt.grid(True)
plt.show()
```



- A line plot is created for each unique value of n_init, showing how the silhouette score changes with different n_clusters.
- This allows for a comparison of a clustering performance when using 10, 15, 20 or 25 initializations.
- The graphs only show the line for n_init = 10 and n_init = 25 because the silhouette scores are exactly the same for n_init = 15, n_init = 20 and n_init = 25. While the scores are identical across different n_init values, the plot may only display one line and the other lines are hidden behind it.
- This graph shows that when n_clusters=10, n_init = 10, the silhouette score is approximately 0.464 and when n_clusters=11, all n_init values pairing with it have the same score of 0.474.

B) Heatmap

```
# Pivot the data for a heatmap (Silhouette Score by n_clusters and n_init)
heatmap_data = results_df.pivot(index="n_clusters", columns="n_init", values="silhouette_score")

plt.figure(figsize=(10, 6))
sns.heatmap(heatmap_data, annot=True, fmt=".3f", cmap='coolwarm')
plt.title('Silhouette Score Heatmap (n_clusters vs n_init)')
plt.xlabel('n_init')
plt.ylabel('Number of Clusters (n_clusters)')
plt.show()
```



- A heatmap provides a detailed, visual comparison of silhouette scores across all parameter combinations. The heatmap shows how the silhouette score varies with different n_clusters and n_init values.
- The highest the silhouette scores, the better the clustering.
- The visual representation allows for quick identification of the best-performing parameters.
- This graph clearly shows the same Silhouette score across different n_init values for a given n_clusters. For n_clusters=10, the score is approximately 0.460 (except n_init = 10) and for n_clusters=11, the score is around 0.474.
- This heatmap confirms the observation that the Silhouette score does not change with different n_init values. The colour in the heatmap is consistent across the entire row, further supporting the idea that n_init does not affect the Silhouette score in this case.

5. Displaying the Best Parameters

```
print("\nBest Hyperparameters:")
print(f"Number of clusters: {best_params['n_clusters']}")
print(f"n_init: {best_params['n_init']}")
print(f"Best Silhouette Score: {best_score}")
```

```
Best Hyperparameters:
Number of clusters: 11
n_init: 10
Best Silhouette Score: 0.47434679843992716
```

The above results show the best combination of parameters that gives the highest silhouette score, which is 0.474.

Implementation of Final K-Means Clustering Modelling

Number of clusters **k = 11** and **n_init= 10**

A) Training the Final KMeans Model

```
# Best parameters
best_n_clusters = best_params['n_clusters']
best_n_init = best_params['n_init']

# Train final model
final_kmeans = KMeans(n_clusters=best_n_clusters, n_init=best_n_init,init="k-means++", random_state=42)
final_labels = final_kmeans.fit_predict(scaled_features_df)
```

Best Parameters: The optimal values of n_clusters and n_init were identified during hyperparameter tuning. These values are used to initialise the final KMeans model.

Initialization: The "k-means++" initialization is used to ensure that initial cluster centroids are chosen in a way that accelerates convergence and improves cluster quality.

B) Cluster Distribution

```
# Calculate cluster distribution
unique, counts = np.unique(final_labels, return_counts=True)
cluster_distribution = dict(zip(unique, counts))
sorted_cluster_distribution = dict(sorted(cluster_distribution.items(), key=lambda item: item[1], reverse=True))
print("Cluster Distribution :")
for cluster, count in sorted_cluster_distribution.items():
    print(f"Cluster {cluster}: {count} ")
```

```
Cluster Distribution :
Cluster 5: 236533
Cluster 2: 59510
Cluster 6: 51207
Cluster 0: 47798
Cluster 3: 34108
Cluster 10: 32184
Cluster 8: 31572
Cluster 1: 25132
Cluster 9: 10288
Cluster 7: 9689
Cluster 4: 2434
```

Purpose: After assigning each data point to a cluster, it is important to understand how many data points fall into each cluster.

Distribution: In the coding, it calculates the number of data points in each cluster that shows how the data is divided among 11 clusters. Clusters with more data points may represent more dominant segments while smaller clusters could indicate niche or outliers groups.

Sorted Output: The clusters are sorted in descending order based on the number of data points they contain and making it easier to analyse dominant clusters.

C) Generating Cluster Profiles

```
retail['Cluster']=final_kmeans.labels_
customer_clusters = retail[['CustomerID', 'Cluster']].drop_duplicates()
```

```
cluster_profiles=retail.groupby('Cluster').mean(numeric_only=True)
# Rounding and converting specific columns to integers for categories
categories = ['Quantity', 'InvoiceMonth', 'Category_Christmas Items', 'Category_Decorative Items',
              'Category_Easter Items', 'Category_Fashion Acessories', 'Category_Gardening',
              'Category_Gift Items', 'Category_Household Items', 'Category_Kitchenware',
              'Category_Other', 'Category_Stationery', 'Category_Toys']

for category in categories:
    cluster_profiles[category] = cluster_profiles[category].round().astype(int)

# Formatting UnitPrice and TotalPrice to two decimal places
cluster_profiles['UnitPrice'] = cluster_profiles['UnitPrice'].round(2)
cluster_profiles['TotalPrice'] = cluster_profiles['TotalPrice'].round(2)

if 'Country_United Kingdom' in cluster_profiles.columns:
    cluster_profiles['Country_United Kingdom'] = cluster_profiles['Country_United Kingdom'].round()
print('Cluster Profile: ')
print(cluster_profiles)
print(customer_clusters.head(20))
```

Cluster Profile:

Cluster	Quantity	InvoiceMonth	UnitPrice	TotalPrice
0	7	8	2.35	11.76
1	8	10	2.17	12.85
2	6	7	3.09	13.10
3	5	8	3.82	16.36
4	6	3	2.08	9.44
5	6	7	3.24	13.03
6	7	7	2.45	13.44
7	9	7	2.01	9.17
8	8	8	1.56	7.82
9	5	8	3.03	11.87
10	6	7	3.82	14.64

Cluster	Category_Christmas Items	Category_Decorative Items
0	0	1
1	1	0
2	0	0
3	0	0
4	0	0
5	0	0
6	0	0
7	0	0
8	0	0
9	0	0
10	0	0

Cluster	Category_Easter Items	Category_Fashion Acessories
0	0	0
1	0	0
2	0	0
3	0	0
4	1	0
5	0	0
6	0	1
7	0	0
8	0	0
9	0	0
10	0	0

Cluster	Category_Gardening	Category_Gift Items	Category_Household Items
0	0	0	0
1	0	0	0
2	0	0	0
3	0	0	1
4	0	0	0
5	0	0	0
6	0	0	0
7	0	1	0
8	0	0	0
9	0	0	0
10	1	0	0

Cluster	Category_Kitchenware	Category_Other	Category_Stationery
0	0	0	0
1	0	0	0
2	1	0	0
3	0	0	0
4	0	0	0
5	0	1	0
6	0	0	0
7	0	0	0
8	0	0	1
9	0	0	0
10	0	0	0

Cluster	Category_Toys	Country_United Kingdom
0	0	1.0
1	0	1.0
2	0	1.0
3	0	1.0
4	0	1.0
5	0	1.0
6	0	1.0
7	0	1.0
8	0	1.0
9	1	1.0
10	0	1.0

	CustomerID	Cluster
0	17850	0
3	17850	3
4	17850	5
9	13047	5
12	13047	9
13	13047	2
15	13047	3
26	12583	5
30	12583	7
32	12583	8
33	12583	10
34	12583	3
38	12583	9
44	12583	0
46	13748	1
53	17850	2
65	15100	3
82	15291	3
83	15291	0
86	14688	6

Cluster Profile: Each cluster is analysed by averaging the key variables for the data points within that cluster.

Purpose: Cluster profiles help identify the characteristics of each cluster. For example, cluster 0 has lower average in Quantity and UnitPrice and low values across all category features. Thus, it represents a general transaction pattern with no specific category dominance.

Rounding and Formatting: Certain numeric columns are rounded to provide a clearer output and make the profiles more interpretable. For instance, categories like Christmas Items or Kitchenware are rounded to the integers for clarity.

Customer Segmentation: After clustering, each customer is assigned to a cluster label that indicates which segments they belong to. The final result is a customer segmentation where customers are grouped based on their purchasing behaviour or other features. Each cluster represents a distinct customer segment.

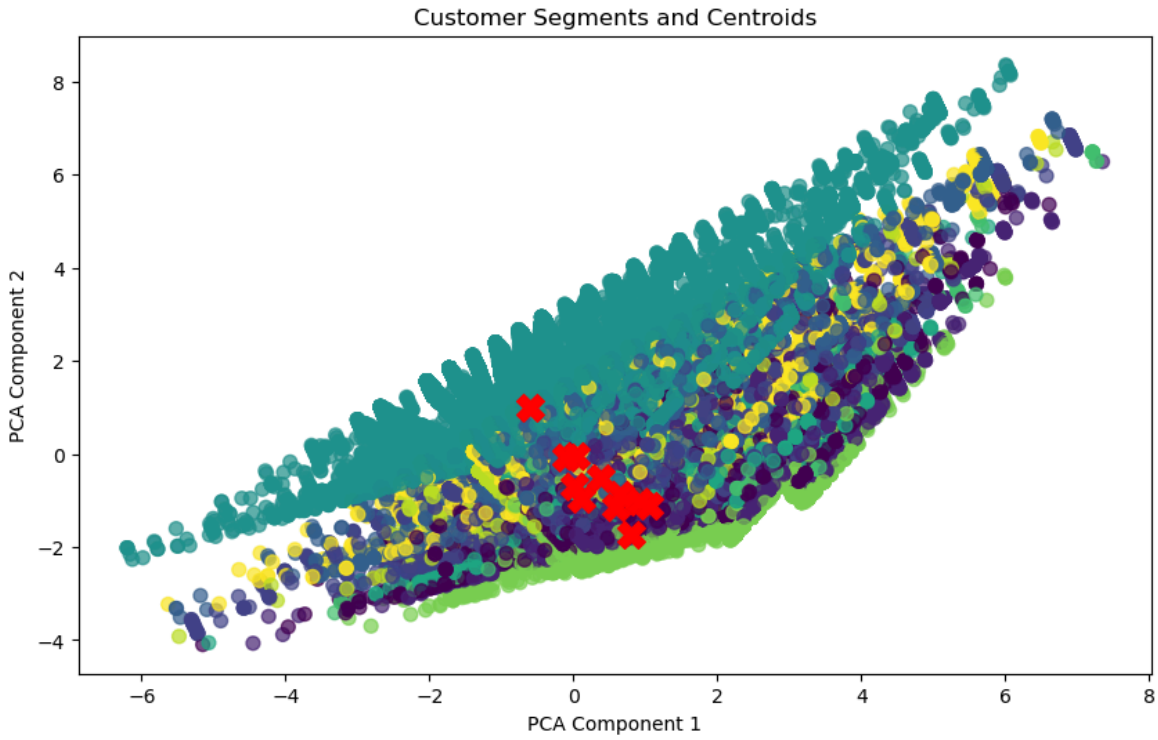
D) PCA Diagram

Principal Component Analysis (PCA) is a widely used dimensionality reduction technique that simplifies large datasets by transforming them into a smaller set of variables, while retaining most of the original information (Jaadi, 2024). The PCA graph is designed to visualise high-dimensional data in a reduced-dimensional space. It typically displays the first two principal components, which are the new axes that capture the most variance in the data.

By representing data points in this reduced space, the graph facilitates the identification of patterns, clusters, and distributions. It is particularly useful for visualising the results of clustering algorithms like K-Means, showing how clusters are organised and how distinct or overlapping they are.

In the provided code, the centroids from the final K-Means clustering model are converted into a DataFrame. This conversion aligns the centroids with the feature names of the original dataset, allowing them to be transformed and visualised in the same reduced-dimensional space as the data points. This approach ensures that the cluster centroids are accurately represented in the PCA graph, providing a clear view of their positions relative to the data points.

```
from sklearn.decomposition import PCA

# Fit PCA on the original scaled features
pca = PCA(n_components=2)
reduced_pca = pca.fit_transform(scaled_features_df)

# Convert centroids to a DataFrame with the same columns as scaled_features_df
centroids_df = pd.DataFrame(final_kmeans.cluster_centers_, columns=scaled_features_df.columns)

# Transform centroids using PCA
centroids_pca = pca.transform(centroids_df)

# Plot the clusters and centroids
plt.figure(figsize=(10, 6))
plt.scatter(reduced_pca[:, 0], reduced_pca[:, 1], c=final_labels, cmap='viridis', s=50, alpha=0.7)
plt.scatter(centroids_pca[:, 0], centroids_pca[:, 1], c='red', s=200, marker='X')  # Plot centroids
plt.title('Customer Segments and Centroids')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.show()
```



PCA Analysis :

This plot visualises the customer segments and centroids using PCA to reduce the datasets' dimensions to two principal components. Below is the analysis of the PCA result:

1. Data Distribution

   The plot shows the data point(customers) distributed along two principal components. The spread of points suggests that there is some variation in the dataset across these two components with the point being respectively concentrated towards the centre and fanning out as the data move further from the origin. The colour of the points likely indicates different clusters assigned by the K-Means algorithm and showing multiple overlapping clusters with some being more distinct than others.

2. Clusters

   The red "X" markers represent the centroids of the clusters found by the K-Means algorithm. These centroids are the centre points of each cluster and help identify the general location of each cluster. The cluster centroids are somewhat close to each other, indicating that the cluster might not be highly separated and leading to possible overlaps between them.

3. Overlap and Separation

   There appears to be some overlap between the clusters as the points in different colours are not entirely separated. This could suggest that some cluster segments have similarities in

behaviour or characteristics and making it harder to distinguish them based purely on these two principal components.

4. Variance Explained

   Since only the first two principal components are used for the visualisation, the total variance in the dataset is compressed into these two dimensions. While the plot helps visualise the clusters, it may not fully capture the multi-dimensional nature of the original data, which could explain why the clusters seem to overlap.

Comparison Between Difference Distance Metric

Distance metrics provide the backbone of clustering techniques. It is a mathematical function that calculates how similar or dissimilar two data points are. The choice of distance metric has a considerable impact on clustering results since it determines the shape and structure of the clusters. The distance metric used has a considerable impact on the quality of the clusters created and the insights gained from them. A well-selected distance metric can result in meaningful clusters that highlight the deeper trends in the data, but a badly chosen measure can produce misleading or irrelevant clusters (GeeksforGeeks, 2024).

In the coding, it has included a *compute_silhouette()* function which is used to calculate the Silhouette Score for a given distance metric which is the euclidean distance metric, manhattan distance metric and cosine distance metric.

It has used the sample data due to the reason of clustering algorithms and distance calculations can be computationally expensive on large datasets. Thus, with using sample data , it can greatly reduce the size while preserving the structure and making the computations more efficient.

A) Euclidean Distance Metric

The euclidean distance is the most widely used distance metric in clustering. It calculates the straight line distance between two points in n-dimensional space (GeeksforGeeks, 2024).

```
# Testing EUCLIDEAN DISTANCE METRIC

# Define the function to compute silhouette score with a given distance metric
def compute_silhouette(metric, X, k):
    print(f"Computing silhouette score for metric: {metric}")
    kmeans = KMeans(n_clusters=k, init='k-means++', n_init='auto', random_state=50)
    kmeans.fit(X)
    labels = kmeans.labels_
    silhouette_avg = silhouette_score(X, labels, metric=metric)
    print(f"Completed metric: {metric}, Silhouette Score: {silhouette_avg}")
    return silhouette_avg

# Sample the data that is too large
sample_size = min(50000, len(scaled_features_df))  # Sample of 50 000
sample_data = scaled_features_df.sample(n=sample_size, random_state=42)

# Test for Euclidean distance
k = best_params['n_clusters']
silhouette_scores = {}
euclidean_silhouette = compute_silhouette('euclidean', sample_data, k)
silhouette_scores['euclidean'] = euclidean_silhouette
print(f"Euclidean Distance - Silhouette Score: {euclidean_silhouette}")
```

```
Computing silhouette score for metric: euclidean
Completed metric: euclidean, Silhouette Score: 0.44775432947189225
Euclidean Distance - Silhouette Score: 0.44775432947189225
```

B) Manhattan Distance Metric

The Manhattan distance is the sum of the absolute differences between their Cartesian coordinates, commonly known as the L1 distance or city block distance. This movement is represented by the Manhattan distance, which is calculated as the total distance travelled along each dimension to reach a new data point. When it comes to categorical data, this metric outperforms Euclidean distance since it is less vulnerable to outliers (GeeksforGeeks, 2024).

```
# Testing MANHATTAN DISTANCE METRIC

# Test for Manhattan distance
k = best_params['n_clusters']
manhattan_silhouette = compute_silhouette('manhattan', sample_data, k)
silhouette_scores['manhattan'] = manhattan_silhouette
print(f"Manhattan Distance - Silhouette Score: {manhattan_silhouette}")
```

```
Computing silhouette score for metric: manhattan
Completed metric: manhattan, Silhouette Score: 0.5381878331352504
Manhattan Distance - Silhouette Score: 0.5381878331352504
```

C) Cosine Distance Metric

Cosine distance metric looks at the data point orientation instead than concentrating on the exact distance between data points. It calculates the cosine of the angle between two data points with higher cosine value indicating greater similarity (GeeksforGeeks, 2024).

```
# Testing COSINE DISTANCE METRIC

# Test for Cosine distance
k = best_params['n_clusters']
cosine_silhouette = compute_silhouette('cosine', sample_data, k)
silhouette_scores['cosine'] = cosine_silhouette
print(f"Cosine Distance - Silhouette Score: {cosine_silhouette}")
```

```
Computing silhouette score for metric: cosine
Completed metric: cosine, Silhouette Score: 0.5781687488181796
Cosine Distance - Silhouette Score: 0.5781687488181796
```

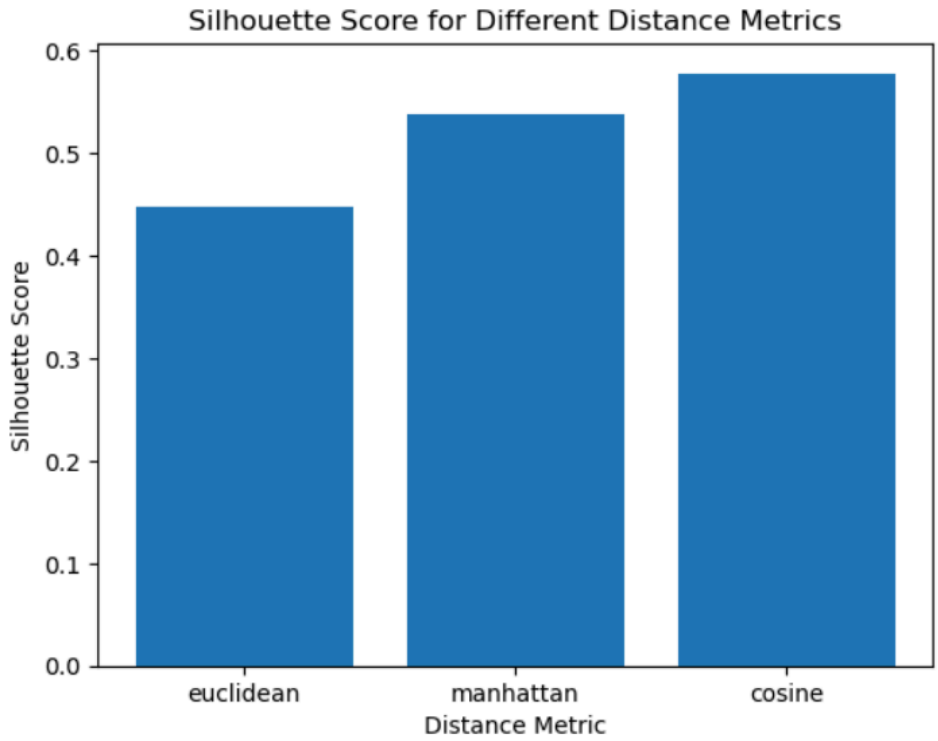
D) Determine Best Distance Metric

After calculating the Silhouette Scores for each distance metric, the code determines which distance metric produces the highest score and it will be the best distance metric that can be used in the KMeans clustering algorithm. In the online retail datasets, the result below clearly indicates that Cosine distance metric is the best distance metric because it produces the highest silhouette score. Besides, a silhouette score that greater than 0.5 generally indicates well separated clusters.

```python
# Find the best metric based on the highest silhouette score
best_metric = max(silhouette_scores, key=silhouette_scores.get)
best_score = silhouette_scores[best_metric]

# Print the best metric and its silhouette score
print(f"\nBest Distance Metric: {best_metric}")
print(f"Best Silhouette Score: {best_score}")

# Optional: Print all metrics and their scores for reference
print("\nAll Distance Metrics and Scores:")
for metric, score in silhouette_scores.items():
    print(f"{metric}: {score}")
```

```
Best Distance Metric: cosine
Best Silhouette Score: 0.5781687488181796

All Distance Metrics and Scores:
euclidean: 0.44775432947189225
manhattan: 0.5381878331352504
cosine: 0.5781687488181796
```

E) Visualisation in Bar Chart



To easily visualise the result of three different distance metrics, the bar chart has been used to show the silhouette score for Euclidean, Manhattan and Cosine distance metric. It allows us to see which metric performed the best.

Implementation of K-Means Clustering Modelling With Cosine Distance Metric

Number of clusters, **n_clusters = 11** and **distance metric= cosine**

A) Normalisation of Data

Normalisation scales the features values so that all data points have a unit norm. Since cosine distance depends on the angles between data points, normalisation is essential to ensure that differences in scale do not affect the clustering process.

The normalize() function ensures that all features have a consistent scale and makes the cosine distance metric more efficient.

```
import faiss
import numpy as np
from sklearn.preprocessing import normalize
from sklearn.utils import resample
from collections import Counter

sample_size = min(50000, len(scaled_features_df))  # Sample of 50 000
sample_data = scaled_features_df.sample(n=sample_size, random_state=50)
X_normalized = normalize(sample_data)
n_clusters = best_params['n_clusters']
```

B) Running K-Means with Cosine Distance Approximation

```
# Set up faiss KMeans with cosine distance
d = X_normalized.shape[1]  # Dimensionality of data

# Using inner product (cosine similarity)
kmeans = faiss.Kmeans(d, n_clusters, niter=20, spherical=True)  # spherical=True ensures cosine distance

# Train KMeans on the normalized data
kmeans.train(X_normalized.astype(np.float32))

# Get the cluster centroids and labels
centroids = kmeans.centroids
D, labels = kmeans.index.search(X_normalized.astype(np.float32), 1)
```

This code sets up and runs K-Means clustering using cosine distance by normalising the data, then training the KMeans model with spherical clustering to approximate cosine similarity. After training, it retrieves the cluster centroids and assigns cluster labels to the data points.

C) Silhouette Score Calculation

```
silhouette_avg = silhouette_score(X_normalized, labels)
print(f'Silhouette Score: {silhouette_avg}')
```

```
Silhouette Score: 0.4089689832003254
```

A silhouette score of 0.409 is just a moderately well-defined cluster with moderate clustering performance. The data points in each cluster are more similar to each other than they are to data points in other clusters, but there might be some overlap or less clear boundaries between clusters.

D) Cluster Distribution

```
# Analyze the cluster distribution
labels = labels.ravel()  # Flatten the labels to a 1D array
cluster_sizes = Counter(labels)

# Sort the cluster sizes in descending order
sorted_cluster_sizes = sorted(cluster_sizes.items(), key=lambda item: item[1], reverse=True)

# Print the sorted cluster sizes
print("Cluster Sizes Distribution (Descending):")
for cluster, size in sorted_cluster_sizes:
    print(f"Cluster {cluster}: {size} points")
```

```
Cluster Sizes Distribution (Descending):
Cluster 7: 6360 points
Cluster 5: 5709 points
Cluster 1: 5537 points
Cluster 8: 5464 points
Cluster 2: 5369 points
Cluster 10: 5066 points
Cluster 0: 4707 points
Cluster 3: 4353 points
Cluster 4: 3231 points
Cluster 6: 2986 points
Cluster 9: 1218 points
```

The cluster sizes are fairly well distributed across most clusters, maintaining a reasonable balance overall. This distribution supports meaningful segmentation for customer profiling.

E) Cluster Profile

```python
# Add Cluster Labels to Sampled Data
sample_data_with_labels = sample_data.copy()
sample_data_with_labels['Cluster_Cosine'] = labels  # Assign the cluster labels to the sampled data

# Merge the cluster labels back into the original dataset using the index
scaled_features_df_with_clusters_cosinekmeans = pd.merge(scaled_features_df,
                                                         sample_data_with_labels[['Cluster_Cosine']],
                                                         how='left',
                                                         left_index=True,
                                                         right_index=True)
# Create cluster profiles for cosine KMeans
cluster_profiles_cosinekmeans = scaled_features_df_with_clusters_cosinekmeans.groupby('Cluster_Cosine').mean(numeric_only=True)

# Extract the customer clusters for cosine KMeans
customer_clusters_cosinekmeans = retail[['CustomerID']].drop_duplicates().merge(
    sample_data_with_labels[['Cluster_Cosine']],
    how='left',
    left_index=True,
    right_index=True
)
customer_clusters_cosinekmeans['Cluster_Cosine'] = customer_clusters_cosinekmeans['Cluster_Cosine'].fillna(-1, inplace=False)
filtered_customer_clusters = customer_clusters_cosinekmeans[customer_clusters_cosinekmeans['Cluster_Cosine'] != -1.0]
customer_clusters_cosinekmeans = filtered_customer_clusters
```

```python
# Displaying Cluster Profiles
print("\nCluster Profiles:")
print(cluster_profiles_cosinekmeans)

# Displaying Customer Clusters
print("\nCustomer Clusters:")
print(customer_clusters_cosinekmeans.head(30))
```

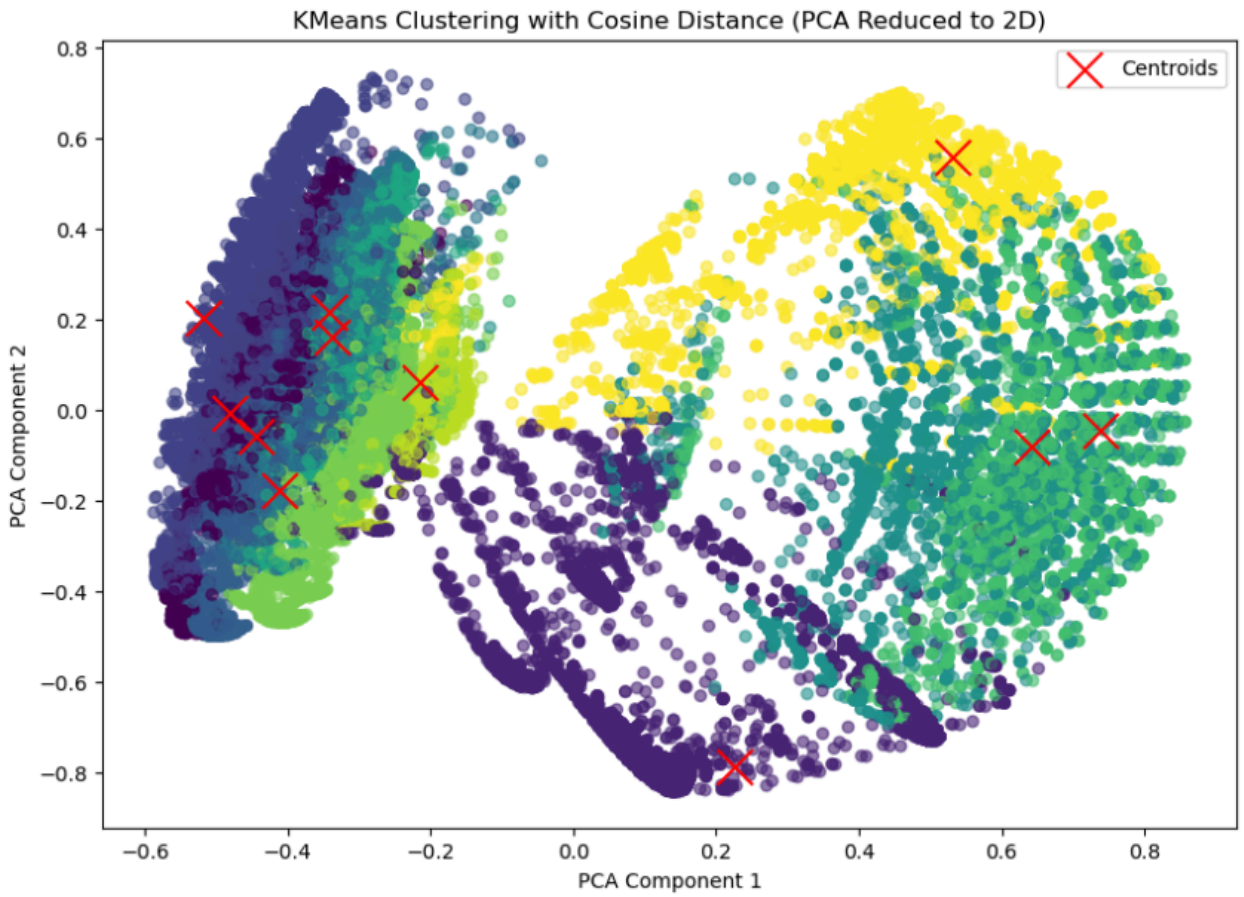
```
Cluster Profiles:
                Quantity  InvoiceMonth  UnitPrice  TotalPrice  \
Cluster_Cosine
0.0             0.046381     -0.083516  -0.221456    0.027557
1.0             1.430189     -0.004962  -0.520199    0.834572
2.0            -0.036444     -0.129567   0.063063   -0.019225
3.0             0.089815      0.007314  -0.264523   -0.075911
4.0            -0.138439      0.182089   0.367999    0.200907
5.0            -0.422062     -1.104554  -0.325401   -0.380277
6.0            -0.152768     -0.191680   0.387846    0.122796
7.0            -0.511017      0.854027  -0.308325   -0.431341
8.0             0.202970      0.392405  -0.482309   -0.198528
9.0            -0.151422     -0.175269  -0.044626   -0.107629
10.0           -0.591440     -0.039729   1.827292    0.171437

                Category_Christmas Items  Category_Decorative Items  \
Cluster_Cosine
0.0                            -0.220838                  -0.311482
1.0                            -0.197680                  -0.309573
2.0                            -0.220838                  -0.311482
3.0                            -0.220838                   3.210462
4.0                            -0.220838                  -0.311482
5.0                            -0.220838                  -0.311482
6.0                            -0.220838                  -0.311482
7.0                            -0.220838                  -0.311482
8.0                             1.799940                  -0.311482
9.0                            -0.220838                  -0.311482
10.0                           -0.220838                  -0.311482

                Category_Easter Items  Category_Fashion Acessories  \
Cluster_Cosine
0.0                         -0.067261                    3.091006
1.0                         -0.067261                   -0.323519
2.0                         -0.067261                   -0.323519
3.0                         -0.067261                   -0.323519
4.0                         -0.067261                   -0.323519
5.0                         -0.067261                   -0.323519
6.0                         -0.067261                   -0.323519
7.0                         -0.067261                   -0.323519
8.0                         -0.067261                   -0.323519
9.0                          2.961391                   -0.323519
10.0                        -0.067261                   -0.323519

                Category_Gardening  Category_Gift Items  \
Cluster_Cosine
0.0                      -0.251636            -0.135110
1.0                      -0.251636             0.559057
2.0                      -0.251636            -0.135110
3.0                      -0.251636            -0.135110
4.0                      -0.251636            -0.135110
5.0                      -0.251636            -0.067785
6.0                       3.973998            -0.135110
7.0                      -0.251636            -0.132740
8.0                      -0.251636             0.183507
9.0                      -0.251636            -0.135110
10.0                     -0.251636             0.053823

                Category_Household Items  Category_Kitchenware  \
Cluster_Cosine
0.0                            -0.259540             -0.351761
1.0                            -0.259540             -0.342529
2.0                            -0.259540              2.842842
3.0                            -0.259540             -0.351761
4.0                             3.852973             -0.351761
5.0                            -0.259540             -0.351761
6.0                            -0.259540             -0.351761
7.0                            -0.259540             -0.351761
8.0                            -0.259540             -0.351761
9.0                            -0.259540             -0.351761
10.0                           -0.259540             -0.351761

                Category_Other  Category_Stationery  Category_Toys  \
Cluster_Cosine
0.0                   -0.882195            -0.249082      -0.139303
1.0                    0.931125            -0.249082      -0.139303
2.0                   -0.882195            -0.249082      -0.139303
3.0                   -0.882195            -0.249082      -0.139303
4.0                   -0.882195            -0.249082      -0.139303
5.0                    1.115529            -0.249082      -0.139303
6.0                   -0.882195            -0.249082      -0.139303
7.0                    1.132902            -0.249082      -0.139303
8.0                   -0.882195             2.020173      -0.139303
9.0                   -0.882195            -0.249082       5.694607
10.0                   1.083003            -0.249082      -0.139303
```

Customer Clusters:

	CustomerID	Cluster_Cosine
82	15291	4.0
105	17809	1.0
168	17420	4.0
252	13747	6.0
415	14045	6.0
1309	16955	8.0
1377	15165	3.0
2252	14496	8.0
2346	12947	4.0
2373	14142	4.0
3282	17017	4.0
3328	13108	4.0
3410	14766	7.0
4056	17235	10.0
4561	15640	8.0
4726	17228	7.0
5326	12841	8.0
6350	14589	2.0
6492	13013	6.0
6560	14210	7.0
8991	14161	0.0
10832	18219	10.0
11428	14748	1.0
11457	16638	4.0
11761	13497	2.0
11858	15048	3.0
11915	14708	4.0
12007	18113	7.0
12071	15078	1.0
12266	17259	3.0

F) PCA Diagram

```
# 7. Apply PCA to reduce dimensions for visualization (2D)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_normalized)

# Apply PCA on the centroids as well for plotting
centroids_pca = pca.transform(centroids)

# 8. Plot the clusters and centroids in 2D
plt.figure(figsize=(10, 7))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap='viridis', s=30, alpha=0.6)
plt.scatter(centroids_pca[:, 0], centroids_pca[:, 1], s=300, c='red', marker='x', label='Centroids')  # Centroids
plt.title("KMeans Clustering with Cosine Distance (PCA Reduced to 2D)")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.legend()
plt.show()
```



PCA Diagram Analysis

This PCA graph shows the results of clustering using KMeans with the Cosine Distance metric.

1. Data Points

   Each point on the graph represents a customer in a reduced two-dimensional space. These points are grouped into clusters based on their similarities, determined by the model. The

PCA projection reduces the complexity of the data, making the clusters more visually interpretable.

2. Cluster Separation

   The plot shows three prominent clusters:

   - **Left cluster:** A large group of closely clustered points, showing a distinct customer group.
   - **Right cluster**: Another group of points representing different segments of customers.
   - **Middle cluster:** A more dispersed set of points, having some transition or overlap between the left and right clusters.

3. Insights

   - Clear distinct group of left and right shows that the model successfully identified separate customer behaviours.
   - Overall, the separation of clusters suggests that the KMeans algorithm implementing the Cosine distance metrics successfully detected meaningful patterns in the data.

MiniBatch K-Means Clustering

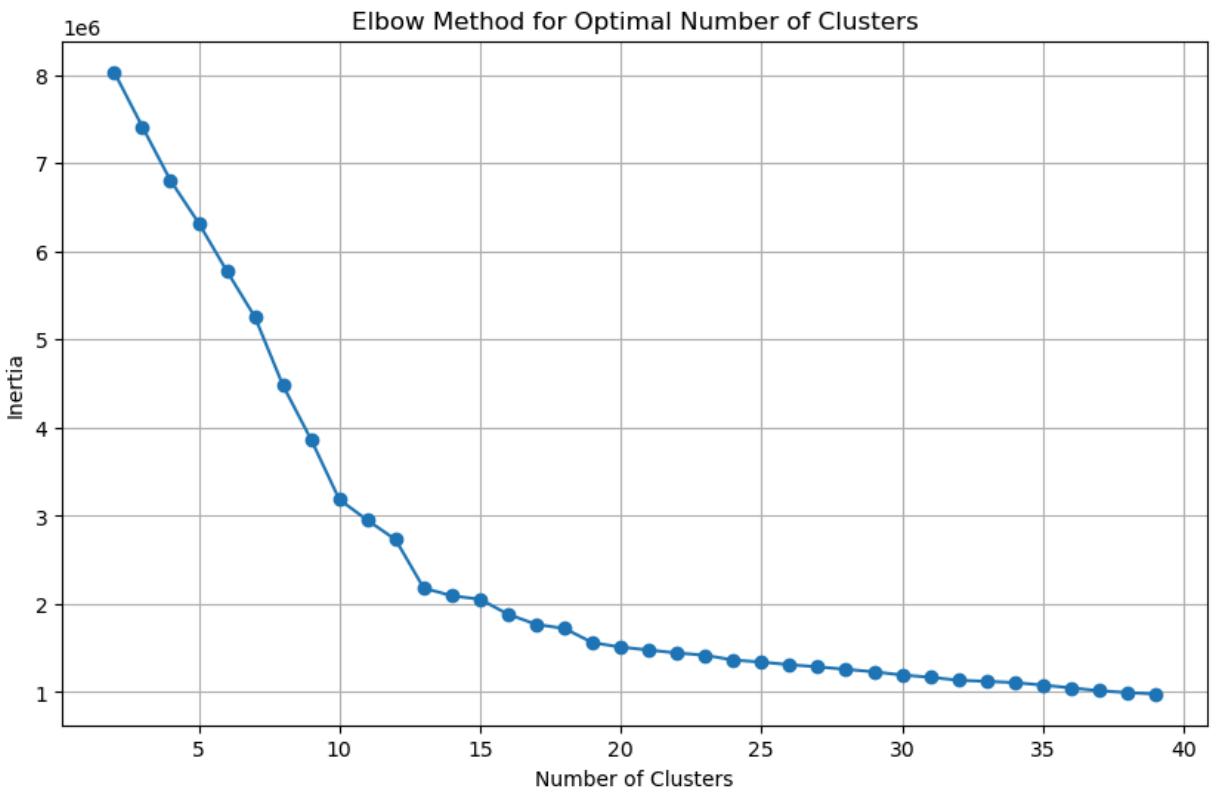
Mini-batch K-Means is a variation of the traditional K-Means clustering algorithm that is designed to handle large datasets. It is because in traditional K-Means, the algorithm processes the entire datasets in each iteration, which can be computationally expensive for large datasets. Mini-batch K-Means uses small random subsets (mini-batches) of data to update the clusters for each iteration until convergence (GeeksforGeeks, 2024b). It is a popular clustering technique due to its good time performance which is particularly useful for large datasets just like the online retail dataset we handled.

Test Design

1. **Elbow Method**

   The SSE was computed for different values of k (from 2 to 40) and the results are plotted to identify the optimal number of clusters.

```
# ---- MiniBatchKMeans ----
#Elbow Method To get the number of clusters
from sklearn.cluster import MiniBatchKMeans
# Define a range of cluster numbers to test
cluster_range = range(2, 40)
inertia_values = []

for n_clusters in cluster_range:
    model = MiniBatchKMeans(n_clusters=n_clusters, batch_size=5000, random_state=42)
    model.fit(scaled_features_df)  # Your scaled feature data
    inertia_values.append(model.inertia_)

# Plotting the Elbow Method
plt.figure(figsize=(10, 6))
plt.plot(cluster_range, inertia_values, marker='o')
plt.title('Elbow Method for Optimal Number of Clusters')
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia')
plt.grid(True)
plt.show()
```



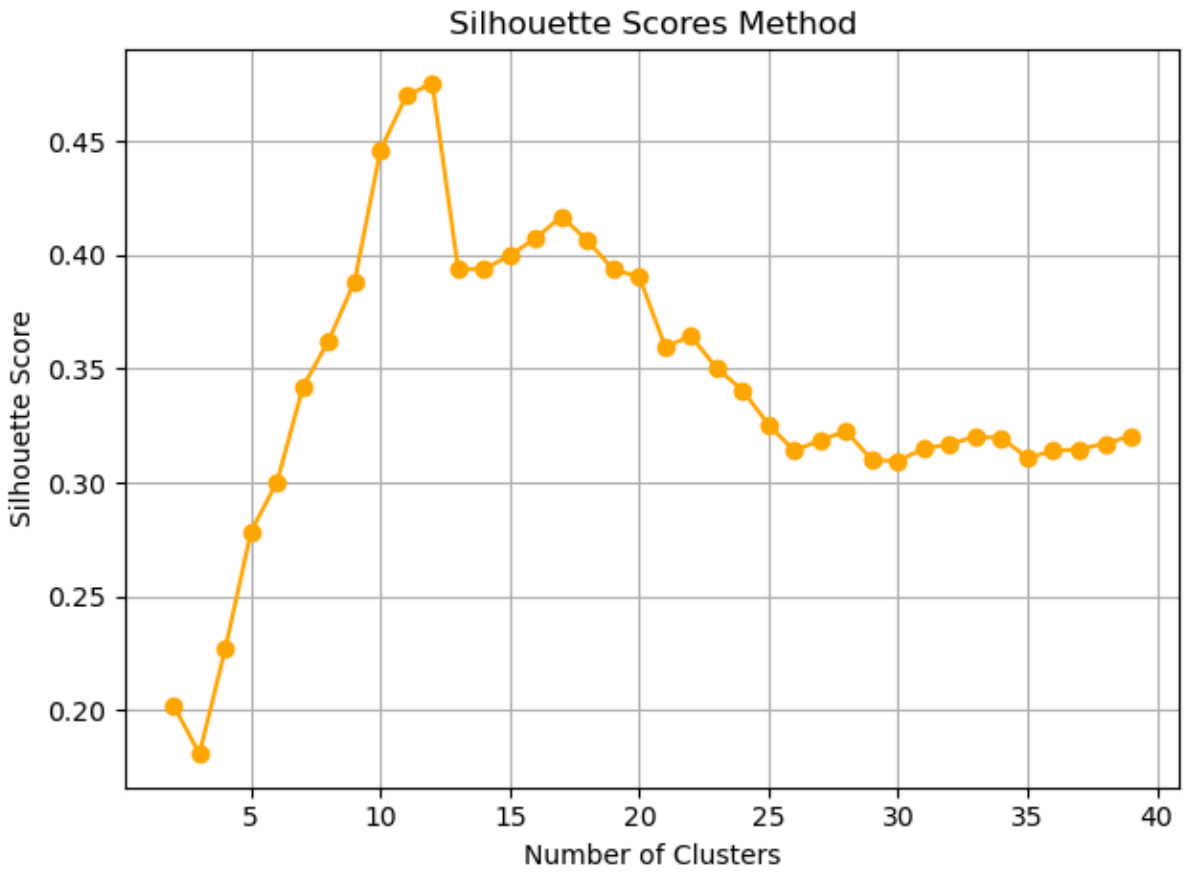
From the graph, the "elbow" point is roughly around 8 – 12.

2. Silhouette Score

For MiniBatch K-Means, we have tested and calculated the silhouette score for the number of clusters ranging from 2 to 40.

```
# Define a range of cluster numbers to test
inertia_values = []
silhouette_scores = []

# Sample the data that is too large
sample_size = min(40000, len(scaled_features_df))  # Sample of 40 000
sample_data = scaled_features_df.sample(n=sample_size, random_state=42)

for n_clusters in range(2, 40):
    # Initialize and fit the model
    model = MiniBatchKMeans(n_clusters=n_clusters, batch_size=5000, random_state=42)
    cluster_labels = model.fit_predict(sample_data)  # Scaled feature data

    # Compute inertia
    inertia = model.inertia_
    inertia_values.append(inertia)

    silhouette_avg = silhouette_score(sample_data, cluster_labels)
    silhouette_scores.append(silhouette_avg)
    print(f"Number of clusters: {n_clusters}, Silhouette Score: {silhouette_avg}")
```

```
Number of clusters: 2, Silhouette Score: 0.20233298961884796
Number of clusters: 3, Silhouette Score: 0.18095208346333103
Number of clusters: 4, Silhouette Score: 0.2270364146347876
Number of clusters: 5, Silhouette Score: 0.27873107224012206
Number of clusters: 6, Silhouette Score: 0.30033694464316596
Number of clusters: 7, Silhouette Score: 0.34206329975204236
Number of clusters: 8, Silhouette Score: 0.362496729020083
Number of clusters: 9, Silhouette Score: 0.38850839995963027
Number of clusters: 10, Silhouette Score: 0.44625299196969
Number of clusters: 11, Silhouette Score: 0.4701051666611889
Number of clusters: 12, Silhouette Score: 0.4753845104743485
Number of clusters: 13, Silhouette Score: 0.39378790865340574
Number of clusters: 14, Silhouette Score: 0.39385038525476457
Number of clusters: 15, Silhouette Score: 0.3997872371991859
Number of clusters: 16, Silhouette Score: 0.4074853294669331
Number of clusters: 17, Silhouette Score: 0.4169357001945675
Number of clusters: 18, Silhouette Score: 0.4061953709853513
Number of clusters: 19, Silhouette Score: 0.39368093606146803
Number of clusters: 20, Silhouette Score: 0.39042511984939293
Number of clusters: 21, Silhouette Score: 0.3597783484431162
Number of clusters: 22, Silhouette Score: 0.3643445602179172
Number of clusters: 23, Silhouette Score: 0.3501832710378108
Number of clusters: 24, Silhouette Score: 0.3405850737563809
Number of clusters: 25, Silhouette Score: 0.32535623787902845
Number of clusters: 26, Silhouette Score: 0.3139554517332389
Number of clusters: 27, Silhouette Score: 0.3188927719243126
Number of clusters: 28, Silhouette Score: 0.3225377223011663
Number of clusters: 29, Silhouette Score: 0.30984184788534824
Number of clusters: 30, Silhouette Score: 0.30968692498509176
Number of clusters: 31, Silhouette Score: 0.31496948783558165
Number of clusters: 32, Silhouette Score: 0.31689377369808747
Number of clusters: 33, Silhouette Score: 0.3203614434862559
Number of clusters: 34, Silhouette Score: 0.31960259616823516
Number of clusters: 35, Silhouette Score: 0.3109541836275167
Number of clusters: 36, Silhouette Score: 0.31398338817815036
Number of clusters: 37, Silhouette Score: 0.3146858618816736
Number of clusters: 38, Silhouette Score: 0.3172477200200111
Number of clusters: 39, Silhouette Score: 0.32049845081404243
```



Based on the results, it clearly shows that the number of clusters that give the highest silhouette score is k = 12 with Silhouette score: 0.475.

Hyperparameter Tuning

1. **Sampling the Data**

   Similar to the KMeans modelling previously, a sample of 40 000 data points is taken due to the large datasets. Random seed is set to random_state = 42 so that the results are consistent and reproducible.

```
# Sample the data that is too large
sample_size = min(50000, len(scaled_features_df))  # Sample of 50 000
sample_data = scaled_features_df.sample(n=sample_size, random_state=42)
```

2. **Defining the Hyperparameter Grid**

   For MiniBatch K-Means Clustering, the key parameters to tune by applying grid search are:

   - **n_clusters**: The number of clusters to present the data. In this case, it is tested from 8 to 12 clusters based on the evaluations from elbow method and silhouette score, to identify the optimal number of clusters.
   - **batch_size**: MiniBatch K-Means uses random mini-batches to speed up the convergence process. The size of the random batches used to update the centroids is controlled by the batch_size parameter. Different batch_size values will be explored, such as 2500, 5000, and 10000, to find the most efficient batch size for the data.

```
# Define hyperparameters to tune
param_grid = {
    'n_clusters': [8,9,10,11,12,13,14],
    'batch_size': [2500,5000,10000,15000]
}
```

3. Parameters Combination Results

The grid search iterates over all possible combinations of n_clusters and batch_size, recording the silhouette scores. After running the grid search, the results for each combination of parameters will be stored, and the highest silhouette score will be identified, which is then selected as the best-performing model.

```
# Function to evaluate MiniBatch K-Means with given parameters
def evaluate_mini_batch_kmeans(n_clusters, batch_size):
    kmeans = MiniBatchKMeans(n_clusters=n_clusters, batch_size=batch_size, random_state=42)
    kmeans.fit(sample_data)
    labels = kmeans.labels_
    inertia = kmeans.inertia_
    silhouette_avg = silhouette_score(sample_data, labels)
    return inertia, silhouette_avg
```

```
# Iterate over all combinations of hyperparameters
results = []
for params in ParameterGrid(param_grid):
    inertia, silhouette_avg = evaluate_mini_batch_kmeans(params['n_clusters'], params['batch_size'])
    results.append({
        'n_clusters': params['n_clusters'],
        'batch_size': params['batch_size'],
        'inertia': inertia,
        'silhouette_score': silhouette_avg
    })
```

Hyperparameter Tuning Results:

n_clusters	batch_size	inertia	silhouette_score
0	8	2500 469711.605317	0.300774
1	9	2500 379938.362632	0.379254
2	**10**	**2500 310340.210693**	**0.444846**
3	11	2500 274801.426104	0.371199
4	12	2500 219857.909035	0.406109
5	13	2500 215916.766808	0.380570
6	14	2500 199221.095940	0.358980
7	8	5000 409451.556390	0.382095
8	9	5000 411422.138003	0.341890
9	10	5000 379436.434170	0.263443
10	11	5000 302836.902600	0.348156
11	12	5000 222776.043958	0.408720
12	13	5000 202193.519693	0.408727
13	14	5000 181309.369787	0.415075
14	8	10000 437028.899261	0.377095
15	9	10000 376117.832644	0.400098
16	10	10000 373554.159488	0.382885
17	11	10000 320588.212085	0.426532
18	12	10000 254206.875877	0.374085

19	13	10000	232658.342132	0.367176
20	14	10000	186590.371159	0.390223
21	8	15000	434058.457123	0.320510
22	9	15000	381126.493227	0.340847
23	10	15000	324524.654186	0.384960
24	11	15000	301825.852366	0.376150
25	12	15000	252615.570774	0.394755
26	13	15000	235057.030428	0.401925
27	14	15000	228041.984232	0.409512

```python
# Find the best parameters based on silhouette score (or inertia)
best_params = results_df.loc[results_df['silhouette_score'].idxmax()]
print("\nBest Parameters:")
print(best_params)
```

```
Best Parameters:
n_clusters              10.000000
batch_size            2500.000000
inertia             310340.210693
silhouette_score         0.444846
```

Key evaluation metrics:

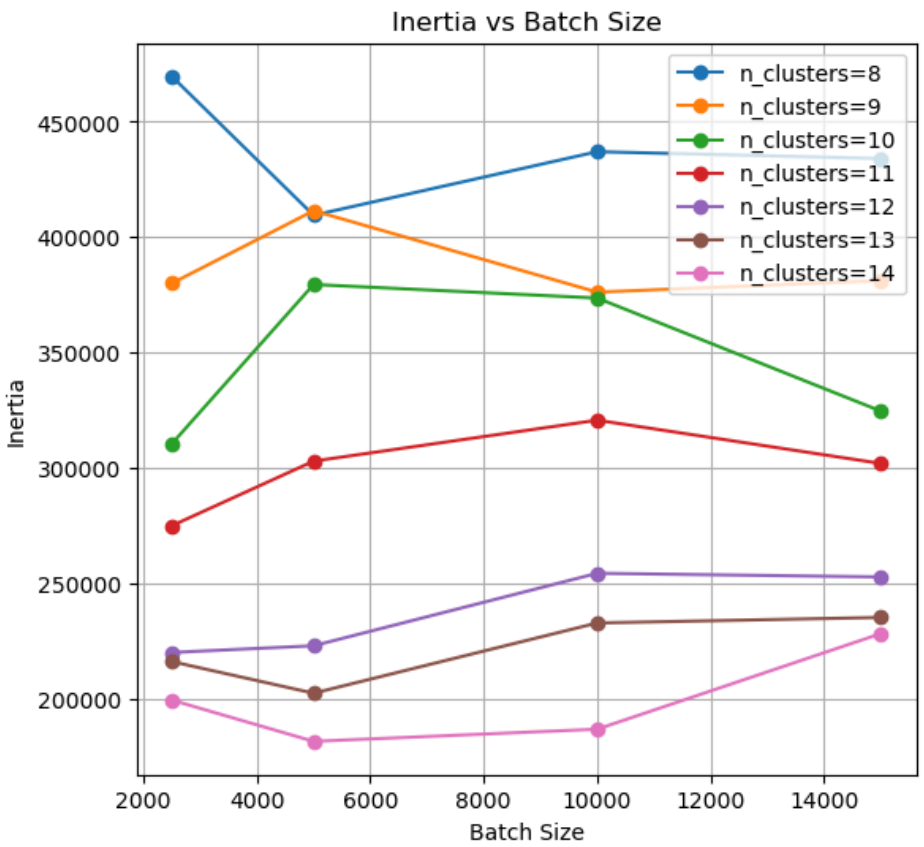
- **Inertia**: The sum of squared distances between each data point and its centroid, a lower inertia means that the clustering model is more compact and good.
- **Silhouette score**: This measures how similar a point is to its own cluster compared to other clusters. A higher score indicates better separation between clusters.

Based on the results, the best number of clusters is 10, while the best batch size is 2500, which yields the best balance between low inertia and high silhouette score.

4. Visualising Results

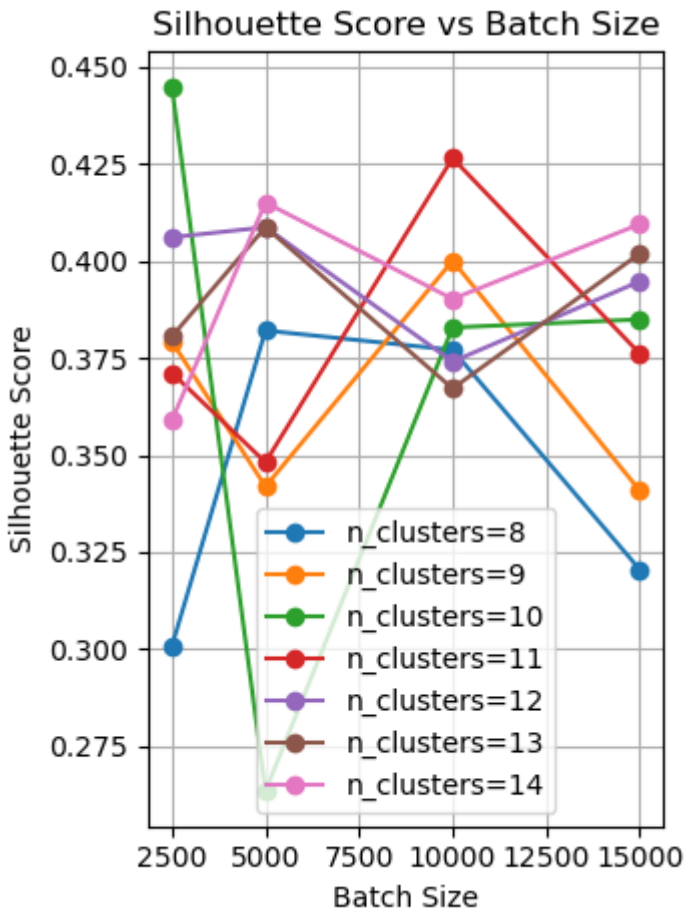
A) Line plot - Inertia vs Batch Size

- A line plot is created for each unique value of n_clusters, showing the inertia value changes with different batch_size.
- It allows us to compare how adding more clusters, ranging from 8 to 14 clusters, impacts inertia across different batch sizes, of 2500, 5000, 10 000 and 15 000.
- Inertia decreases consistently with higher n_clusters.
- For higher n_clusters values (12 to 14), the inertia remains more stable across different batch sizes. While there is a slight increase in inertia as batch_size grows, they have a lower inertia to form tight clusters and do not significantly worsen clustering quality.
- The lowest inertia is achieved with n_clusters=14 and a batch_size of 5000.



B) Line Plot - Silhouette Score vs Batch Size

- A line plot is created for each unique value of n_clusters, showing the silhouette score changes with different batch_size.
- The silhouette score fluctuates for different n_clusters values rather than steadily increasing or decreasing with more clusters.
- For some clusters, silhouette scores dip after batch sizes of 10,000, indicating that very large batch sizes might not improve cluster quality and could lead to less distinct clusters.
- The highest silhouette score is achieved with n_clusters = 10 at batch_size = 2500, with silhouette score: 0.445.



Implementation of Final MiniBatch K-Means Clustering Modelling

Best parameters from hyperparameter tuning:

Number of clusters, **n_clusters = 12** and Batch Size, **batch_size = 5000**

A) Training the Final KMeans Model

```
best_n_clusters = best_params['n_clusters']
best_batch_size = best_params['batch_size']

# Initialize and fit the model with the best parameters
final_kmeans = MiniBatchKMeans(n_clusters=int(best_n_clusters), batch_size=int(best_batch_size), random_state=42)
final_kmeans.fit(scaled_features_df)

# Obtain final cluster labels
final_labels = final_kmeans.labels_
centroids = final_kmeans.cluster_centers_
```

B) Cluster Distribution

```
# Create a DataFrame from scaled features with cluster labels
retails = pd.DataFrame(scaled_features_df, columns=features.columns)

cluster_labels = final_kmeans.labels_
cluster_distribution = pd.Series(cluster_labels).value_counts()
print("Cluster Distribution:\n", cluster_distribution)
```

```
Cluster Distribution:
 1    238967
2     59510
3     51207
7     47798
4     34108
6     32184
8     31572
5     25132
9     10288
0      9689
```

- Cluster 1 contains 238967 data points, making it the largest cluster while Cluster 0 is the smallest cluster with only 9689 data points.
- Cluster 1 holds a substantial portion of the dataset, representing the largest customer segment that dominates and shares general purchasing characteristics.
- Cluster 0 is a relatively small cluster that could possibly represent specialised and uncommon buying patterns.

C) Cluster Profile

A summary of the clusters was generated by aggregating key metrics across different customer segments.

Each cluster is profiled by calculating the mean values of these metrics, followed by rounding and formatting for better readability. The cluster profiles provide insights from different behaviour of each segment, helping to identify the types of products and time frames preferred by specific clusters.

```
retail['Cluster'] = final_minikmean.labels_
customer_clusters_minikmean = retail[['CustomerID', 'Cluster']].drop_duplicates()
```

```
cluster_profiles_minikmean =retail.groupby('Cluster').mean(numeric_only=True)
categories = ['Quantity', 'InvoiceMonth', 'Category_Christmas Items', 'Category_Decorative Items',
              'Category_Easter Items', 'Category_Fashion Acessories', 'Category_Gardening',
              'Category_Gift Items', 'Category_Household Items', 'Category_Kitchenware',
              'Category_Other', 'Category_Stationery', 'Category_Toys']

for category in categories:
    cluster_profiles_minikmean[category] = cluster_profiles_minikmean[category].round().astype(int)

# Formatting UnitPrice and TotalPrice to two decimal places
cluster_profiles_minikmean['UnitPrice'] = cluster_profiles_minikmean['UnitPrice'].round(2)
cluster_profiles_minikmean['TotalPrice'] = cluster_profiles_minikmean['TotalPrice'].round(2)

if 'Country_United Kingdom' in cluster_profiles.columns:
    cluster_profiles_minikmean['Country_United Kingdom'] = cluster_profiles_minikmean['Country_United Kingdom'].round()

print('Cluster Profile: ')
print(cluster_profiles_minikmean)
print(customer_clusters_minikmean.head(20))
```

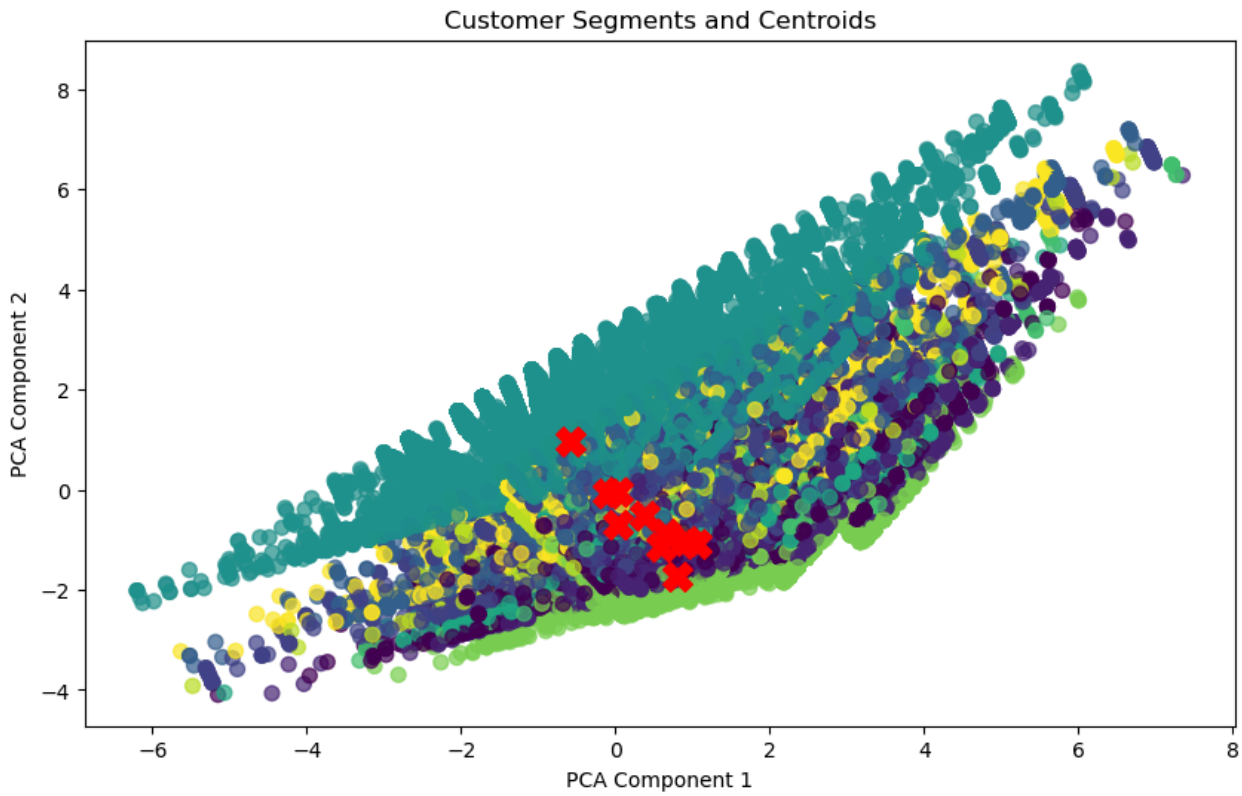
```
Cluster Profile:
         Quantity  InvoiceMonth  UnitPrice  TotalPrice  \
Cluster
0               9             7       2.01        9.17
1               6             7       3.23       13.00
2               6             7       3.09       13.10
3               7             7       2.45       13.44
4               5             8       3.82       16.36
5               8            10       2.17       12.85
6               6             7       3.82       14.64
7               7             8       2.35       11.76
8               8             8       1.56        7.82
9               5             8       3.03       11.87

         Category_Christmas Items  Category_Decorative Items  \
Cluster
0                               0                          0
1                               0                          0
2                               0                          0
3                               0                          0
4                               0                          0
5                               1                          0
6                               0                          0
7                               0                          1
8                               0                          0
9                               0                          0

         Category_Easter Items  Category_Fashion Acessories  \
Cluster
0                            0                            0
1                            0                            0
2                            0                            0
3                            0                            1
4                            0                            0
5                            0                            0
6                            0                            0
7                            0                            0
8                            0                            0
9                            0                            0

         Category_Gardening  Category_Gift Items  Category_Household Items  \
Cluster
0                         0                    1                         0
1                         0                    0                         0
2                         0                    0                         0
3                         0                    0                         0
4                         0                    0                         1
5                         0                    0                         0
6                         1                    0                         0
7                         0                    0                         0
8                         0                    0                         0
9                         0                    0                         0

         Category_Kitchenware  Category_Other  Category_Stationery  \
Cluster
0                           0               0                    0
1                           0               1                    0
2                           1               0                    0
3                           0               0                    0
4                           0               0                    0
5                           0               0                    0
6                           0               0                    0
7                           0               0                    0
8                           0               0                    1
9                           0               0                    0

         Category_Toys  Country_United Kingdom
Cluster
0                    0                     1.0
1                    0                     1.0
2                    0                     1.0
3                    0                     1.0
4                    0                     1.0
5                    0                     1.0
6                    0                     1.0
7                    0                     1.0
8                    0                     1.0
9                    1                     1.0
```

```
-                    -
   CustomerID  Cluster
0       17850        7
3       17850        4
4       17850        1
9       13047        1
12      13047        9
13      13047        2
15      13047        4
26      12583        1
30      12583        0
32      12583        8
33      12583        6
34      12583        4
38      12583        9
44      12583        7
46      13748        5
53      17850        2
65      15100        4
82      15291        4
83      15291        7
86      14688        3
```

D) PCA(Principal Component Analysis)

PCA helps to reduce the data to two components for visualisation in a 2D scatter plot that allows us to better understand the separation between clusters based on customer purchasing patterns.

```
pca = PCA(n_components=2)
reduced_pca = pca.fit_transform(scaled_features_df)

# Convert centroids to a DataFrame with the same columns as scaled_features_df
centroids_df = pd.DataFrame(final_kmeans.cluster_centers_, columns=scaled_features_df.columns)

# Transform centroids using PCA
centroids_pca = pca.transform(centroids_df)

plt.figure(figsize=(10, 6))
plt.scatter(reduced_pca[:, 0], reduced_pca[:, 1], c=final_labels, cmap='viridis', s=50, alpha=0.7)
plt.scatter(centroids_pca[:, 0], centroids_pca[:, 1], c='red', s=200, marker='X')  # Plot centroids
plt.title('Customer Segments and Centroids')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.show()
```



PCA Diagram Analysis:

1. Data Points (Coloured Dots)

   Each point represents a customer projected into the PCA reduced space. The colours of the points represent the different clusters assigned to each customer based on your clustering algorithm. Different colours indicate the grouping of data points into separate clusters.

2. Centroids (Red X's)

   The red crosses ('X') represent the centroids of the clusters. These centroids are the central points around which the clusters are formed. Each centroid is positioned at the geometric centre of the cluster which means it represents the average location of all points within that cluster in this reduced space.

3. Cluster Separation

   Similar to the standard KMeans model's PCA, there is some overlap which suggests that customers in these groups have similar characteristics.

4. Insights from the Plot

   The cluster represented by dark green seems to be the most distinct with little overlap which could imply a unique customer segment with clearly different behaviour compared to other clusters.

Comparison Between Difference Distance Metric

In the coding, it has included a compute_silhouette() function which is used to calculate the Silhouette Score for a given distance metric which is the euclidean distance metric, manhattan distance metric and cosine distance metric.

A) Euclidean Distance Metric

```
# Testing EUCLIDEAN DISTANCE METRIC
from sklearn.cluster import MiniBatchKMeans

# Sample the data that is too large
sample_size = min(50000, len(scaled_features_df))  # Sample of 50 000
sample_data = scaled_features_df.sample(n=sample_size, random_state=42)

# Test for Euclidean distance
k = int(best_n_clusters)
silhouette_scores = {}
euclidean_silhouette = compute_silhouette('euclidean', sample_data, k)
silhouette_scores['euclidean'] = euclidean_silhouette
print(f"Euclidean Distance - Silhouette Score: {euclidean_silhouette}")
```

```
Computing silhouette score for metric: euclidean
Completed metric: euclidean, Silhouette Score: 0.3719932764467731
Euclidean Distance - Silhouette Score: 0.3719932764467731
```

B) Manhattan Distance Metric

```
# Testing MANHATTAN DISTANCE METRIC

# Test for Manhattan distance
k = 12
manhattan_silhouette = compute_silhouette('manhattan', sample_data, k)
silhouette_scores['manhattan'] = manhattan_silhouette
print(f"Manhattan Distance - Silhouette Score: {manhattan_silhouette}")
```

```
Computing silhouette score for metric: manhattan
Completed metric: manhattan, Silhouette Score: 0.43599235225955124
Manhattan Distance - Silhouette Score: 0.43599235225955124
```

C) Cosine Distance Metric

```
# Testing COSINE DISTANCE METRIC

# Test for Cosine distance
k = 12
cosine_silhouette = compute_silhouette('cosine', sample_data, k)
silhouette_scores['cosine'] = cosine_silhouette
print(f"Cosine Distance - Silhouette Score: {cosine_silhouette}")
```

```
Computing silhouette score for metric: cosine
Completed metric: cosine, Silhouette Score: 0.515376855809705
Cosine Distance - Silhouette Score: 0.515376855809705
```

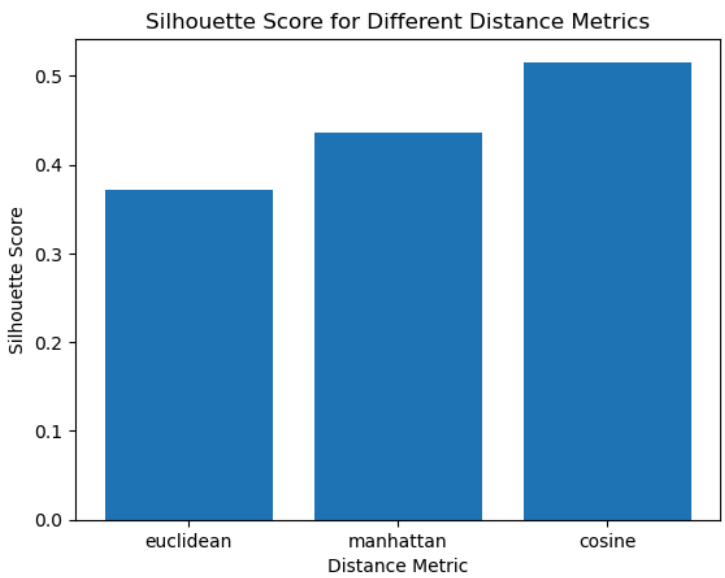
D) Determine Best Distance Metric

In the online retail datasets, the result below clearly indicates that Cosine distance metric is the best distance metric that can be used in Mini-batch K-Means clustering modelling because it produces the highest silhouette score.

```
# Find the best metric based on the highest silhouette score
best_metric = max(silhouette_scores, key=silhouette_scores.get)
best_score = silhouette_scores[best_metric]

# Print the best metric and its silhouette score
print(f"\nBest Distance Metric: {best_metric}")
print(f"Best Silhouette Score: {best_score}")

# Optional: Print all metrics and their scores for reference
print("\nAll Distance Metrics and Scores:")
for metric, score in silhouette_scores.items():
    print(f"{metric}: {score}")
```

```
Best Distance Metric: cosine
Best Silhouette Score: 0.515376855809705

All Distance Metrics and Scores:
euclidean: 0.3719932764467731
manhattan: 0.43599235225955124
cosine: 0.515376855809705
```

E) Visualisation in Bar Chart



To easily visualise the result of three different distance metrics, the bar chart has been used to show the silhouette score for Euclidean, Manhattan and Cosine distance metric.

Implementation of MiniBatch K-Means Clustering Modelling With Cosine Distance Metric

Number of clusters, **n_clusters = 10** and Batch Size, **batch_size = 5000** and **distance metric= cosine**

A) Normalisation of Data

```
sample_size = min(50000, len(scaled_features_df))  # Sample of 50 000
sample_data = scaled_features_df.sample(n=sample_size, random_state=50)
X_normalized = normalize(sample_data)
n_clusters = int(best_n_clusters)
```

F) Running MiniBatch K-Means with Cosine Distance Approximation

```
# Function to run MiniBatchKMeans with cosine distance approximation
def cosine_minibatch_kmeans(X, k, batch_size=5000):
    # MiniBatchKMeans with normalized data
    minibatch_kmeans = MiniBatchKMeans(n_clusters=k, batch_size=batch_size, n_init=10, random_state=42)
    labels = minibatch_kmeans.fit_predict(X)
    return labels, minibatch_kmeans

# Run MiniBatchKMeans
labels, cosine_minibatch_model = cosine_minibatch_kmeans(X_normalized, int(best_n_clusters))
```

G) Silhouette Score Calculation

```
silhouette_avg = silhouette_score(X_normalized, labels)
print(f'Silhouette Score: {silhouette_avg}')
```

```
Silhouette Score: 0.40625166982676225
```

A silhouette score of 0.406 is just a moderate clustering performance. The score suggests that the clusters are reasonably well-defined. The data points in each cluster are more similar to each other than they are to data points in other clusters, but there might be some overlap or less clear boundaries between clusters.

H) Cluster Distribution

```
sample_data['Cluster_MiniCosine'] = labels
cluster_distribution = sample_data['Cluster_MiniCosine'].value_counts().sort_index()
sorted_cluster_distribution = dict(sorted(cluster_distribution.items(), key=lambda item: item[1], reverse=True))
print("Cluster Distribution :")
for cluster, count in sorted_cluster_distribution.items():
    print(f"Cluster {cluster}: {count} ")
```

```
Cluster Distribution :
Cluster 3: 8941
Cluster 5: 6776
Cluster 0: 6242
Cluster 6: 5369
Cluster 8: 5125
Cluster 1: 4707
Cluster 2: 4352
Cluster 7: 3228
Cluster 4: 2908
Cluster 9: 2352
```

I) Cluster Profile

```
def cluster_profile(df, cluster_col):
    cluster_profiles = df.groupby(cluster_col).mean()  # Use .median() if preferred
    return cluster_profiles

# Get cluster profiles
cluster_profiles_minicosine = cluster_profile(sample_data, 'Cluster_MiniCosine')

customer_clusters_minicosine = retail[['CustomerID']].drop_duplicates().merge(
    sample_data[['Cluster_MiniCosine']],
    how='left',
    left_index=True,
    right_index=True
)

# Formatting UnitPrice and TotalPrice to two decimal places
cluster_profiles_minicosine['UnitPrice'] = cluster_profiles_minicosine['UnitPrice'].round(2)
cluster_profiles_minicosine['TotalPrice'] = cluster_profiles_minicosine['TotalPrice'].round(2)

print("\nCluster Profiles:")
print(cluster_profiles_minicosine)
print(customer_clusters.head(20))
```

Cluster Profiles:

Cluster_MiniCosine	Quantity	InvoiceMonth	UnitPrice	TotalPrice
0	1.280312	-0.083137	-0.49	0.79
1	0.046381	-0.083516	-0.22	0.03
2	0.089304	0.007025	-0.26	-0.08
3	-0.582958	-0.164298	-0.44	-0.52
4	0.235227	0.108995	-0.59	-0.28
5	-0.588424	0.158812	1.44	0.03
6	-0.036444	-0.129567	0.06	-0.02
7	-0.140719	0.183377	0.37	0.19
8	-0.073781	-0.176395	0.14	-0.00
9	0.273199	0.744806	-0.31	0.02

Cluster_MiniCosine	Category_Christmas Items	Category_Decorative Items
0	-0.220838	-0.309225
1	-0.220838	-0.311482
2	-0.220838	3.210462
3	-0.220838	-0.311482
4	-0.220838	-0.311482
5	-0.220838	-0.311482
6	-0.220838	-0.311482
7	-0.220838	-0.311482
8	-0.220838	-0.311482
9	4.528207	-0.311482

Cluster_MiniCosine	Category_Easter Items	Category_Fashion Acessories
0	-0.067261	-0.323519
1	-0.067261	3.091006
2	-0.067261	-0.323519
3	-0.067261	-0.323519
4	-0.067261	-0.323519
5	-0.067261	-0.323519
6	-0.067261	-0.323519
7	-0.067261	-0.323519
8	0.652524	-0.323519
9	-0.067261	-0.323519

Cluster_MiniCosine	Category_Gardening	Category_Gift Items
0	-0.251636	-0.13511
1	-0.251636	-0.13511
2	-0.251636	-0.13511
3	-0.251636	-0.13511
4	-0.251636	-0.13511
5	-0.251636	-0.13511
6	-0.251636	-0.13511
7	-0.251636	-0.13511
8	2.210363	1.21925
9	-0.251636	-0.13511

Cluster_MiniCosine	Category_Household Items	Category_Kitchenware
0	-0.257563	-0.343572
1	-0.259540	-0.351761
2	-0.259540	-0.351761
3	-0.259540	-0.351761
4	-0.259540	-0.351761
5	-0.259540	-0.351761
6	-0.259540	2.842842
7	3.852973	-0.351761
8	-0.259540	-0.351761
9	-0.259540	-0.351761

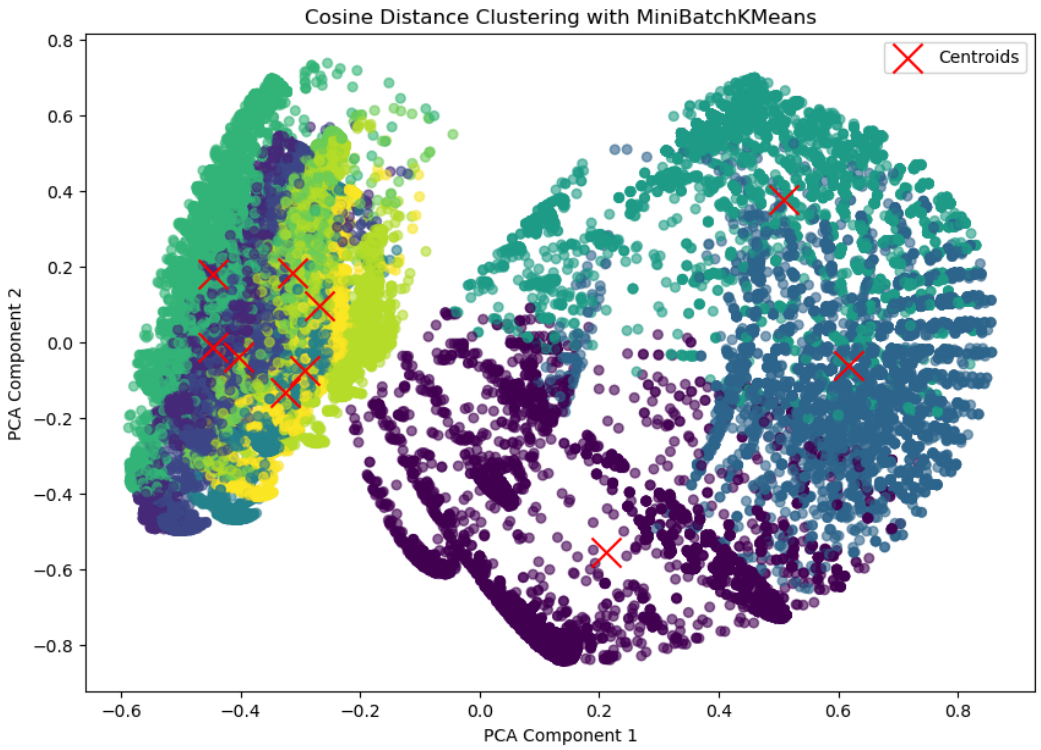
Cluster_MiniCosine	Category_Other	Category_Stationery	Category_Toys
0	1.126108	-0.249082	-0.139303
1	-0.882195	-0.249082	-0.139303
2	-0.882195	-0.249082	-0.139303
3	1.133536	-0.249082	-0.139303
4	-0.882195	4.014745	-0.139303
5	1.133536	-0.249082	-0.139303
6	-0.882195	-0.249082	-0.139303
7	-0.882195	-0.249082	-0.139303
8	-0.882195	-0.249082	1.247176
9	-0.882195	-0.249082	-0.139303

	CustomerID	Cluster
0	17850	0
3	17850	3
4	17850	5
9	13047	5
12	13047	9
13	13047	2
15	13047	3
26	12583	5
30	12583	7
32	12583	8
33	12583	10
34	12583	3
38	12583	9
44	12583	0
46	13748	1
53	17850	2
65	15100	3
82	15291	3
83	15291	0
86	14688	6

F) PCA Diagram

```python
# Visualizing the clusters using PCA for dimensionality reduction (optional, for 2D visualization)
def plot_clusters(X, labels, title="Clusters Visualization"):
    pca = PCA(n_components=2)
    reduced_data = pca.fit_transform(X)
    plt.figure(figsize=(10, 7))
    scatter = plt.scatter(reduced_data[:, 0], reduced_data[:, 1], c=labels, cmap='viridis', s=50, alpha=0.6)
    plt.title(title)
    plt.xlabel("PCA Component 1")
    plt.ylabel("PCA Component 2")
    plt.colorbar(scatter, label='Cluster Label')
    plt.show()
```

```python
# Plot the clusters (optional, if you want to visualize)
plot_clusters(X_normalized, labels, title="Cosine Distance Clustering with MiniBatchKMeans")
```



PCA Diagram Analysis

This PCA graph shows the results of clustering using MiniBatchKMeans with the Cosine Distance metric.

1. Data Points

   Each dot represents a data point (customer) in the reduced dimensional space. The different colours represent the cluster labels. Purple represents cluster labels close to 0. Yellow represents higher cluster labels. This colour scheme helps visually differentiate between different clusters and understand how well-separated they are.

2. Cluster Separation

   The plot reveals three main clusters (one on the left, one in the middle, and one on the right):

   - The **left cluster** contains mostly light green and teal points, corresponding to clusters with mid-range labels.
   - The **middle cluster** has points that transition from dark purple at the bottom to lighter colours (green/yellow) at the top, showing a gradient.
   - The **right cluster** primarily consists of yellow points (clusters with higher labels).

   The **clusters** seem fairly well-separated in this 2D projection, meaning the model was able to distinguish distinct groups based on cosine distance.

3. Insights
   - The clusters on the left and right are clearly separated and showing distinct patterns in the data. These might represent distinct customer segments or product categories.
   - The middle cluster with a gradient of colour suggests that these data points are more complex which means that they might have mixed characteristics of both surrounding clusters.
   - The clear segmentation based on cosine distance is promising and may suggest that the MiniBatchKMeans algorithm was able to detect meaningful groupings in the data.

E. Evaluation

Review of Process

1. **Data Cleaning and Preparation:**
   - To ensure the accuracy, the dataset which included consumer transaction records was properly cleaned. Any missing values, duplicates, or incorrect entries were fixed to ensure data integrity. This step was essential in order to avoid errors and noise that could lower the quality of the clustering results.

   - During data transformation, the derived *Category* feature was introduced to group products, simplifying customer segmentation. The *TotalPrice* (Quantity x Unit Price) became key for identifying premium buyers and bulk purchasers. Time-based data, like *InvoiceMonth*, provided insights into seasonal buying patterns, such as holiday-driven purchases. These transformations enhanced the segmentation process by capturing both financial and customer behaviour.

   - Numeric features like *TotalQuantity, Quantity, InvoiceMonth, UnitPrice* were scaled and standardised to facilitate the K-means clustering model. Binary features like *Country_United Kingdom* and *product categories* were left unchanged as they did not require scaling. One-Hot encoding was applied to categorical variables like product categories, ensuring compatibility with the clustering model.

2. **Feature Selection**
   - With the intention of dividing up the client base according to their purchase patterns, careful consideration was taken in selecting the variables that would be used for clustering. *CustomerID, Country, Quantity, UnitPrice, InvoiceMonth, and TotalPrice* were critical variables. These played a key role in classifying consumers into discrete groups, such as those who purchased various kinds of goods.

   - This decision to employ these basic features guaranteed that the segmentation was comprehensible and in line with business goals, including recognising high-value customers or understanding regional purchasing patterns.

- **Areas that needed improvement** were identified to improve model granularity in future iterations, which could increase the effectiveness of targeted marketing campaigns, more behavioural elements might be added:
  i) **Purchase frequency**, providing information on recurrent purchasing trends.
  ii) **Browsing history of the customer**, which would reveal which product categories they had been looking at but did not buy.

3. **Modelling**

- The elbow method and silhouette scores were used to validate the optimal number of clusters. Using the elbow technique, the ideal number of clusters was found by tracking changes of inertia as the number of clusters increased, while silhouette scores evaluated how well-separated and cohesive the clusters were. Ordinary KMeans had a silhouette score of 0.474, while MiniBatchKMeans achieved better with a score of 0.445.

- Hyperparameter tuning is done for each modelling (Kmeans, MiniBatchKmeans) to get the optimal performance.

- In addition distance metrics were tested for both Kmeans, and MiniBatchKmeans. MiniBatchKMeans model based on Cosine distance was chosen to be modelled since it is time efficient and provides the highest silhouette score as compared to the Euclidean and Manhattan.

- Each modelling provides results of the cluster distribution, cluster profiles and the cluster labelling for each customer (based on their customer ID) as well as the PCA diagram to visualise the clusters distribution for better results analysis in order to gain useful insights.

- Standard KMeans was selected due to its highest silhouette score as compared to the others which provides better cluster separation, and the creation of 11 distinct clusters that capture a broad range of customer behaviours. The model's cluster profiles offer more actionable segmentation across product categories.

Model Selection

There are a total of 4 models being done. The selection criteria can be done from the aspect of silhouette score, number of clusters, cluster profiles and cluster distributions.

	KMeans	Cosine Kmeans	Minibatch Kmeans	Cosine Minibatch Kmeans
Silhouette score	0.474	0.409	0.445	0.406
Number of clusters	11	11	10	10

Selected model: K Means

Justification:

1. **Highest Silhouette Score**

   KMeans gives the highest silhouette score of 0.4743, providing better cluster separation compared to other models. This means that the clusters formed are more distinct and better represent differences in customer behaviour.

2. **11 Clusters**

   It offers a rich segmentation of the customers, which can capture a broad range of behaviours and spending habits. This is very useful when we need diverse segments for actionable insights such as targeted marketing, product recommendations, or customer analysis.

3. **Better Cluster Profiles**

   The cluster profile from KMeans modelling offers more specific category clusters, which is highly actionable for business segmentation and targeting strategies since it gives clearer segmentation across product categories unlike MiniBatch KMeans which has broader clusters where the model shows less variety in category clustering.

4. **Cluster Distribution**

   The KMeans model produces a cluster distribution that is skewed, but still offers a broad range of customer segmentation. The largest cluster (Cluster 5) has 236,533 data points, but this is offset by smaller clusters that help capture more variety of customer behaviours. The smallest cluster (Cluster 4) has 2,434 points, ensuring that even rare spending patterns are identified.

Model Analysis

Model: **K-means clustering**

The model has segmented customers into **11 clusters** ranging from **Cluster 0 to Cluster 10**.

Model Interpretation:

1. **Cluster Distribution Analysis**

```
Cluster Distribution :
Cluster 5: 236533
Cluster 2: 59510
Cluster 6: 51207
Cluster 0: 47798
Cluster 3: 34108
Cluster 10: 32184
Cluster 8: 31572
Cluster 1: 25132
Cluster 9: 10288
Cluster 7: 9689
Cluster 4: 2434
```

Cluster 5: 236,533 transactions (~44% of total)
Cluster 2: 59,510 transactions
Cluster 6: 51,207 transactions
Cluster 0: 47,798 transactions
Cluster 3: 34,108 transactions
Cluster 10: 32,184 transactions
Cluster 8: 31,572 transactions
Cluster 1: 25,132 transactions
Cluster 9: 10,288 transactions
Cluster 7: 9,689 transactions
Cluster 4: 2,434 transactions

The cluster distribution shows that **Cluster 5 has the highest number of data points** (236533) making it the most prominent cluster and common pattern in the retail datasets. Therefore, cluster 5 could be the primary cluster for major customer segments or product trends.

Cluster 4 has the smallest number of data points (2434 points) indicating it may be a smaller or less frequent pattern in the retail datasets. It could be explored for specialised insights or marketing strategies.

The **distribution is quite skewed** with a few clusters having significantly more data points than others. For instance, Cluster 5 alone has a much larger count compared to Cluster 7, 9 and 4 combined. Thus, this imbalance might indicate that the retail datasets have a few dominant patterns or features that are stronger, influencing cluster formation and making some clusters more populated than others.

2. Cluster Profile Analysis

Cluster Profile:

Cluster	Quantity	InvoiceMonth	UnitPrice	TotalPrice
0	7	8	2.35	11.76
1	8	10	2.17	12.85
2	6	7	3.09	13.10
3	5	8	3.82	16.36
4	6	3	2.08	9.44
5	6	7	3.24	13.03
6	7	7	2.45	13.44
7	9	7	2.01	9.17
8	8	8	1.56	7.82
9	5	8	3.03	11.87
10	6	7	3.82	14.64

Cluster	Category_Christmas Items	Category_Decorative Items
0	0	1
1	1	0
2	0	0
3	0	0
4	0	0
5	0	0
6	0	0
7	0	0
8	0	0
9	0	0
10	0	0

Cluster	Category_Easter Items	Category_Fashion Acessories
0	0	0
1	0	0
2	0	0
3	0	0
4	1	0
5	0	0
6	0	1
7	0	0
8	0	0
9	0	0
10	0	0

Cluster	Category_Gardening	Category_Gift Items	Category_Household Items
0	0	0	0
1	0	0	0
2	0	0	0
3	0	0	1
4	0	0	0
5	0	0	0
6	0	0	0
7	0	1	0
8	0	0	0
9	0	0	0
10	1	0	0

Cluster	Category_Kitchenware	Category_Other	Category_Stationery
0	0	0	0
1	0	0	0
2	1	0	0
3	0	0	0
4	0	0	0
5	0	1	0
6	0	0	0
7	0	0	0
8	0	0	1
9	0	0	0
10	0	0	0

Cluster	Category_Toys	Country_United Kingdom
0	0	1.0
1	0	1.0
2	0	1.0
3	0	1.0
4	0	1.0
5	0	1.0
6	0	1.0
7	0	1.0
8	0	1.0
9	1	1.0
10	0	1.0

Summarise results:

Cluster	Avg. Qty	Avg. Unit Price (£)	Avg. Total Price (£)	Invoice Month	Key Categories	Country
0	7	2.35	11.76	8	Decorative items	UK
1	8	2.17	12.85	10	Christmas items	UK
2	6	3.09	13.10	7	Kitchenware	UK
3	5	3.82	16.36	8	Household items	UK
4	6	2.08	9.44	3	Easter items	UK
5	6	3.24	13.03	7	Others (Varied)	UK
6	7	2.45	13.44	7	Fashion Accessories	UK
7	9	2.01	9.17	7	Gift Items	UK
8	8	1.56	7.82	8	Stationery	UK
9	5	3.03	11.87	8	Toys	UK
10	6	3.82	14.64	7	Gardening items	UK

Observations:

- **Cluster 0:**
  - This cluster features mid-range quantities (7) and pricing (2.35, 11.76).
  - The focus on **decorative items** suggests that customers in this group prefer small, aesthetically pleasing items like home decorations, which are typically purchased for home improvement or gifting purposes.

- **Cluster 1**:
  - The higher quantity (8) and unit price (2.17, 12.85) in this cluster align with seasonal purchasing behaviour, likely driven by preparations for the holiday season.
  - High emphasis on **Christmas Items**, highlighting potential for **holiday promotions**, which also explains the **October** purchasing periods before the upcoming Christmas holiday.

- **Cluster 2**:
  - Higher unit prices (3.09) while lower quantity (6) with an average total price of (13.10) suggest that this cluster of transactions might be higher-quality products that are more expensive.
  - This cluster focuses on **kitchenware** which also holds the second largest cluster distribution.

- **Cluster 3**:
  - Highest total price per transaction (16.36) with a focus on **Household Items**. Represents a valuable segment for higher-priced household goods.
  - Household items tend to be priced higher than other products, and customers are purchasing fewer (avg qty 5) but higher-priced items (3.82).

- **Cluster 4 (Smallest Cluster)**:
  - Small segments for **Easter Items**, suggesting opportunities for **seasonal campaigns** during the Easter period during **March**, which is reasonable as Easter day normally lies at the end of March.
  - Lower unit prices and quantities align with typical consumer behaviour during smaller seasonal events compared to major holidays such as Christmas. It also holds the smallest distribution of transactions.

- **Cluster 5 (Largest Cluster)**:
  - **General purchases** across categories, making it ideal for broad marketing strategies. It shows a moderate unit price (3.24) and total price (13.03).
  - This cluster represents a more generalised purchasing pattern with **varied product categories**.

- **Cluster 6**:
  - This cluster is characterised by a focus on **accessories** such as jewellery or small **fashion** items.
  - These items typically have mid-range pricing and appeal to customers interested in personal or gift purchases.

- **Cluster 7**:
  - High average quantity (9) and lower unit price (2.01) with a focus on **Gift Items**, suggest customers are buying small, affordable items, likely for gifting purposes.
  - Gift items often have wide appeal and are commonly high volume of low-cost purchases, especially for events or celebrations.

- **Cluster 8**:
  - This cluster reflects customers interested in **stationery** or small office supplies, which typically have lower average unit prices (1.56) and are bought in higher quantities (9).
  - Stationery purchases are common for office work, school, or personal use.

- **Cluster 9**:
  - **Toys** often fall into the higher price range (3.03), but customers purchase fewer units (5).
  - The average total price (11.87) reflects small purchases, possibly for children's birthdays or seasonal gifts.

- **Cluster 10**:
  - High spending on **Gardening Items**, making it a good target for **seasonal promotions** related to gardening.
  - It has the highest average total price (14.64) representing customers purchasing gardening items at a higher price.

3. Customer Data Analysis

	CustomerID	Cluster
0	17850	0
3	17850	3
4	17850	5
9	13047	5
12	13047	9
13	13047	2
15	13047	3
26	12583	5
30	12583	7
32	12583	8
33	12583	10
34	12583	3
38	12583	9
44	12583	0
46	13748	1
53	17850	2
65	15100	3
82	15291	3
83	15291	0
86	14688	6

- **Diverse Buying Behaviour**

  Customers like 17850, 13047 and 12583 were labelled with multiple clusters, which reflects the variety in purchasing behaviour across different product types. Such customers may have a wider range of interests or shopping needs.

- **Focused Shoppers**

  Some customers like 13748 assigned to Cluster 1 - Christmas Items, and 15100 assigned to Cluster 3 - Household Items are assigned to specific clusters, showing specialised preference for certain products. These customers might focus on fulfilling particular needs rather than buying across multiple categories.

Evaluation of Results

Primary Goal / Business Objectives:

To identify customer segments based on purchasing behaviour in order to optimise decision making for marketing strategies, inventory management, sales growth, and targeted promotions.

By using K-Means clustering, we have successfully segmented customers into distinct groups in order to identify high-spenders, product category preferences, time-based, or region-specific purchasing patterns. The modelling process was carefully handled and tuned so that it gets the optimal performance as good as possible.

Model Performance: The model groups customers into 11 clusters (K-means) to 10 clusters (MiniBatch K-Means) based on key features such as "*Quantity", "InvoiceMonth", "UnitPrice", 'TotalPrice*", product categories (Christmas Items, Decorative Items, Easter Items, Fashion Accessories, Gardening, Gift Items, Household Items, Kitchenware, Stationary, Toys, and Others), and "*Country_United Kingdom*".

Cluster Insights:

The clustering results provide clear customer segments, which achieved the main goal of identifying customer segmentation based on their purchasing behaviours to improve marketing strategies and resource allocation. Further cluster selection and analysis is written in the section below.

Does the model meet business goals?

1. **Identify distinct customer segments**

   The model successfully grouped customers into 10 to 11 distinct clusters, each demonstrating unique purchasing behaviours and preferences, such as high-volume buyers of seasonal buyers. The distribution of cluster profiles also gives us useful information about the major customer group and the segments that have less proportion which can further gain insights into how different groups contribute to overall sales.

2. **Personalise marketing strategies**

   Each cluster focuses on specific product categories which allows the retailer to develop marketing campaigns, such as having special offers for high-spender customers or holiday-related promotions based on the seasonal features obtained.

3. **Optimise inventory management**

   The cluster distribution and the product categories, as well as the invoice month in each cluster highlight the need to align inventory with demand from the key clusters. This could prevent problems of inventory such as lack of stock by preparing enough product under the category that has major customer demand to ensure the stock level especially during peak seasons, and overstock by reducing the inventory storage for certain products that have less demand.

4. **Enhance customer engagement and sales**

   With detailed customer profiles, the business can focus on high engagement clusters with more general campaigns, while designing targeted offers for smaller segments. This segmentation enables the business to tailor promotions, boost customer satisfaction, and potentially increase sales conversion rates through personalization.

Assessment of Data Mining Results

1. **Model Performance and Insights**

   a. **Clustering for Customer Segmentation**

      By using K-means clustering, the clustering model achieved a silhouette score of 0.474, optimal number of customer segments was determined to 11. This silhouette score indicates that the clusters are well-separated and the data points are assigned within each of their respective clusters.

      These insights make us customised marketing efforts and retention strategies which focus on these high-value segments. It emphasises the opportunities for promotions and engagement marketing strategies by referring to the clustering of additional segments of infrequent buyers or regionally concentrated customers.

   b. **Product Category**

      Products in the category 'Christmas Gift' (Cluster 1) consistently sold out during the peak seasons. Meanwhile, products in the category 'Variety/Others' (Cluster 5) are the largest product category that customers generally purchase , remaining understocked, leading to missed sales opportunities. A complete inventory management system will help balance supply and demand.

c. **Geographical and Seasonal Trends**

In monthly sales analysis, it shows that UK customers are most active during the Christmas period for buying Christmas gifts, while international customers usually continue buying throughout the year. Marketing should focus on the peak season for domestic customers and also maintaining a steady international presence.

2. **Business Impact and Return on Investment (ROI)**

a. **Customer Segmentation:** Cluster 10 (high-spending customers) shows a high potential for increased customer retention through personalised marketing efforts by targeting this group.

b. **Product Performance:** Product Category varied contributes 44% of total sales, while product Category easter items underperforms with less than 1%, indicating opportunities for rebalancing inventory management.

c. **Geographical/Seasonal Trends:** Sales peak during November only in the UK (December 2010 to November 2011) where sales exceeded 700 000 units (*data found during EDA phase*), with international purchases remaining stable year-round. Focused marketing during these periods is projected to increase revenue.

In summary, the data mining results meet the primary business objectives of determining key customer segments, know the product performance and revealing geographic and seasonal trends while providing additional deep insights that can further refine business strategies and the current analysis provide a strong basis for data-driven decision-making. The insights gained will be valuable in developing future marketing strategies, improving inventory management, and driving sales growth.

Possible Actions and Decisions

1. **Targeted Marketing Campaigns**

   Start targeted advertising campaigns aimed at certain groups of customers to optimise interaction and revenue.

   - Based on the cluster profiles, launch targeted campaigns for relevant customer segments. For example, create seasonal promotions in October and November that highlight Christmas-related products from Cluster 1. Promotions for gift items might also be directed towards Cluster 7 clients.
   - Cluster 1: Focus on Christmas merchandise, suggesting a seasonal trend in consumer behaviour.
   - Cluster 7: Shows a high amount of gift item purchases, which makes them suitable for marketing focused on celebrations and events.

   Schedule promotions to align with purchasing peaks and make sure the relevant customers notice them. In order to prevent overspending on specific campaigns, clusters with smaller customer bases should pay special attention to Return on Investment (ROI).

2. **Inventory Management Optimization**

   Stock management should be in line with the major consumer segments' purchase habits in order to increase stock availability and reduce over or understocking.

   - Adapt inventory management to main clusters' purchase patterns, highlighting to supply high-demand items for the most active segments. For example, the largest cluster, Cluster 5.
   - Cluster 3 and Cluster 5: Shows great demand for household items.
   - It is possible to avoid losing out on potential sales by keeping household items' stock levels constant throughout the entire year.
   - Seasonal Clusters: Specialised inventory management techniques to prevent stockouts during the peak buying period and avoid overstocking after the season ends for clusters like Cluster 1 (Christmas) and Cluster 4 (Easter).

3. **Specialised Marketing Tactics for Smaller Clusters**

   Provide targeted offers and individualised marketing for smaller clusters that are concentrated on particular product categories.

   - Create targeted marketing campaigns for smaller clusters, like Cluster 4, which buys Easter-related products. Toy advertisements could also be directed towards customers labelled with Cluster 9, especially around the holidays.
   - Cluster 4: This small cluster is a great target market for Easter-related specialised promotions because it shows a targeted interest in products relevant to the occasion. Offering exclusive discounts or packages could boost sales to this group.
   - Cluster 9: This segment's customers are primarily focused on buying toys. Holiday promotions or parent-focused marketing may boost involvement from this particular group of customers.
   - Even though they might not make purchases as frequently, small clusters of customers can represent profitable client segments. Their clear product preferences enable highly targeted campaigns that might produce positive outcomes when combined with a comprehensive strategy.

   Campaigns for a small audience may have a lower return than those in larger segments. To guarantee a beneficial effect on sales, it's critical to figure out the financial value of such tactics and carefully manage resources.

4. **Customer Retention and Loyalty Programs**

   Set up loyalty programmes or customised suggestions and exclusive deals to keep these clients.

   - Clusters that show higher spending per transaction, such as Cluster 3 that focuses on household items and Cluster 10 that focuses on gardening items, indicate important client categories that might generate higher revenue through targeted loyalty programs.
   - Provide exclusive deals, discounts, or early access to new products for customers who spend a considerable high amount on each purchase to maintain their loyalty.

   Monitor customer satisfaction in order to make sure that loyalty programs offer relevant benefits that encourage customers to make further purchases and stay engaged.

5. **Offer Diversify Products**

   Provide a broad range of product categories for clusters who buy varied items.

   - These clients in Cluster 5, where they make general purchases across multiple categories, can be attracted by the availability of products sold.
   - Customers in Cluster 5 may respond positively to cross-category promotions or product recommendations based on their diverse shopping patterns across a number of product categories.

   It is important to understand customer preferences and ensure that the promotions offered resonate with this broad range of interests within this cluster in order to achieve profitable conversions.

Determine Next Steps

To guarantee the model works effectively and continues to develop, a number of key subsequent actions are proposed based on the findings of the customer segmentation research. In addition to improving marketing tactics, inventory control, and customer interaction, these steps concentrate on utilising the clustering insights to enhance the model for subsequent iterations.

1. **Deployment of K-Means Model**

   Apply the chosen K-Means clustering model into practical applications. This involves applying the concept to **customised email marketing** efforts that are aimed at particular clusters. Implement cluster purchase behaviour to modify stock levels to improve **inventory forecasting**. In order to make sure the approaches continue to be in line with consumer behaviour and market demand, the model's implementation should be continuously reviewed in order to assess its effects on sales, customer engagement, and overall business production.

2. **Development of Targeted Advertising Plans**

   Make use of each cluster's unique profile while developing marketing strategies. Promotional content that is in line with these customer preferences can boost engagement and increase sales rates.

3. **Further Model Iterations and Refinement**

   The model can be improved by including features like **customer searching history and purchase frequency** to increase the accuracy and granularity of customer segmentation. The model will be able to generate more comprehensive client behaviour profiles through to these extra behavioural data, which will result in better marketing strategies.

4. **Track Customer Behaviour and Cluster Progression**

   Continually **tracking the change** of clusters is necessary to keep strategies up to date and in line with changing consumer preferences. Frequent model updates that include new behavioural and sales data will make it easier to monitor shifts in consumer behaviour or the formation of new trends. The company may remain flexible and modify its strategies to seize new possibilities and trends as they arise by closely monitoring cluster behaviours. This will eventually guarantee a dynamic and responsive marketing and inventory plan.

5. **Optimisation of Inventory Management**

   Optimise inventory management by taking into account the buying habits of the various consumer categories. Restock frequently purchased items on a regular basis for clusters with extensive and regular buying habits. Use strategic stocking and procurement for clusters with seasonal demand. The retail store may cut expenses, increase stock selling rate, and make sure that popular products are available when demand is at its peak by modifying inventory levels in response to these findings. This will ultimately increase sales and improve customer satisfaction.

F. Deployment

Objective: To integrate the clustering results into the operational workflows of the online retail business to enhance customer retention, increase sales, and optimise inventory management.
Insights gained from customer segmentation, product category analysis, and regional or seasonal trends will be deployed to guide decision making in marketing, sales, and inventory strategies.

Deployment Plan

1. **Data Integration**

   The clusters identified from the modelling and analysis on customer spending habits can be integrated into the Customer Relationship Management (CRM) system, which helps manage customer data, provides actionable information that assists sales management. CRM system offers analytical, reporting capabilities and dashboard customisation that further helps in recognition of sales patterns for strategic plannings (*What Is a CRM System? Everything You Need to Know*, n.d.).

2. **Automated Recommendations**

   Each customer can be automatically tagged with their corresponding cluster label, enabling targeted marketing and loyalty programs. An algorithm can be built to dynamically recommend products based on customers' segments, aligning with their purchase behaviours and preferences (Involve.Me, 2023). The automated product recommendation system helps enhance customer experience as they are able to find things that match their interests and personalised suggestions, leading to repeat purchase in the future.

   For example, customers who frequently purchase stationery products (Cluster 8) will receive automated recommendations on items such as notebooks, card envelopes, pen holders etc.

3. **Targeted Marketing Campaigns**

- **Seasonal Promotions**

  Cluster-specific marketing campaigns, such as Christmas or Easter promotions for Clusters 1 and 4, will be deployed during their respective seasons. Marketing automation tools can be set to trigger promotions based on the purchasing peaks identified in the dataset.

- **Personalised email campaigns**

  After using transaction data to segment customers based on their purchase history and preferences, this valuable insight can be used in personalised email generating systems with product recommendations, special offers, and discounts.

  For example, after purchasing kitchen utensils, these customers (Cluster 2: Kitchenware buyers) will receive emails suggesting matching cutlery sets, mugs and cups, or oven gloves.

- **Social media campaigns**

  Use social media platforms to showcase and run targeted ads to a specific group of customers that is segmented, and engage with the targeted audience through interactive content like polls and giveaways for hot selling items during specific seasons.

4. **Inventory Optimisation**

   The inventory management system will be updated to reflect the insights on best-seller and underperforming categories. This will help optimise stock levels and promotions for high-demand products (e.g., major clusters of decorative items, household goods).

   The inventory levels are aligned with demand patterns by continuously updating stock based on real-time sales data and seasonal forecasts.

   Seasonal stock will be adjusted to reflect seasonal peaks, ensuring sufficient availability for Christmas items in Cluster 1 around December or Easter items in Cluster 4 around March. Precise forecasting techniques will be implemented to avoid over- or under-stocking.

5. **Customer Retention and Loyalty Programs**

   Loyalty programs will target customers with high spending per transaction, such as those in Clusters 3 and 10. CLV, Customer Lifetime Value can also be calculated and integrated to gain knowledge on which customer contributes most to the business. These programs may offer exclusive discounts, such as 20% off on premium items, early access to new products, or personalised recommendations, or even bonus points accumulation for reward giving, which effectively retain customers to repeatedly purchase or spend more.

Monitoring and Maintenance Plan

1. **Operational Maintenance**
   - As the volume of data such as purchase transactions and customer data keeps growing, data pipelines can be considered. Data pipeline is a systematic process used in computing environments for the effective transformation, management and transfer of data, allowing seamless flow of information during data preprocessing (Hazelcast, 2024).
   - The deployed system should be scale effectively to accommodate an increasing number of transactions and customers. This may involve optimising storage and computational resources for faster processing.

2. **Cluster Model Updates**

   Customer purchase behaviours and trends may change over time. It is crucial to regularly update the clustering model with the latest data, specifically every quarter of the year in order to retain the clustering algorithm and model, ensuring that it provides accurate results with the current trends.

3. **Monitoring Marketing Effectiveness**

   The Return on Investment, ROI, can be used to measure whether a campaign conducted successfully for each customer segment and adjust strategies accordingly. It measures the money spends on marketing campaigns against the revenue those campaigns generate (*What Is Marketing ROI?*, n.d.).

4. **Monitoring Inventory**

   Implement a real-time monitoring system to track inventory levels, adjusting based on purchasing trends and ensuring optimal stock levels year-round. The efficiency of inventory optimization strategies can be evaluated by measuring the selling rate for best-selling products and monitoring stock availability for seasonal items.

5. **Feedback**

   Customer: Surveys can be sent to ask about product recommendations to them and the loyalty programs to gather insights on their experience. This can help in refining the segmentation strategy and loyalty rewards based on direct customer feedback.

   Internal staff: Store managers and marketing teams can provide regular feedback on the effectiveness of inventory management and marketing strategies through quarterly surveys.

G. Dashboard/Visualization/Presentation

1. **Customer Cluster Dashboard**

   Input: Customer ID

   Output: Scatter plot showing which clusters the customer belongs to, following with the cluster's details.

   In the example provided, Customer ID *17850* is associated with clusters 0, 2, 3, 5, 10 which are stated and visualised in a colour-labelled plot. Each cluster is represented by a different colour on the scatter plot, giving a quick overview of how the customer fits into different clusters.

   Purpose / Applications:

   - Display customer segments based on purchasing patterns and product preferences
   - Automated personalised product recommendations that suggest relevant products to specific customers based on their past behaviours

Cluster 0 Profile:

- Quantity: 7.0
- InvoiceMonth: 8.0
- UnitPrice: 2.35
- TotalPrice: 11.76
- Category_Decorative Items: 1.0
- Country_United Kingdom: 1.0

Cluster 2 Profile:

- Quantity: 6.0
- InvoiceMonth: 7.0
- UnitPrice: 3.09
- TotalPrice: 13.1
- Category_Kitchenware: 1.0
- Country_United Kingdom: 1.0

Cluster 3 Profile:

- Quantity: 5.0
- InvoiceMonth: 8.0
- UnitPrice: 3.82
- TotalPrice: 16.36
- Category_Household Items: 1.0
- Country_United Kingdom: 1.0

Cluster 5 Profile:

- Quantity: 6.0
- InvoiceMonth: 7.0
- UnitPrice: 3.24
- TotalPrice: 13.03
- Category_Other: 1.0
- Country_United Kingdom: 1.0

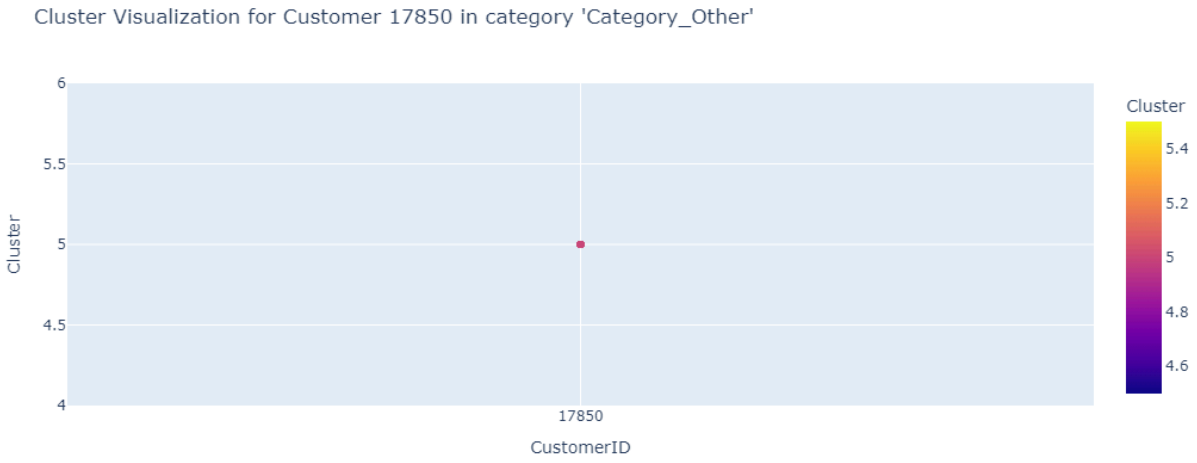
Cluster 10 Profile:

- Quantity: 6.0
- InvoiceMonth: 7.0
- UnitPrice: 3.82
- TotalPrice: 14.64
- Category_Gardening: 1.0
- Country_United Kingdom: 1.0

Users also can choose to focus on which categories and the dashboard will show the result whether they have found the categories in the cluster result or categories not included in the cluster result before.

Other

Show Cluster Info



Customer 17850 belongs to the following clusters in category 'Category_Other': 5

Cluster 5 Profile:

- Quantity: 6.0
- InvoiceMonth: 7.0
- UnitPrice: 3.24
- TotalPrice: 13.03
- Category_Other: 1.0
- Country_United Kingdom: 1.0

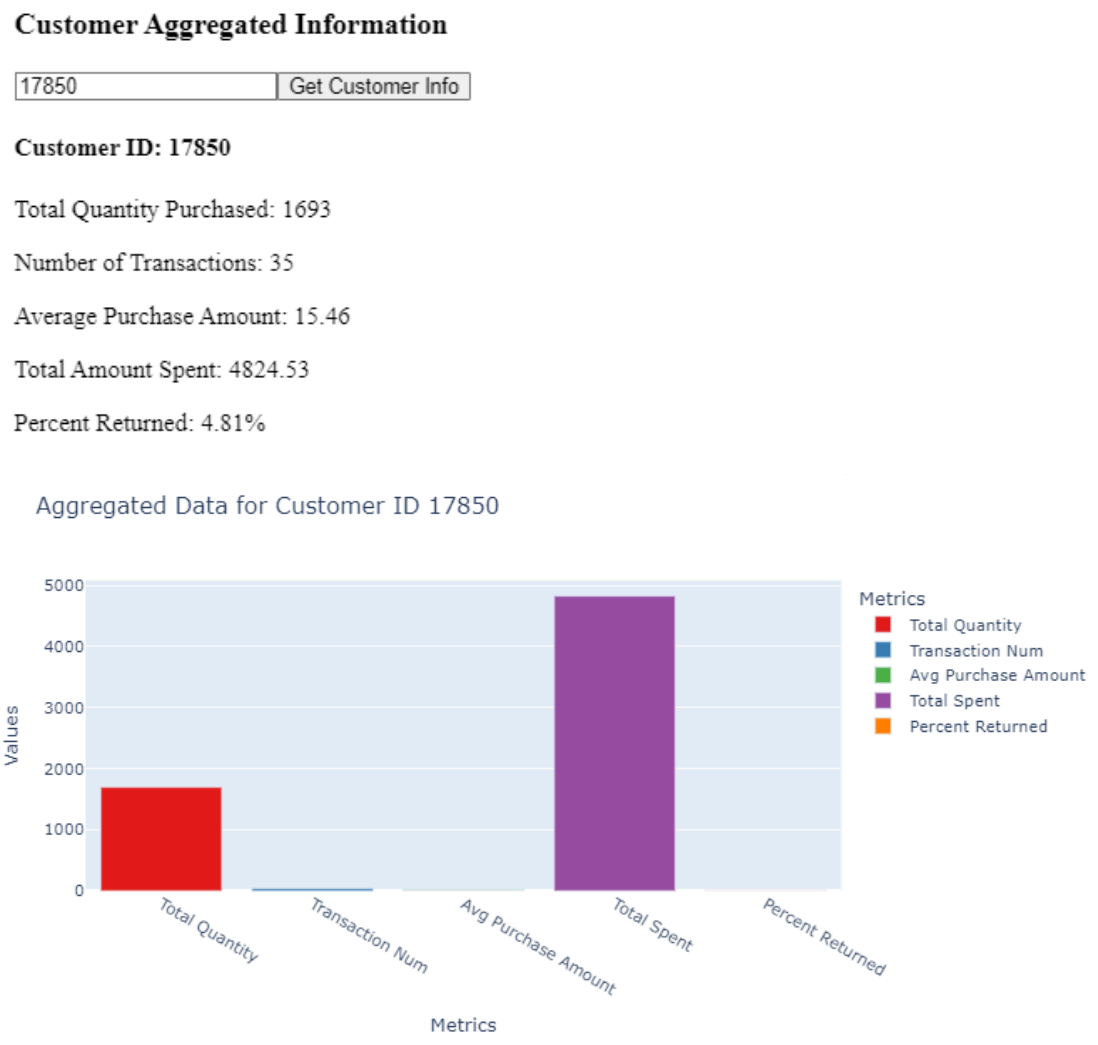
2. **Customer Aggregated Information**

Input: Customer ID

Output: Detailed view of the selected customer's transaction summary. A bar chart is used to display the aggregated data, making it easy to compare each metric at a glance.

Purpose / Application:

- Targeted marketing - guides the marketing team in creating tailored promotions, enhancing the effectiveness of loyalty programs



3. **Top N Customers by Total Spending**

Input: *N* values

Output: Breakdown of the top N customers, ranked by their total spending in a form of bar chart with the respective customer ID.

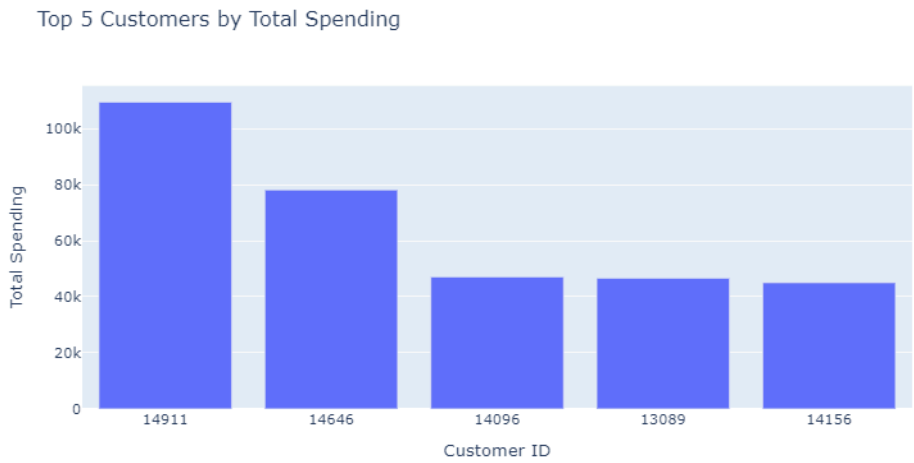
In this example, the dashboard displays the top five (N=5) customers with the highest total spending.

Purpose / Applications:

- Identifying key customers contributing the most to overall sales, highlighting high-value customers.
- Understanding the purchasing behaviour of top customers allows for more accurate CLV estimation.
- Enables focusing efforts on these customers, providing them with personalised deals, loyalty rewards, or premium services to encourage further engagement and loyalty.

Top N Customers by Total Spending

5 | Show Top N Customers



4. **Top N Customers by Total Quantity**

Input: *N* values

Output: Breakdown of the top N customers, ranked by their total quantity (item units) bought in a form of bar chart with the respective customer ID.

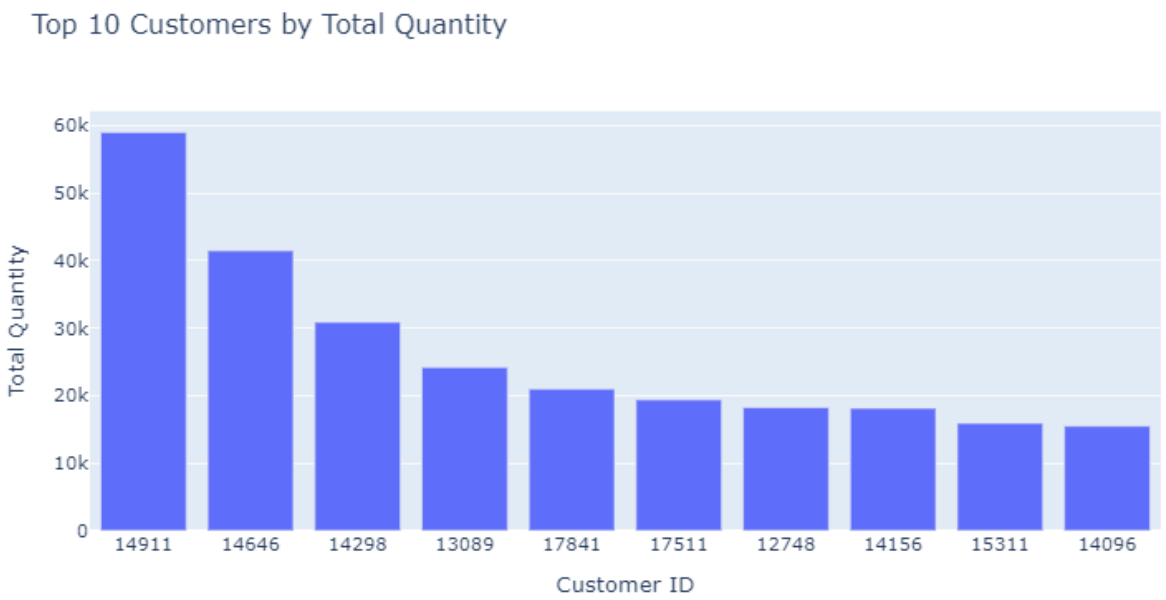
In this example, the dashboard displays the top ten (N=10) customers with the highest total quantity of items bought.

Purpose / Applications:

- Identifying bulk buyers who are purchasing the largest quantities of items regardless of total spending
- Understanding which customers consistently buy in large quantities allows the retail store to optimise their supply chain and distribution processes, as well as align their inventory and stock levels with actual demand.

Top N Customers by Total Quantity

10 | Show Top N Customers



5. **Top N Customers by Total Transaction Number**

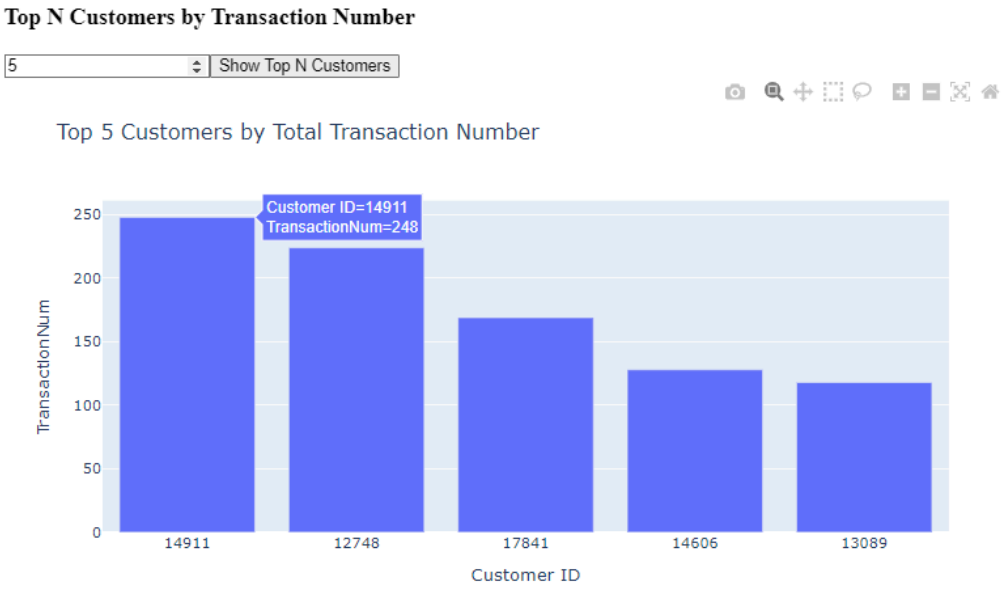
Input: *N* values

Output: Breakdown of the top N customers, ranked by their number of transactions (invoices) they made in a form of bar chart with the respective customer ID.

In this example, the dashboard displays the top five (N=5) customers with the highest transaction count.

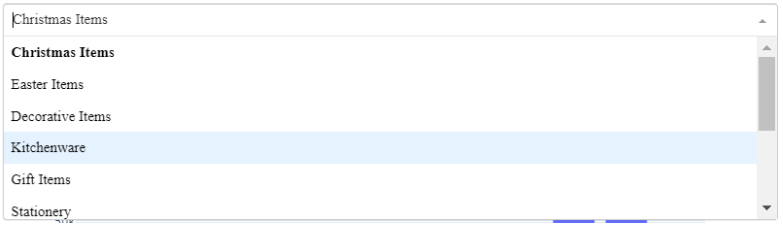
Purpose / Applications:

- Identifying repeat customers who make frequent purchases, meaning that they have strong loyalty and regular buying patterns.
- Offering rewards, exclusive deals, or early access to products for frequent customers can strengthen their relationship with the retail store.



6. Product Category Information for Total Quantity

Input: Product category option

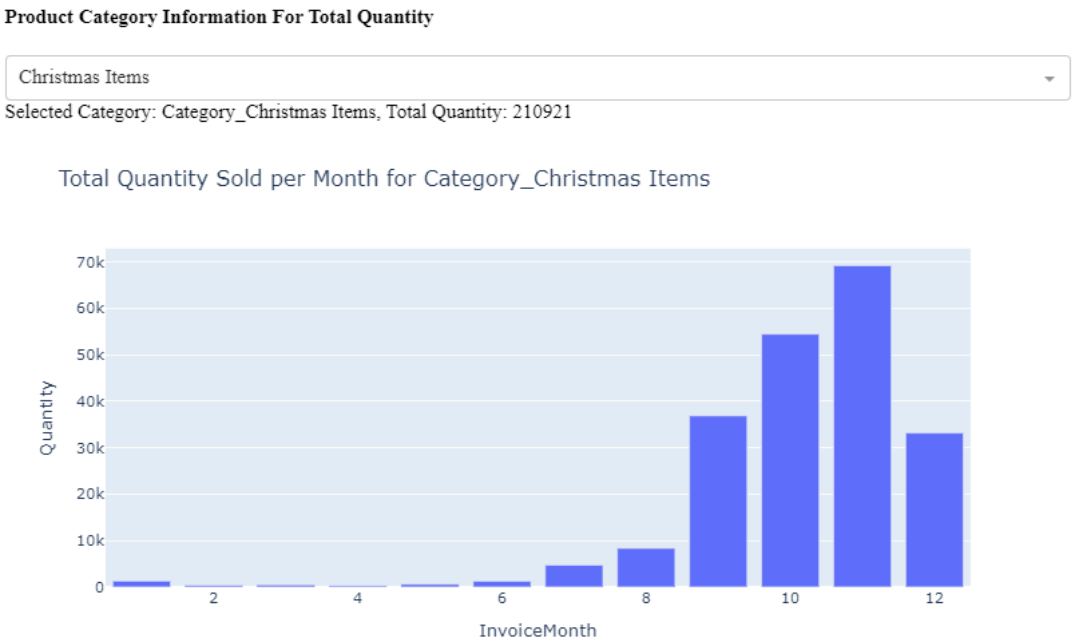


Output: A short statement about the total quantity sold of the selected product category. A bar chart showing the sales trends over a year on a monthly basis.

In this example, the *Chrismas_Items* category was selected and it shows that there are a total of 210921 units sold in the past 12 months and sold the most during the end of the year especially November for Christmas Day preparation.

Purpose / Applications:

- This allows businesses to identify seasonal trends and fluctuations in product category sales.
- By recognizing peak sales months, businesses can plan supply chain logistics more effectively and prevent stock outs or overstocking issues, allowing better inventory management.
- Businesses can plan promotions, advertising, and special deals around periods of high demand (e.g. Christmas Special Deals).



7. **Product Category Information for Total Sales**

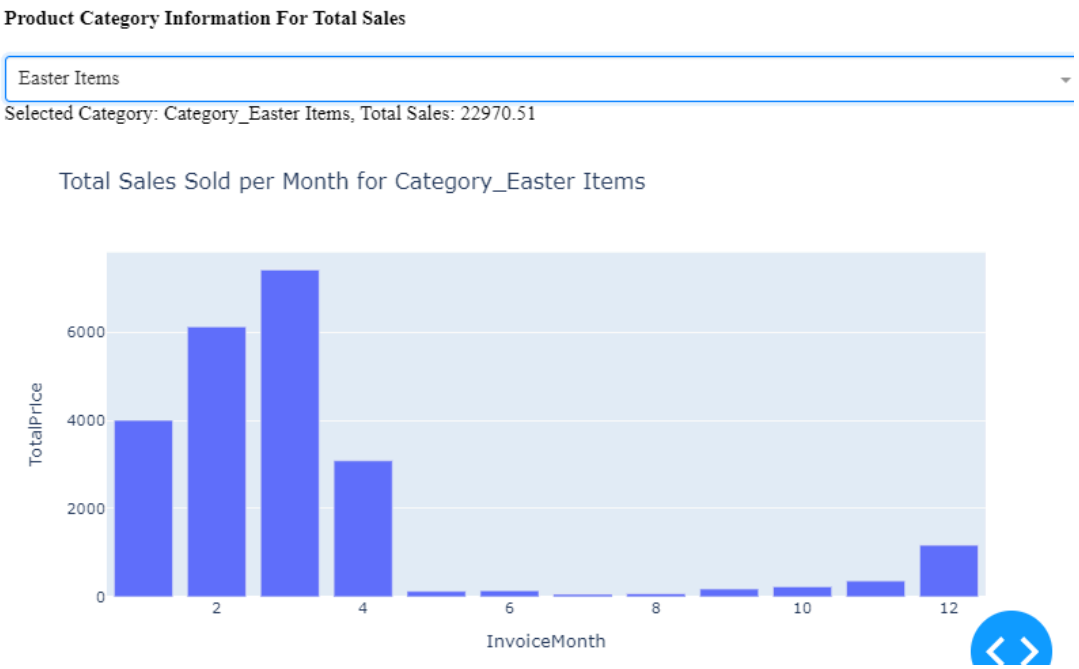
Input: Product category option

Output: A short statement about the total sales generated by the selected product category. A bar chart showing the sales trends over a year on a monthly basis.

In this example, the *Easter_Items* category was selected and it shows that there are a total sales of £22970.51 in the past 12 months and having the highest sales during March around Easter celebration season.

Purpose / Applications:

- Businesses can optimise revenue by analysing which months generate the most sales for each product category.
- Sales trends assist in determining how much budget to allocate for inventory and marketing.
- Businesses can focus their marketing budgets on high-sales months to increase ROI. For example, promoting Easter items more heavily in February and March can help boost sales ahead of Easter.



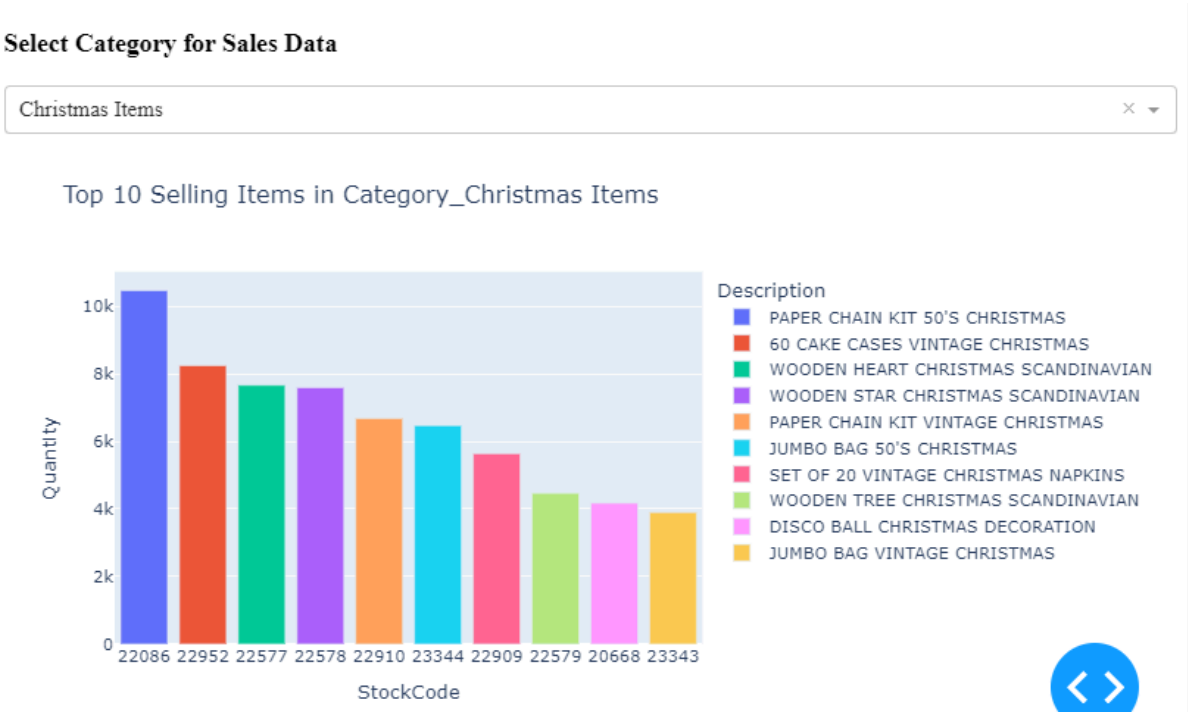
8. **Select Category for Sales Data**

Input: Product category option

Output: Display the top 10 selling items under the selected category in the form of bar chart with respect to their stock code labelled with the product descriptions and total quantity sold.

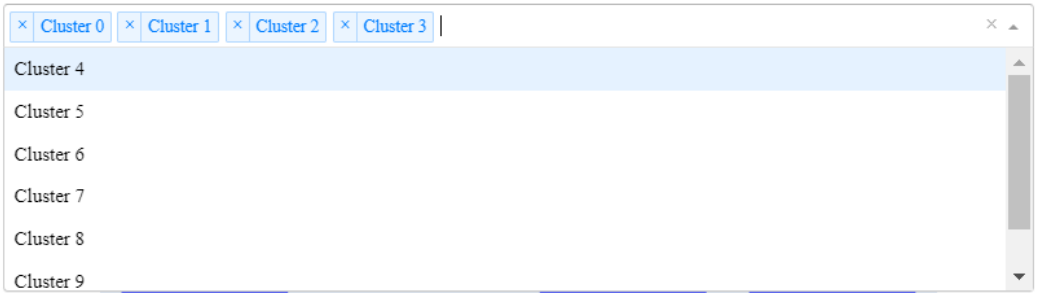
Purpose / Applications:

- Product demand insights - Identifies the most in-demand items within a product category, helping businesses understand customer preferences.
- Businesses can plan stock replenishment more effectively. For example, if certain items under the Christmas_Items category are best-sellers, inventory can be adjusted to avoid stockouts during peak season.
- Automated personalised product recommendations system can suggest popular / hot selling items to customers under specific cluster with respect to the category.



9. Compare Clusters

Input: Multi-value field for cluster selection



Output: Bar chart showing the total spending over the selected clusters

Purpose / Applications:

- Businesses can use this information to identify their most profitable customer segments, enabling better targeting for marketing campaigns and product promotions.
- If certain clusters have higher spending due to purchasing large volumes of particular products, businesses can ensure that inventory stock levels align with the demand trends of it.

10. Select Month For Sales Data

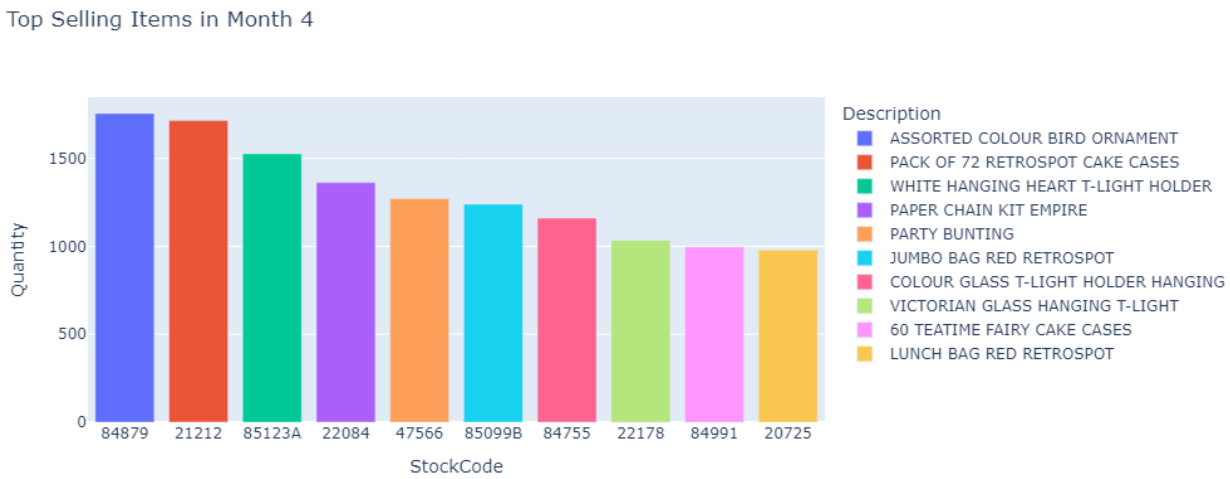
Input: Month option

A. Top Selling Items In Selected Month

Output: The results will display the top 10 selling items for the month selected by the user. For example, if the user selects April, they will see the top selling items for that month.

Purpose/Application:

- The graph provides insights into the sales performance of specific items over a selected month and it allows businesses to assess which products are performing well.
- By identifying top sellers, businesses can optimise inventory levels, ensuring that popular items are well-stocked while minimising excess inventory of slower-moving products.
- Insights from the graph can guide marketing campaigns and allow businesses to focus on promoting top-selling items or related products to enhance sales.

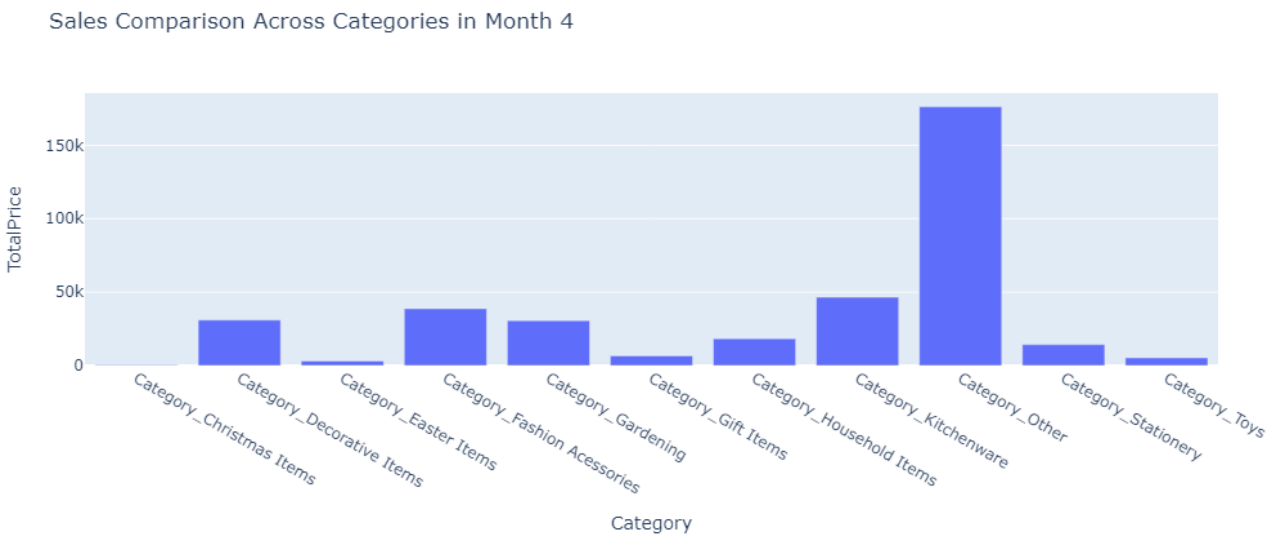


B. Sales Comparison Across Categories in Selected Month

Output: Bar chart showing the total price spent or sales generated over all categories for the specific month selected

Purpose/Application:

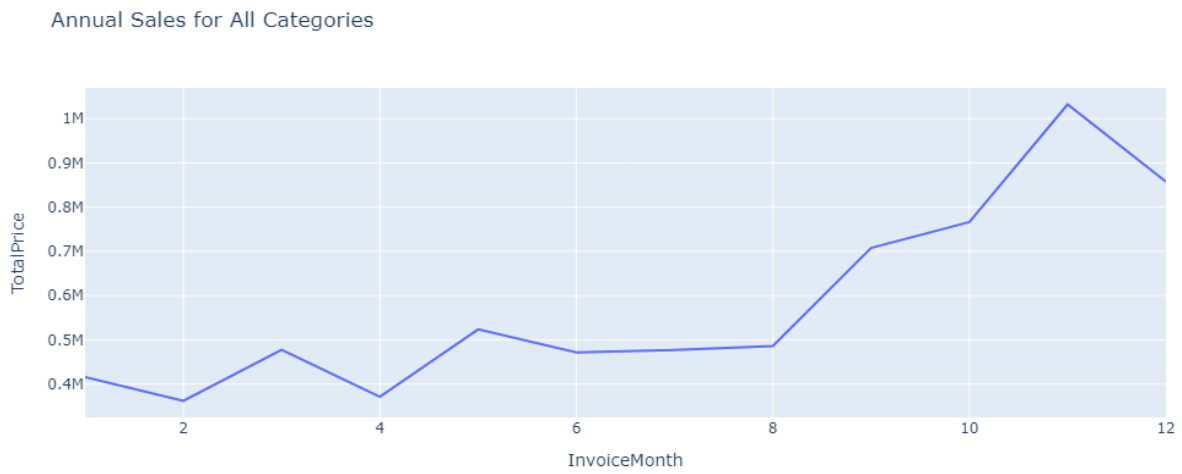
- Provides a clear view of how different product categories performed during a selected month.
- Businesses can adjust inventory orders to align with sales patterns, minimising the risk of overstocking or stock out during a specific month.
- Seasonal analysis to find out which category of product performs better for a certain month that helps in marketing planning and development to boost sales.



C. Annual Sales for All Categories

Output: The line chart will illustrate the annual sales for all item categories

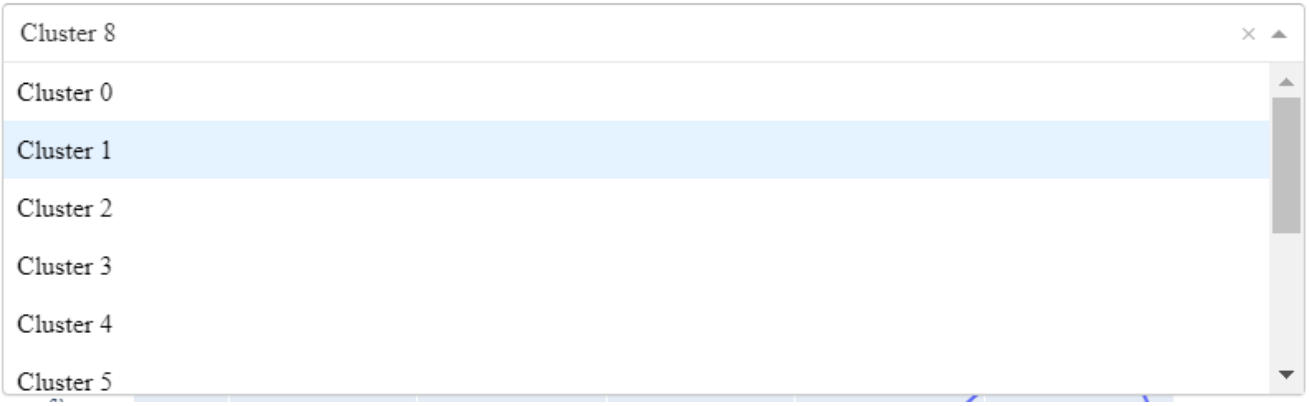
Purpose/Application:



- The graph provides a clear visualisation of the total sales and makes it easier for businesses to track overall revenue generation month by month.
- By understanding which months see higher sales can help businesses design targeted marketing campaigns or promotions during those times to maximise revenue.
- The graph can be included in sales performance reports to provide stakeholders with a visual summary of sales trends over the year.

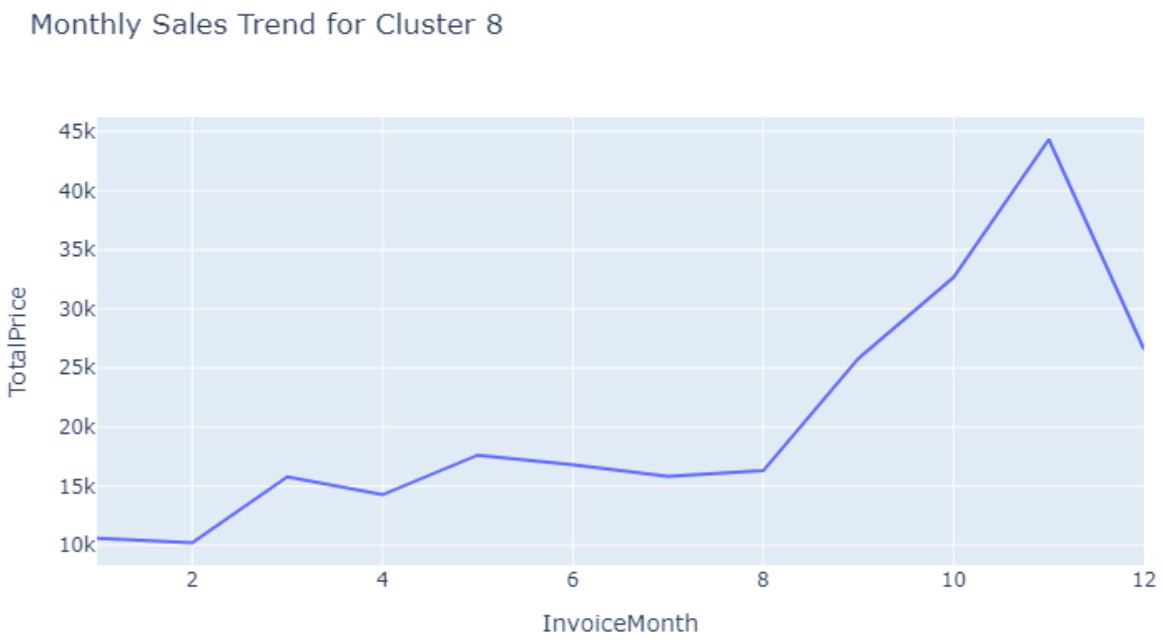
11. **Monthly Sales Trends**

Input: Cluster Option



Output: The graph will show the monthly sales trends for selected clusters by the user.

Purpose/Application:



- The graph helps in monitoring the sales performance of specific customer clusters over time and allowing businesses to track how sales change month by month.
- Understanding the monthly sales trends of selected clusters enables businesses to design targeted marketing campaigns that resonate with specific customer groups based on their purchasing behaviours.
- The graph can reveal clusters that are experiencing significant sales growth, indicating potential areas for further investment and marketing efforts.

H. Conclusion

In this project, we explored the implementation of a clustering model to analyse customer behaviour in an online retail environment. By applying advanced data science techniques, we learned much about the whole data mining project, from business understanding, to data cleaning, data selection, data preprocessing, modelling, as well as evaluation and analysis of results, planning for deployment and dashboard or data visualisation development.

By applying these data science techniques, we successfully segmented the customer base into distinct clusters based on purchasing patterns and preferences.

Advantages of the model (KMeans)

1. **Enhance Customer Insights**
   KMeans effectively segments customers based on purchasing behaviour, enabling the identification of distinct groups. This insight allows the business to tailor marketing strategies to target high-value segments, leading to improved customer retention and sales growth.

2. **Data-Driven Decision Making**
   KMeans enables data-driven decisions to develop and plan various actionable strategies by clustering customers. This approach analyses factors such as purchase frequency, product categories, and spending patterns, which informs inventory management and promotional strategies for optimal stock levels and targeted marketing efforts.

3. **Scalable Analysis**
   KMeans clustering is easy to implement and understand with just a few parameters and steps. It is fast and scalable, which runs efficiently especially on handling large datasets like the transactional data collected over a year in online retail dataset (What Are the Advantages and Disadvantages of Using K-means Clustering for Data Segmentation?, 2023). Its scalability allows for in-depth analysis without compromising speed, making it suitable for real-time applications.

Weaknesses / Limitations of the model

1. **Silhouette score limitations**

   The interpretation of customer segments and their distinctiveness can be further improved as the silhouette score of 0.474 implies that there may be overlap between clusters despite a moderate level of cluster cohesiveness and separation.

2. **Skewed Cluster Distribution**

   The distribution of clusters is uneven, as seen with the largest cluster (Cluster 5) containing 236,533 data points, while smaller clusters (Cluster 4) with only 2,434 points may not adequately represent the customer segments. This skew can affect the reliability of insights derived from the model.

In conclusion, despite its limitations, the KMeans model presents significant advantages for your online retail business. By implementing this algorithm, the retail merchandiser can gain valuable insights into customer segmentation, enabling targeted marketing strategies, enhanced inventory management, and ultimately driving sales growth.

I. References

GeeksforGeeks. (2023, December 9). *Demonstration of KMEANs assumptions*. GeeksforGeeks. https://www.geeksforgeeks.org/demonstration-of-k-means-assumptions/

GeeksforGeeks. (2024, May 24). *Clustering distance measures*. GeeksforGeeks. https://www.geeksforgeeks.org/clustering-distance-measures/

GeeksforGeeks. (2024b, June 2). *ML | Mini Batch Kmeans clustering algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/ml-mini-batch-k-means-clustering-algorithm/

Gültekin, H. (2023, September 7). What is Silhouette Score? - Hazal Gültekin - Medium. *Medium*. https://medium.com/@hazallgultekin/what-is-silhouette-score-f428fb39bf9a

Hazelcast. (2024, May 13). *What is a data pipeline? Considerations & Examples | Hazelcast*. https://hazelcast.com/glossary/data-pipeline/

Involve.Me. (2023, October 2). *How retailers can automate product recommendation Process*. involve.me. https://www.involve.me/blog/how-retailers-can-automate-product-recommendation-process#benefits-of-automating-product-recommendations

Jaadi, Z. (2024, February 23). *Principal Component Analysis (PCA): A Step-by-Step Explanation*. Built In. https://builtin.com/data-science/step-step-explanation-principal-component-analysis

Kumaratunga, S., PhD. (2021, December 14). KMeans Hyper-parameters Explained with Examples - Towards Data Science. *Medium*. https://towardsdatascience.com/kmeans-hyper-parameters-explained-with-examples-c93505820cd3

McGregor, M. (2020, September 21). *8 Clustering Algorithms in Machine Learning that All Data Scientists Should Know*. freeCodeCamp.org. https://www.freecodecamp.org/news/8-clustering-algorithms-in-machine-learning-that-all-data-scientists-should-know/

Nailman, A. (2024, May 22). *Clustering in Data Analysis: key considerations and best practices*. Machine Learning Models. https://machinelearningmodels.org/clustering-in-data-analysis-key-considerations-and-best-practices/

Outlier Detection and Treatment: Methods for Cleaner Data. (2024, January 6). *Data Headhunters*. https://dataheadhunters.com/academy/outlier-detection-and-treatment-methods-for-cleaner-data/

Smart Vision Europe. (2020, June 17). *Crisp DM methodology - Smart Vision Europe*. https://www.sv-europe.com/crisp-dm-methodology/

UCI Machine Learning Repository. (n.d.). https://archive.ics.uci.edu/dataset/352/online+retail

What are the advantages and disadvantages of using k-means clustering for data segmentation? (2023, March 13). www.linkedin.com. https://www.linkedin.com/advice/3/what-advantages-disadvantages-using-k-means#:~:text=K%2Dmeans%20clustering%20is%20a%20widely%20used%20method%20of%20data,efficiently%20and%20parallelized%20or%20distributed.

What is a CRM System? Everything You Need to Know. (n.d.). Salesforce. https://www.salesforce.com/ap/crm/what-is-crm/crm-systems/#:~:text=A%20customer%20relationship%20management%20(CRM)%20system%20helps%20manage%20customer%20data,an%20ecosystem%20of%20bespoke%20apps.

What is Marketing ROI? (n.d.). Oracle. https://www.oracle.com/cx/marketing/marketing-roi/#:~:text=ROI%20is%20short%20for%20return,the%20revenue%20those%20campaigns%20generate.

J. Appendix

Appendix A

Files Used for Data Science Project

The following files were used in the data preparation, modelling, evaluation, deployment, dashboard and data visualisation stages of the project:

A) Original Datasets

Online Retail Dataset

B) Data Preparation, Modelling, Evaluation Part Python File

Assignment Python(Part C - E)

C) Deployment Part Python Files

Dashboard

D) File that are used for to run the dashboard

Clustered_Data

Cluster_Profiles

Customer_agg